UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2026 Oleksandr Mytro

Univerzita Pardubice
Fakulta elektrotechniky a informatiky

Time Capsule - webová aplikace pro odesílání zpráv do budoucnosti

Bakalářská práce

2026 Oleksandr Mytro

Univerzita Pardubice
Fakulta elektrotechniky a informatiky
Akademický rok: 2025/2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	**Oleksandr Mytro**
Osobní číslo:	**I22122**
Studijní program:	**B0688A140009 Informační technologie**
Téma práce:	**Time Capsule – webová aplikace pro odesílání zpráv do budoucnosti**
Zadávající katedra:	**Katedra informačních technologií**

Zásady pro vypracování

Cílem práce je navrhnout a realizovat webovou aplikaci, která umožňuje uživatelům vytvářet a spravovat osobní časové kapsle – zprávy, které bude možné otevřít až v předem stanoveném budoucím datu. Aplikace umožní odesílání zpráv sobě nebo jiným uživatelům s možností přidání textu, obrázků nebo videí. Součástí práce je analýza požadavků, návrh uživatelského rozhraní a návrh datového modelu, který zajistí bezpečné a časově podmíněné zpřístupnění uložených zpráv.

Rozsah pracovní zprávy: **30-40 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná/elektronická**

Seznam doporučené literatury:

SCIORE, Edward. *Database design and implementation*. Hoboken, N.J.: John Wiley, c2009. ISBN 978-0-471-75716-0.
Exploring Modern Web Development — Yakov Fain & Anton Moiseev

Vedoucí bakalářské práce: **Ing. Monika Borkovcová, Ph.D.**
Katedra informačních technologií

Datum zadání bakalářské práce: **12. prosince 2025**
Termín odevzdání bakalářské práce: **15. května 2026**

L.S.

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

V Pardubicích dne 27. února 2026

Prohlašuji:

Práci s názvem Time Capsule jsem vypracoval samostatně a uvedl v ní všechny použité literární a informační zdroje v souladu s právními předpisy, vnitřními předpisy a vnitřními normami Univerzity Pardubice.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2025 Pravidla pro odevzdávání, zpřístupňování závěrečných prací veřejnosti a jejich formální úpravu, bude závěrečná práce zpřístupněna veřejnosti prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 10. 05. 2026

Oleksandr Mytro

PODĚKOVÁNÍ

Chtěl bych moc poděkovat svojí rodině za to, že mě celou dobu podporovala a dala mi vůbec šanci studovat tak daleko od domova. Bez vás bych to prostě nezvládl.

Děkuju taky všem kamarádům, které jsem tady potkal. Byli jste pro mě neskutečnou oporou a hlavně díky vám na tyhle roky nikdy nezapomenu.

A na závěr chci opravdu upřímně poděkovat vedoucí mé práce, paní Ing. Monice Borkovcové, Ph.D. Nebýt její trpělivosti a skvělých rad, asi bych vůbec nevěděl, jak s prací začít, a nikdy by nevznikla.

ANOTACE

Cílem práce je navrhnout a realizovat webovou aplikaci, která umožňuje uživatelům vytvářet a spravovat osobní časové kapsle – zprávy, které bude možné otevřít až v předem stanoveném budoucím datu. Aplikace umožní odesílání zpráv sobě nebo jiným uživatelům s možností přidání textu, obrázků nebo videí. Součástí práce je analýza požadavků, návrh uživatelského rozhraní a návrh datového modelu, který zajistí bezpečné a časově podmíněné zpřístupnění uložených zpráv.

KLÍČOVÁ SLOVA

webová aplikace, softwarová architektura, React, Spring Boot, MongoDB, Docker, REST API, WebSocket, časová kapsle, klient-server.

TITLE

Time Capsule Web Application

ANNOTATION

The aim of this thesis is to design and implement a web application that allows users to create and manage personal time capsules—messages that can be opened only at a predetermined future date. The application enables sending messages to oneself or to other users, with the option to attach text, images, or videos. The thesis includes requirement analysis, user interface design, and data model design to ensure secure and time-conditioned access to the stored messages.

KEYWORDS

web application, software architecture, React, Spring Boot, MongoDB, Docker, REST API, WebSocket, time capsule, client-server.

OBSAH

SEZNAM ILUSTRACÍ A TABULEK 11
SEZNAM VÝPISŮ ZDROJOVÉHO KÓDU 12
SEZNAM ZKRATEK A ZNAČEK 13
TERMINOLOGIE 14
ÚVOD 15
1 Webové aplikace a principy jejich fungování 16
1.1 Klient-serverový model 16
1.2 HTTP komunikace a princip request-response 17
1.3 REST jako styl návrhu webových API 18
1.4 Komunikace v reálném čase pomocí WebSocket 18
2 Architektura webových aplikací 20
2.1 Vrstvená architektura 20
2.2 Monolitická architektura 21
2.3 Mikroslužbová architektura 21
2.4 Modulární monolit 21
2.5 Zhodnocení architektonických přístupů 21
3 Technologie používané při vývoji moderních webových aplikací 23
3.1 Backendové technologie 23
3.2 Frontendové technologie 24
3.3 Databázové technologie 26
3.4 Kontejnerizace a nasazení 27
3.5 Bezpečnost webových aplikací 27
4 Přístupy k návrhu uživatelského rozhraní 28
4.1 MVC 28
4.2 MVP a MVVM 29
4.3 MVI a jednosměrný tok dat 30
4.4 Srovnání a zhodnocení architektonických vzorů 31

5 Analýza systému Time Capsule 32
5.1 Charakteristika systému 32
5.2 Analýza předmětné oblasti 32
5.3 Analýza hlavních funkcí systému 33
5.4 Uživatelské role a scénáře použití 34
5.5 Funkční požadavky 35
5.6 Nefunkční požadavky 36
5.7 Omezení a specifika navrhovaného řešení 37
5.8 Modelování procesů systému 37
5.8.1 Hlavní navigační tok systému 38
5.8.2 Proces vytvoření časové kapsle 38
5.8.3 Proces prohlížení a interakce s obsahem 39
6 Návrh systému Time Capsule 40
6.1 Návrh celkové architektury systému 40
6.2 Návrh komunikace mezi jednotlivými komponentami 40
6.3 Návrh doménového modelu 41
6.4 Životní cyklus kapsle 42
6.5 Návrh databázového řešení 43
6.6 Návrh bezpečnostního řešení 44
6.7 Návrh notifikační a časové logiky 45
6.8 Zdůvodnění výsledného návrhu 46
7 Implementace 47
7.1 Implementace backendové části 47
7.1.1 Struktura backendové části 47
7.1.2 Použité závislosti a frameworky 47
7.1.3 REST API a kontrolery 48
7.1.4 Servisní vrstva a business logika 49
7.1.5 Přístup k databázi 50
7.1.6 Real-time komunikace pomocí WebSocket a STOMP 51
7.1.7 Scheduler a automatické otevírání kapslí 52

7.1.8 Bezpečnost a autorizace 54
7.2 Implementace frontendové části 56
7.2.1 Struktura frontendové části 57
7.2.2 Komponenty a routing 57
7.2.3 Správa stavu a práce s hooky 58
7.2.4 Formuláře a práce s daty 59
7.2.5 Integrace mapy 61
7.2.6 Notifikace a WebSocket klient 62
7.2.7 Stylování uživatelského rozhraní 64
7.3 Implementace databázové vrstvy 65
7.4 Infrastruktura a nasazení 66
7.4.1 Docker a Docker Compose 66
7.4.2 Konfigurace Nginx 67
7.4.3 Propojení služeb a sítě 67
7.4.4 Lokální a produkční spuštění 68
7.5 Testování backendové části 68
7.5.1 Spuštění aplikace a testování webové vrstvy 68
7.5.2 Testování API pro časové kapsle 68
7.5.3 Testování servisní logiky kapslí 69
7.5.4 Testování autentizačních cookies 69
7.5.5 Testování chatu a auditního logování 70
7.5.6 Zhodnocení testování 70
ZÁVĚR 71
POUŽITÁ LITERATURA 72
SEZNAM PŘÍLOH 75
Příloha A: Obsah přiloženého ZIP archivu a struktura zdrojových kódů 76
Příloha B: UML sekvenční diagram automatického otevření kapsle 78
Příloha C: ER diagram datového modelu MongoDB 79
Příloha D: UML diagram nasazení aplikace 80

SEZNAM ILUSTRACÍ A TABULEK

Obrázek 1 Základní schéma klient-serverového modelu ve webové aplikaci Zdroj: (vlastní zpracování podle [1]) 16
Obrázek 2 Princip request-response v HTTP komunikaci Zdroj: (vlastní zpracování) 17
Obrázek 3 Princip obousměrné komunikace pomocí WebSocket Zdroj: (vlastní zpracování) 19
Obrázek 4 Příklad vrstvené architektury webové aplikace Zdroj: (vlastní zpracování podle [8]) 20
Obrázek 5 Tok komunikace mezi komponentami architektury MVC Zdroj: (vlastní zpracování) 28
Obrázek 6 MVP architektura Zdroj: (vlastní zpracování) 29
Obrázek 7 MVVM architektura Zdroj: (vlastní zpracování) 30
Obrázek 8 MVI architektura Zdroj: (vlastní zpracování) 30
Obrázek 9 Hlavní funkční oblasti systému Time Capsule Zdroj: (vlastní zpracování) 33
Obrázek 10 UML diagram případů užití systému Time Capsule Zdroj: (vlastní zpracování) .35
Obrázek 11 UML diagram aktivit – Hlavní navigační tok systému Zdroj: (vlastní zpracování) 38
Obrázek 12 UML diagram aktivit – Proces vytvoření časové kapsle Zdroj: (vlastní zpracování) 39
Obrázek 13 UML diagram aktivit – Proces prohlížení a interakce Zdroj: (vlastní zpracování) 39
Obrázek 14 Stavový diagram životního cyklu kapsle Zdroj: (vlastní zpracování) 42
Obrázek 15 Architektura MongoDB sharded clusteru Zdroj: (vlastní zpracování) 43
Obrázek 16 Hlavní stránka (Landing page) aplikace představující základní funkce Zdroj: (vlastní zpracování) 56
Obrázek 17 Komplexní formulář pro vytvoření nové časové kapsle s nastavením viditelnosti a času odemčení Zdroj: (vlastní zpracování) 59
Obrázek 18 Uživatelský profil s interaktivním 3D glóbem a zobrazením lokace vytvořených kapslí Zdroj: (vlastní zpracování) 62
Obrázek 19 Rozhraní modulu pro real-time chat a soukromé zprávy mezi uživateli Zdroj: (vlastní zpracování) 63
Obrázek 20 UML Sekvenční diagram – Asynchronní proces automatického otevření kapsle s ochranou proti souběhu Zdroj: (vlastní zpracování s využitím umělé inteligence) 78
Obrázek 21 Entity-Relationship (ER) diagram datového modelu MongoDB Zdroj: (vlastní zpracování s využitím softwarového nástroje pro návrh databází) 79
Obrázek 22 UML diagram nasazení a síťové architektury v prostředí Docker Zdroj: (vlastní zpracování s využitím umělé inteligence) 80

Tabulka 1 Srovnání vybraných architektonických přístupů 22
Tabulka 2 Srovnání vybraných backendových technologických stacků 24
Tabulka 3 Srovnání vybraných frontendových technologií 25
Tabulka 4 Srovnání vybraných databázových přístupů 26
Tabulka 5 Srovnání vybraných architektonických vzorů 31

SEZNAM VÝPISŮ ZDROJOVÉHO KÓDU

Výpis 1 Implementace koncových bodů ve třídě CapsuleController 48
Výpis 2 Metoda pro vytvoření nové časové kapsle 49
Výpis 3 Atomické odemčení časové kapsle pomocí MongoTemplate 50
Výpis 4 Rozhraní CapsuleRepository pro práci s kapslemi 50
Výpis 5 Konfigurace WebSocket komunikace ve třídě WebSocketConfig 51
Výpis 6 Odesílání real-time událostí pomocí SimpMessagingTemplate 52
Výpis 7 Doručení chatové zprávy příjemci pomocí WebSocket 52
Výpis 8 Dávkové automatické otevírání kapslí v plánovači 53
Výpis 9 Konfigurace zabezpečení REST API pomocí SecurityFilterChain 54
Výpis 10 Zpracování OAuth2 přihlášení a vytvoření autentizačních tokenů 55
Výpis 11 Nastavení bezpečnostních atributů autentizační cookie 56
Výpis 12 Konfigurace routingu a lazy loadingu ve frontendové části 57
Výpis 13 Komponenta MainLayout pro společné rozvržení aplikace 58
Výpis 14 Správa stavu uživatelské relace pomocí hooků useState a useEffect 58
Výpis 15 Zpracování formuláře pro vytvoření časové kapsle 60
Výpis 16 Dynamické načítání 3D glóbu pomocí knihovny Cesium 61
Výpis 17 Připojení klienta k WebSocket streamům notifikací a chatu 62
Výpis 18 Definice motivu a CSS proměnných v Tailwind CSS 64
Výpis 19 Stylování komponenty pomocí utility tříd Tailwind CSS 64
Výpis 20 Mapování entity Capsule na dokument MongoDB 65
Výpis 21 Deklarativní dotazy v rozhraní CapsuleRepository 66
Výpis 22 Atomická aktualizace stavu kapsle pomocí MongoTemplate 66
Výpis 23 Postup lokálního spuštění databázové a aplikační vrstvy 68
Výpis 24 Test validace vstupu při úpravě kapsle 69
Výpis 25 Test zákazu úpravy cizí kapsle běžným uživatelem 69
Výpis 26 Test bezpečnostních atributů autentizačních cookies 69
Výpis 27 Test odmítnutí prázdné chatové zprávy 70

SEZNAM ZKRATEK A ZNAČEK

ACID – Atomicity, Consistency, Isolation, Durability (Atomicita, konzistence, izolovanost, trvalost)

API – Application Programming Interface

BSON – Binary JSON

CRUD – Create, Read, Update, Delete

CSRF – Cross-Site Request Forgery

CSS – Cascading Style Sheets

DTO – Data Transfer Object

ER – Entity-Relationship

HTML – HyperText Markup Language

HTTP – Hypertext Transfer Protocol

HTTPS – Hypertext Transfer Protocol Secure

ID – Identifier (Identifikátor)

JAR – Java ARchive

JRE – Java Runtime Environment

JSON – JavaScript Object Notation

JWT – JSON Web Token

MVC – Model-View-Controller

MVI – Model-View-Intent

MVP – Model-View-Presenter

MVVM – Model-View-ViewModel

REST – Representational State Transfer

SMTP – Simple Mail Transfer Protocol

SPA – Single Page Application

STOMP – Simple Text Oriented Messaging Protocol

TLS – Transport Layer Security

UML – Unified Modeling Language

URI – Uniform Resource Identifier

URL – Uniform Resource Locator

XSS – Cross-Site Scripting

TERMINOLOGIE

Autentizace – proces ověření identity uživatele v systému

Autorizace – proces, který určuje, zda má daný, již autentizovaný uživatel, oprávnění provést požadovanou akci nebo přistoupit k datům

Bezstavovost (**Statelessness**) – princip komunikace, při kterém každý požadavek obsahuje všechny informace potřebné ke zpracování, takže server nemusí uchovávat stav předchozí komunikace

Časová kapsle (**Capsule**) – stěžejní datový objekt systému s vlastním stavem a časovým chováním, který uchovává uživatelský obsah (text, média) a plně se zpřístupní až po dosažení nastaveného času odemčení

Event-driven (**architektura řízená událostmi**) – přístup, kdy dosažení určitého stavu (např. času odemčení kapsle) představuje událost, která automaticky spouští další procesy

Kontejner – izolovaný proces se všemi soubory a závislostmi, které potřebuje ke spuštění. Díky této izolaci se snižuje závislost aplikace na konkrétním nastavení hostitelského systému.

Lazy loading – technika optimalizace výkonu spočívající v dynamickém načítání datově náročných komponent (např. 3D glóbus Cesium) až v momentě jejich skutečné potřeby

Polyglot persistence – přístup k návrhu datové vrstvy, který vychází z použití různých databázových technologií pro různé druhy dat v rámci jednoho systému

Reverzní proxy – **server** (např. Nginx) fungující jako vstupní bod do systému, který typicky zajišťuje TLS terminaci a směrování požadavků na interní služby

Sharding (**Sharded cluster**) – metoda horizontálního škálování databáze (např. MongoDB), při níž jsou data distribuována přes více uzlů (shardů) pro zachování výkonu a vysoké dostupnosti

Single Page Application (**SPA**) – moderní klientská webová aplikace, ve které veškerá navigace a překreslování obrazovek probíhá asynchronně na straně klienta bez nutnosti znovunačítání celé stránky

Soft delete – databázový ochranný mechanismus, který zabraňuje nevratné fyzické ztrátě dat tím, že je při smazání pouze označí příznakem (např. deletedAt)

WebSocket – technologie umožňující trvalou obousměrnou komunikaci mezi klientem a serverem v reálném čase, využívaná pro chat a okamžité notifikace

ÚVOD

Lidé si dnes ukládají velkou část osobních vzpomínek, zpráv a fotografií v digitální podobě. Takový obsah ale často zůstává rozptýlený mezi sociálními sítěmi, chatovacími aplikacemi, cloudovými úložišti nebo běžnými složkami v počítači. Uživatel se k němu může vrátit kdykoliv, ale většinou bez jasného časového kontextu. Právě práce s časem je přitom u osobních záznamů důležitá. Některé zprávy nebo vzpomínky mají větší hodnotu tehdy, když se k nim člověk vrátí až po určité době.

Na této myšlence je založena webová aplikace Time Capsule. Jejím hlavním účelem je umožnit uživatelům vytvořit digitální časovou kapsli, uložit do ní textový nebo multimediální obsah a nastavit datum, od kterého bude možné daný obsah otevřít. Aplikace tím spojuje běžnou správu uživatelských dat s časově řízeným zpřístupněním obsahu. Uživatel tak nemusí ukládat pouze okamžitě dostupné záznamy, ale může pracovat i s obsahem určeným pro budoucnost.

Téma práce jsem zvolil proto, že na jednom projektu spojuje více částí běžného vývoje webové aplikace. Nejde pouze o vytvoření uživatelského rozhraní nebo jednoduché ukládání dat. Aplikace musí řešit registraci a přihlašování uživatelů, práci s osobním obsahem, pravidla viditelnosti, ukládání mediálních souborů, automatické otevírání kapslí v nastaveném čase a také upozornění v reálném čase. Projekt proto ukazuje celý postup vývoje webového systému: analýzu, návrh, implementaci i spuštění v kontejnerizovaném prostředí.

Cílem této bakalářské práce je navrhnout a vytvořit webovou aplikaci Time Capsule, která bude podporovat vytváření, správu a časově podmíněné otevírání digitálních kapslí. Součástí cíle je navrhnout vhodnou architekturu aplikace, datový model, základní bezpečnostní pravidla a způsob komunikace mezi klientskou a serverovou částí. Práce se zaměřuje také na praktickou implementaci backendu, frontendu, databázové vrstvy a infrastruktury potřebné pro lokální i produkční nasazení.

Práce nejprve shrnuje principy webových aplikací, komunikace mezi klientem a serverem, architektonické přístupy a technologie použité při vývoji. Na teoretickou část navazuje analýza systému Time Capsule, která popisuje uživatelské role, hlavní funkce a požadavky na aplikaci.

Následující část se zaměřuje na návrh architektury, datového modelu, bezpečnostních pravidel a časové logiky kapslí. Praktická část potom popisuje implementaci backendu ve Spring Bootu, frontendu v Reactu, práci s databází MongoDB, nasazení pomocí Dockeru a ověření backendové části pomocí automatizovaných testů.

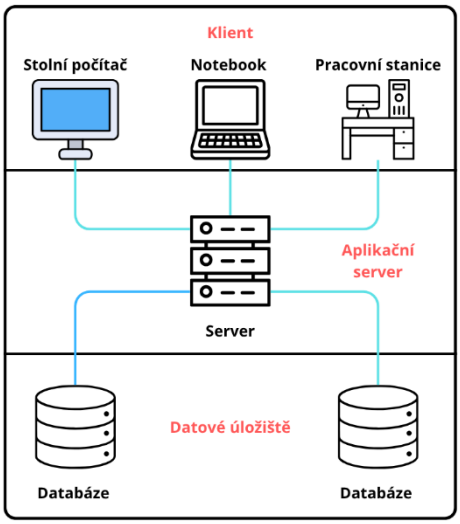
1 Webové aplikace a principy jejich fungování

1.1 Klient-serverový model

Klient-serverový model patří k základním principům fungování webových aplikací. V tomto modelu spolu komunikují dvě části systému, klient a server.

Klientem je aplikace, kterou používá uživatel. Ve webovém prostředí je to nejčastěji webový prohlížeč. Server přijímá požadavky od klienta, zpracovává je a vrací odpověď [2].

Základní princip tohoto uspořádání ukazuje obrázek 1.



Obrázek 1 Základní schéma klient-serverového modelu ve webové aplikaci
Zdroj: (vlastní zpracování podle [1])

Jak uvádí dokumentace MDN, komunikace mezi webovým prohlížečem a serverem probíhá nejčastěji pomocí protokolu HTTP [2]. Prohlížeč odesílá požadavky například při načtení stránky, kliknutí na odkaz nebo odeslání formuláře. Server požadavek zpracuje a vrátí odpověď, která může obsahovat HTML dokument, data nebo jiný typ obsahu [2].

V tomto modelu má klient a server odlišnou roli. Klient zajišťuje zobrazení obsahu a interakci s uživatelem. Server zpracovává požadavky a podle potřeby pracuje s aplikační logikou, databází nebo dalšími službami [2].

Při načtení jedné webové stránky prohlížeč obvykle neodesílá jen jeden požadavek. Po získání HTML dokumentu si vyžádá také další zdroje, například soubory CSS, JavaScript nebo obrázky [2]. Komunikace mezi klientem a serverem proto probíhá jako více navazujících požadavků a odpovědí.

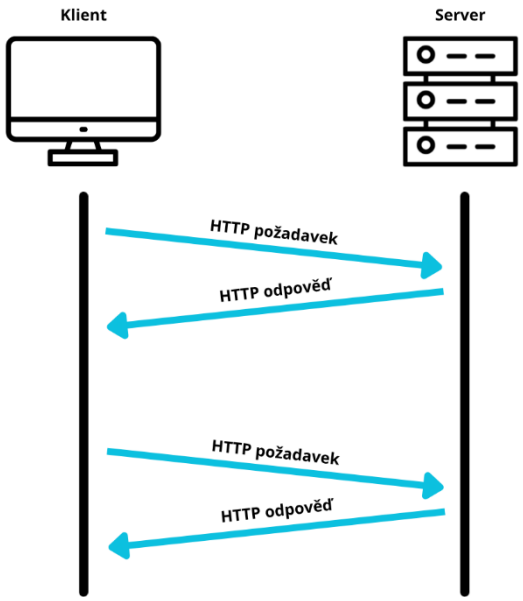
Klient-serverový model tvoří základ běžného fungování webových aplikací. Podrobnější pravidla výměny zpráv, struktura požadavků a odpovědí i způsob zpracování chyb jsou dány protokolem HTTP [2][3].

1.2 HTTP komunikace a princip request-response

HTTP je základní protokol používaný pro komunikaci ve webových aplikacích. Umožňuje výměnu dat mezi klientem a serverem prostřednictvím požadavků (request) a odpovědí (response) [3].

Komunikace probíhá tak, že klient odešle požadavek, server jej zpracuje a vrátí odpověď. Tento způsob výměny zpráv se označuje jako request-response a tvoří základ většiny operací ve webových aplikacích.

Tento princip je znázorněn na obrázku 2.



Obrázek 2 Princip request-response v HTTP komunikaci
Zdroj: (vlastní zpracování)

Jak uvádí dokumentace MDN, HTTP zprávy mají definovanou strukturu [3]. Obsahují úvodní řádek, hlavičky, prázdný řádek a podle potřeby také tělo zprávy. Požadavek obvykle obsahuje adresu zdroje, použitou metodu, například GET nebo POST, a další metadata. Odpověď serveru obsahuje stavový kód a případně i přenášená data [3].

HTTP komunikace probíhá jako reakce na akci klienta. Prohlížeč odesílá požadavky například při načtení stránky, kliknutí na odkaz nebo odeslání formuláře [2][3]. Server na tyto požadavky odpovídá a klient následně zpracovává přijatá data.

Důležitou vlastností HTTP je bezstavovost. Každý požadavek se vyhodnocuje samostatně a server si sám od sebe neuchovává informace o předchozím požadavku. Potřebné údaje proto musí být součástí konkrétní komunikace [3]. Tento princip usnadňuje návrh a škálování systému.

Model request-response se používá při běžných operacích nad daty, například při načítání, vytváření nebo úpravě záznamů. Uplatňuje se také při práci s formuláři a při komunikaci s webovým API. Méně vhodný je tam, kde aplikace musí reagovat na změny okamžitě a bez nového požadavku ze strany klienta.

1.3 REST jako styl návrhu webových API

Na HTTP komunikaci ve webových aplikacích často navazuje architektonický styl REST (Representational State Transfer). Jak uvádí Fielding, nejde o konkrétní technologii, ale o soubor principů pro návrh rozhraní v distribuovaných systémech [4].

REST pracuje se zdroji, které jsou identifikovatelné pomocí URI. Server neposkytuje data ve své interní podobě, ale jejich reprezentaci, například ve formátu JSON nebo HTML [4]. Klient pak s touto reprezentací dále pracuje.

V praxi to znamená, že jednotlivé části systému, například uživatelé nebo příspěvky, jsou chápány jako zdroje. Klient k nim přistupuje pomocí standardních HTTP metod. Metoda GET slouží ke čtení dat, POST k vytvoření nového záznamu, PUT nebo PATCH k úpravě a DELETE k odstranění záznamu.

Důležitou vlastností REST je bezstavovost. Každý požadavek musí obsahovat všechny informace potřebné ke zpracování, takže server nemusí uchovávat stav předchozí komunikace [3][4]. Tento princip podporuje škálování systému a zjednodušuje komunikaci mezi klientem a serverem.

REST je při návrhu webových API vhodný hlavně tam, kde lze jednotlivé části systému popsat jako samostatné zdroje [4]. Klient k těmto zdrojům přistupuje pomocí standardních HTTP metod, například GET, POST, PUT nebo DELETE. Takové rozhraní se hodí pro běžné operace nad daty, jako je načítání, vytváření nebo úprava záznamů.

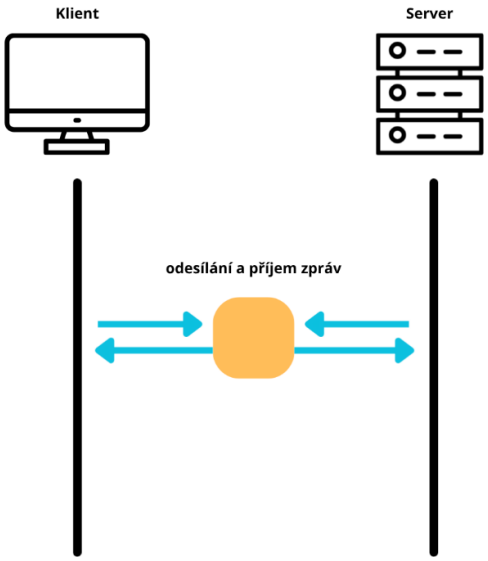
Pro některé části aplikace však samotný REST nestačí. To platí hlavně pro situace, kdy má server předat klientovi informaci okamžitě, bez čekání na další požadavek. V takových případech je vhodnější použít technologii určenou pro komunikaci v reálném čase.

1.4 Komunikace v reálném čase pomocí WebSocket

V některých aplikacích klasický model request-response nestačí. Někdy je potřeba, aby server předal klientovi informaci o změně ihned a bez opakovaného odesílání nových požadavků. Typickým příkladem jsou chatové aplikace, živé notifikace nebo průběžné aktualizace dat.

Tento způsob komunikace řeší technologie WebSocket. Jak uvádí RFC 6455, WebSocket umožňuje obousměrnou komunikaci mezi klientem a serverem v rámci jednoho spojení [5].

Princip této komunikace ukazuje obrázek 3.



Obrázek 3 Princip obousměrné komunikace pomocí WebSocket
Zdroj: (vlastní zpracování)

Na rozdíl od běžného HTTP modelu mohou po navázání spojení odesílat zprávy obě strany kdykoliv [3][5]. Spojení se nejprve inicializuje pomocí HTTP požadavku a poté se přepne na protokol WebSocket.

Po navázání spojení zůstává komunikace otevřená a není nutné vytvářet nové spojení pro každou zprávu. Uživatel tak změnu vidí bez nutnosti obnovit stránku nebo znovu odeslat požadavek. Typickým příkladem je chat, kde se nová zpráva zobrazí ihned po doručení.

WebSocket se proto často používá společně s REST API. Běžné operace, například načítání nebo ukládání dat, probíhají přes HTTP. WebSocket se využívá pro události, které mají být uživateli doručeny okamžitě, například pro chatové zprávy nebo notifikace [3][5].

2 Architektura webových aplikací

Architektura webové aplikace popisuje rozdělení systému na části a způsob jejich vzájemné komunikace. Jak uvádí Microsoft Learn, volba architektury ovlivňuje vlastnosti systému, například jeho přehlednost, rozšiřitelnost a náročnost údržby [6].

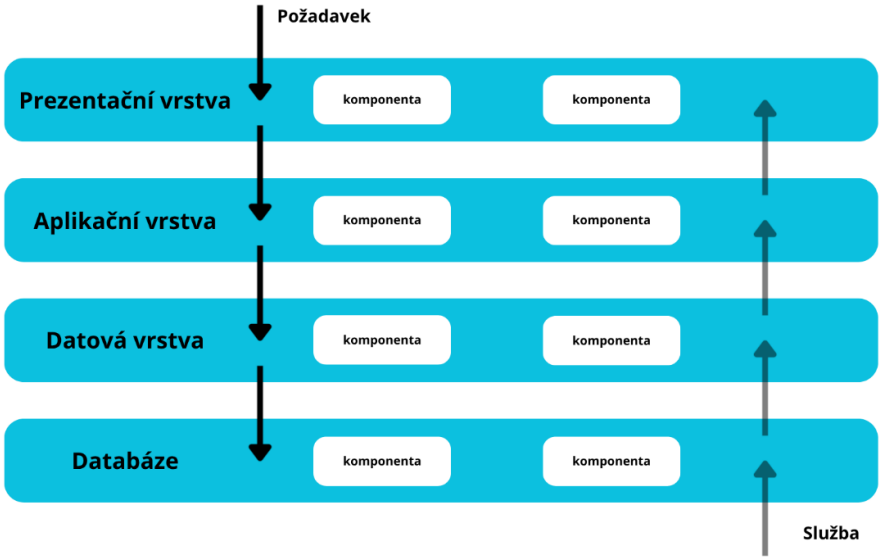
Při návrhu je potřeba jasně určit, která část systému za co odpovídá. Obvykle se odděluje prezentační vrstva, aplikační logika a práce s daty. Pokud se tyto části začnou prolínat bez jasných hranic, systém se hůře upravuje a rozšiřuje.

Ve webových aplikacích se používá více architektonických přístupů. Patří mezi ně vrstvená architektura, monolit, mikroslužby a modulární monolit [6][9][10][11]. Každý z těchto přístupů má odlišné vlastnosti. Vhodná volba závisí hlavně na rozsahu aplikace, jejích požadavcích a očekávaném rozvoji systému.

2.1 Vrstvená architektura

Vrstvená architektura patří mezi běžné způsoby návrhu webové aplikace. Systém je v tomto případě rozdělen do několika vrstev a každá z nich plní jinou roli [7].

Základní schéma tohoto přístupu ukazuje obrázek 4.



Obrázek 4 Příklad vrstvené architektury webové aplikace
Zdroj: (vlastní zpracování podle [8])

Jak uvádí dokumentace Microsoft Learn, vrstvená architektura obvykle rozlišuje prezentační vrstvu, aplikační vrstvu a datovou vrstvu [7]. Každá vrstva zajišťuje jinou část funkcionality a komunikuje pouze s vybranými částmi systému. Aplikace je díky oddělení vrstev čitelnější a jednodušeji se upravuje.

Výhodou tohoto přístupu je srozumitelnější organizace systému. Pokud jsou vrstvy oddělené správně, je snazší určit, kde se nachází konkrétní logika a jak spolu jednotlivé části souvisejí. To usnadňuje vývoj, testování i pozdější údržbu.

Vrstvená architektura ale vyžaduje dodržování jasných hranic mezi vrstvami. Pokud například prezentační vrstva začne komunikovat přímo s databází a obchází aplikační logiku, výhody tohoto přístupu se ztrácejí. Nestačí tedy systém rozdělit na vrstvy jen formálně. Důležité je zachovat jejich oddělení i při implementaci.

2.2 Monolitická architektura

Monolitická architektura chápe aplikaci jako jeden celek, který se vyvíjí, nasazuje a provozuje společně. Jak uvádí Fowler, monolit může být vhodným výchozím řešením, zejména na začátku vývoje systému [9].

Tento přístup zjednodušuje vývoj i nasazení. Tým pracuje s jednou aplikací a změny se obvykle provádějí v rámci jednoho projektu.

Nevýhody se projeví hlavně ve chvíli, kdy aplikace začne růst. Pokud není dobře strukturovaná, zvyšuje se provázanost jednotlivých částí. Změna v jedné oblasti pak může ovlivnit i další části systému. To ztěžuje údržbu i další rozvoj.

Monolitická architektura proto není nevhodná sama o sobě. U menších a středně velkých systémů může jít o rozumnou volbu, pokud má aplikace jasnou vnitřní strukturu [9].

2.3 Mikroslužbová architektura

Mikroslužbová architektura rozděluje systém na menší samostatné služby. Microsoft Learn ji popisuje jako kolekci malých autonomních služeb, které implementují jednotlivé části business logiky a mohou být nasazovány nezávisle [10].

Každá služba zajišťuje konkrétní část systému a lze ji vyvíjet, nasazovat nebo škálovat samostatně. Tento způsob návrhu se hodí hlavně u větších aplikací, kde jednotlivé části rostou různým tempem nebo mají rozdílné nároky na výkon.

Mikroslužby ale zároveň zvyšují složitost celého řešení. Je nutné navrhnout komunikaci mezi službami, monitoring, konfiguraci i způsob nasazení. U menších aplikací proto mohou přinést více režie než užitku [10].

2.4 Modulární monolit

Modulární monolit zachovává aplikaci jako jeden celek, ale uvnitř ji rozděluje na samostatné moduly. Simon Brown tento přístup spojuje s dobře vymezenými komponentami a jasně definovanými rozhraními uvnitř jedné aplikace [11].

Každý modul se zaměřuje na konkrétní část systému a komunikuje s ostatními přes určené rozhraní. To pomáhá omezit zbytečnou provázanost mezi jednotlivými částmi aplikace [11].

Výhodou tohoto přístupu je jednodušší nasazení než u mikroslužeb a zároveň lepší vnitřní struktura než u běžného monolitu. Aplikace tak může zůstat přehledná i při větším rozsahu.

Modulární monolit se hodí tam, kde je potřeba udržet systém dobře uspořádaný, ale není nutné zavádět distribuovanou architekturu [11].

2.5 Zhodnocení architektonických přístupů

Přehled hlavních rozdílů mezi jednotlivými přístupy uvádí tabulka 1.

Tabulka 1 Srovnání vybraných architektonických přístupů

Kritérium	Monolit	Mikroslužby	Modulární monolit
složitost vývoje	nízká	vysoká	střední
nasazení	jednoduché	složité	střední
škálovatelnost	omezená	vysoká	střední
testování	střední	složité	střední
provozní náročnost	nízká	vysoká	střední

Zdroj: (vlastní zpracování podle [9][10][11])

Ze srovnání vyplývá, že neexistuje jedno řešení vhodné pro všechny aplikace. Volba architektury závisí na rozsahu systému, požadavcích na vývoj a provoz i na očekávaném růstu aplikace [9][10][11].

Monolitická architektura bývá vhodná pro menší a středně velké aplikace, kde je důležitá jednoduchost vývoje a nasazení [9]. Mikroslužbová architektura se uplatňuje hlavně u rozsáhlejších systémů, kde je potřeba samostatně vyvíjet a škálovat jednotlivé části [10]. Modulární monolit stojí mezi těmito dvěma přístupy. Zachovává jednodušší provoz jednoho celku, ale zároveň klade důraz na vnitřní členění systému do oddělených modulů [11].

Při výběru architektury je proto nutné zohlednit nejen technické možnosti, ale také zkušenosti týmu a složitost vyvíjeného systému. Složitější řešení nemusí být vždy lepší. V řadě případů je vhodnější zvolit jednodušší přístup, který bude pro daný systém dobře udržitelný.

3 Technologie používané při vývoji moderních webových aplikací

Moderní webová aplikace obvykle nevzniká na základě jediné technologie. V praxi se kombinuje více nástrojů, které zajišťují backend, uživatelské rozhraní, práci s daty i nasazení systému.

Při výběru technologií je důležité sledovat nejen jejich funkce, ale také to, jak zapadají do navržené architektury. Roli hraje i kvalita dokumentace, možnosti rozšíření a dlouhodobá udržitelnost řešení.

3.1 Backendové technologie

Backendová část aplikace zajišťuje zpracování požadavků, aplikační logiku, práci s daty a komunikaci s dalšími službami. Ve webové aplikaci proto tvoří základ většiny operací, které nejsou přímo viditelné v uživatelském rozhraní.

Backendový stack obvykle zahrnuje programovací jazyk, runtime prostředí a aplikační framework. Konkrétní volba závisí na povaze vyvíjeného systému. Některé technologie se více hodí pro API a síťové služby, jiné nabízejí širší podporu pro celé webové aplikace [12][13][14][15][16][17][18].

Node.js je podle oficiální dokumentace asynchronní událostmi řízené runtime prostředí určené pro škálovatelné síťové aplikace [12]. Pokud je doplněno frameworkem NestJS, vzniká řešení vhodné pro serverové aplikace postavené na JavaScriptu nebo TypeScriptu. NestJS je v dokumentaci popsán jako framework pro efektivní a škálovatelné serverové aplikace a staví na běžně používaných HTTP serverech, zejména Expressu nebo Fastify [13].

Pythonový backend je vhodné rozlišit podle použitého frameworku. Django je rozsáhlý webový framework, který nabízí podporu pro modely, views, templates, formuláře, administraci, autentizaci i další běžné části webové aplikace [14]. FastAPI se naopak soustředí hlavně na vývoj API. Oficiální dokumentace jej popisuje jako moderní a výkonný framework pro tvorbu API v Pythonu se silnou vazbou na type hints a automatickou dokumentaci [15].

Spring Boot je založen na ekosystému Springu a podle oficiální dokumentace usnadňuje vytváření samostatných produkčních aplikací, které lze přímo spustit [16]. Nabízí starter dependencies, automatickou konfiguraci a produkční funkce, například health checks nebo metrics [16]. Proto se často používá u robustnějších backendových aplikací a služeb.

ASP.NET Core je podle Microsoft Learn cross-platform open-source framework pro moderní webové aplikace [17]. Dokumentace zároveň uvádí, že aplikace v ASP.NET Core jsou lehké, modulární a mají vestavěnou podporu pro dependency injection. Tato modularita spolu s podporou dependency injection přispívá k testovatelnosti a udržovatelnosti systému [17]. Framework se používá jak pro webová API, tak pro rozsáhlejší podnikové webové aplikace [17].

Go se v oficiální dokumentaci spojuje hlavně s návrhem výkonných síťových služeb. Dokumentace Go uvádí, že concurrency je klíčová pro high performance network services a že goroutines a channels poskytují jednoduchý a efektivní způsob vyjádření souběžného zpracování [18]. Zároveň je ale při práci se souběžností potřeba řešit datové závody. Oficiální

dokumentace k race detectoru upozorňuje, že data races patří mezi časté a obtížně laditelné chyby v concurrent systems [19].

Pro lepší přehled je možné jednotlivé technologie porovnat v Tabulce 2.

Tabulka 2 Srovnání vybraných backendových technologických stacků

Technologický stack	Typické využití	Výhody	Poznámka k použití
Node.js + NestJS	API, realtime služby, serverové aplikace.	asynchronní a událostmi řízené zpracování, škálovatelné síťové aplikace, podpora TypeScriptu, dobrá organizace aplikace přes moduly.	je nutné vyhnout se blokování event loopu [12][13].
Django	klasické webové aplikace, administrace, formuláře, aplikace s širší serverovou částí.	široká vestavěná funkcionalita, modelová vrstva, šablony, formuláře, autentizace, administrace.	jde o širší framework, proto je vhodný hlavně tam, kde projekt využije více jeho částí [14].
FastAPI	webová API, služby orientované na JSON/OpenAPI.	vysoký výkon, type hints, automatická dokumentace, orientace na API standardy.	framework je zaměřen hlavně na API vrstvu, ne na plnohodnotný server-side rendering [15].
Java (Spring Boot)	samostatné produkční služby, REST aplikace, robustnější backendy.	starter dependencies, automatická konfigurace, embedded server, produkční funkce.	stack dává největší smysl tam, kde projekt počítá se Spring ekosystémem [16].
C# (ASP.NET Core)	webová API, moderní webové aplikace, podnikové systémy.	cross-platform přístup, modularita, dependency injection, podpora MVC i Web API.	stack je přirozeně svázaný s .NET ekosystémem [17].
Go	síťové služby, concurrent backendy, výkonné serverové procesy.	jednoduché a efektivní concurrency primitives, vhodné pro network services.	při souběžném přístupu je nutné hlídat data races a synchronizaci [18][19].

Zdroj: (vlastní zpracování podle [12][13][14][15][16][17][18][19])

3.2 Frontendové technologie

Frontend představuje tu část aplikace, kterou uživatel vidí a ovládá v prohlížeči. Zajišťuje zobrazení obsahu, zpracování uživatelských vstupů, navigaci mezi stránkami a komunikaci s backendem.

U současných webových aplikací se často nemění celý dokument, ale pouze konkrétní části stránky. Aplikace tak reaguje rychleji a překresluje jen prvky, kterých se změna týká.

Z technického hlediska frontend stojí hlavně na JavaScriptu a jeho ekosystému. V praxi se často používají knihovny a frameworky založené na komponentovém přístupu. React popisuje uživatelské rozhraní jako sadu komponent, Vue uvádí deklarativní komponentový model a Angular se prezentuje jako framework pro tvorbu škálovatelných webových aplikací [20][21][22].

React je knihovna pro tvorbu uživatelských rozhraní založená na komponentách [20]. Tento přístup umožňuje vývojářům budovat aplikace deklarativním způsobem, přičemž mentální model Reactu čerpá z funkcionálního i objektově orientovaného programování a zaměřuje se na izolované komponenty jako na primární stavební jednotky [24].

Vue je framework pro tvorbu uživatelských rozhraní, který staví na HTML, CSS a JavaScriptu a používá deklarativní komponentový model [21]. Angular je plnohodnotný framework, který spojuje komponenty s dalšími vestavěnými nástroji, například pro routing, formuláře nebo dependency injection [22]. Next.js na React navazuje a rozšiřuje jej o další funkce pro tvorbu webových aplikací, včetně podpory různých způsobů renderování [23].

Volba konkrétní technologie závisí na požadavcích vyvíjené aplikace. Důležitá je velikost aplikace, způsob renderování, nároky na strukturu výsledného řešení i zkušenosti vývojového týmu. Pro menší a středně velké aplikace může být vhodné jednodušší řešení. U rozsáhlejších systémů bývá důležitější konzistentní struktura a širší ekosystém nástrojů.

Přehled základních rozdílů mezi vybranými frontendovými technologiemi uvádí tabulka 3.

Tabulka 3 Srovnání vybraných frontendových technologií

Technologie	Typické využití	Hlavní výhody	Poznámka k použití
React	interaktivní webová rozhraní, SPA, dashboardy.	komponentový přístup, skládání rozhraní z menších částí, práce s UI přes komponenty [20].	React je knihovna pro UI, proto se další části řešení často doplňují samostatně [20].
Vue	menší a střední webové aplikace, postupná integrace do existujících projektů.	deklarativní komponentový model, reaktivita, návaznost na HTML, CSS a JavaScript [21].	Vue je vhodné i pro postupné nasazení do existujícího rozhraní [21].
Angular	rozsáhlejší webové aplikace, podnikové systémy.	framework pro škálovatelné webové aplikace, komponenty, dependency injection, routing, formuláře [22].	Angular nabízí širší integrované řešení než samotné UI knihovny [22].
Next.js	React aplikace s důrazem na rendering strategie, výkon a full-stack přístup.	React framework, App Router, podpora full-stack web applications, SSR a SSG [23] [24].	Next.js je vhodný tam, kde nestačí samotný React a projekt potřebuje i řešení renderování a aplikační struktury [23][24].

Zdroj: (vlastní zpracování podle [20][21][22][23])

3.3 Databázové technologie

Základní definice uvádí, že databáze je kolekce dat uložených v počítači, přičemž tato data jsou typicky organizována do logických záznamů [25].

V kontextu webových aplikací pak databázová vrstva zajišťuje spolehlivé ukládání těchto záznamů – například uživatelských profilů nebo obsahu časových kapslí – a jejich zpřístupnění ostatním částem systému. Při návrhu této vrstvy je důležitý datový model, způsob dotazování i požadavky na konzistenci dat.

Relační databáze pracují s tabulkami tvořenými řádky a sloupci [26]. Tento přístup je vhodný tam, kde je důležitá jasná struktura dat a transakční zpracování. V souvislosti s transakcemi se používá pojem ACID, který označuje vlastnosti zajišťující správnost dat i při souběžném provozu nebo při chybách [27].

Vedle relačního modelu se v moderních aplikacích používají i nerelační databáze. Ty se liší hlavně datovým modelem a způsobem práce s daty.

MongoDB ukládá záznamy jako BSON dokumenty a používá flexibilní schéma. Dokumenty v jedné kolekci proto nemusí mít stejná pole ani stejné datové typy [28][29]. Redis vychází z modelu key-value a často se používá jako cache nebo pro práci se session daty [30]. Neo4j používá property graph model, v němž jsou data reprezentována pomocí uzlů, vztahů a vlastností [31].

V jednom systému lze kombinovat více databázových přístupů. Tento postup se označuje jako polyglot persistence a vychází z použití různých datových technologií pro různé druhy dat [32].

Pro srovnání základních vlastností jednotlivých databázových přístupů slouží tabulka 4.

Tabulka 4 Srovnání vybraných databázových přístupů

Databázový přístup	Datový model	Hlavní vlastnosti	Poznámka k použití
Relační databáze (SQL)	tabulky, řádky, sloupce, vztahy	pevnější struktura dat, transakční zpracování, důraz na konzistenci	vhodné pro systémy, kde je důležitá jasná struktura dat a práce s transakcemi
Dokumentové databáze (např. MongoDB)	BSON dokumenty, pole a vnořené struktury	flexibilní schéma, ukládání souvisejících dat do jednoho dokumentu	vhodné tam, kde se struktura dat častěji mění nebo kde je výhodné ukládat data podle způsobu jejich čtení
Key-value databáze (např. Redis)	dvojice klíč-hodnota	rychlý přístup podle klíče, nízká latence	vhodné pro cache, session data a jednoduché rychlé operace nad daty
Grafové databáze (např. Neo4j)	uzly, vztahy a vlastnosti	přirozená práce se vztahy mezi entitami	vhodné tam, kde je hlavní část problému založena na propojení objektů a jejich vztazích

Zdroj: (vlastní zpracování podle [26][27][28][29][30][31][32])

3.4 Kontejnerizace a nasazení

Součástí vývoje webové aplikace není jen implementace, ale také způsob jejího nasazení a provozu v různých prostředích. V praxi je potřeba řešit závislosti, konfiguraci a zajistit, aby se aplikace chovala předvídatelně při vývoji i v produkci [33][35].

Jedním z běžných přístupů je kontejnerizace. Kontejner je izolovaný proces se všemi soubory, které potřebuje ke spuštění [34]. Tím se snižuje závislost aplikace na konkrétním nastavení hostitelského systému [33][34].

Kontejnery sdílejí jádro hostitelského systému, zatímco virtuální stroj obsahuje celý operační systém s vlastním jádrem [34]. Proto mají kontejnery menší režii a obvykle se spouštějí rychleji než virtuální stroje [34].

V praxi se kontejnerizace často realizuje pomocí Dockeru. Obraz aplikace se vytvoří například pomocí Dockerfile a následně se spustí jako kontejner [33][34]. Pokud aplikace používá více služeb, například backend, databázi a cache, lze je společně definovat pomocí Docker Compose [35].

Docker Compose umožňuje definovat více kontejnerů v jednom YAML souboru a usnadňuje jejich spuštění i správu [35]. Tento přístup je vhodný hlavně pro aplikace, které se skládají z více částí a musí být spouštěny společně.

Hlavní přínos kontejnerizace spočívá v přenositelnosti a opakovatelnosti prostředí. Stejnou konfiguraci lze snadněji sdílet mezi vývojem, testováním a produkcí [33][35].

3.5 Bezpečnost webových aplikací

Bezpečnost webové aplikace zahrnuje několik vzájemně propojených oblastí. Základ tvoří ověření identity uživatele a následné řízení přístupu. V této souvislosti je nutné rozlišovat autentizaci, tedy ověření identity, a autorizaci, která určuje, zda má daný uživatel oprávnění provést požadovanou akci [36][37].

Další důležitou oblastí je ochrana komunikace mezi klientem a serverem. K tomu se používá HTTPS, které zajišťuje šifrování přenášených dat pomocí TLS [40]. Tento požadavek je zvlášť důležitý u aplikací komunikujících přes API, protože se při této komunikaci často přenášejí přihlašovací údaje, tokeny nebo jiná citlivá data [40].

V API orientovaných systémech se často používají tokenové mechanismy. JSON Web Token představuje kompaktní formát pro přenos claims mezi dvěma stranami a v praxi se používá například pro předávání informací o identitě nebo oprávněních [38]. Pro delegovaný přístup se používá OAuth 2.0, který umožňuje aplikaci získat omezený přístup ke službě jménem vlastníka zdroje, aniž by bylo nutné sdílet jeho heslo [39].

Bezpečnost ale nekončí u identity a přenosu dat. Webová aplikace musí zároveň řešit i běžné typy útoků, zejména XSS a CSRF [41][42]. Ochrana proto nespočívá v jedné technologii, ale v kombinaci více opatření, například v kontrole oprávnění na serverové straně, ochraně formulářů a endpointů a správném zabezpečení celé komunikace [37][40][41][42].

4 Přístupy k návrhu uživatelského rozhraní

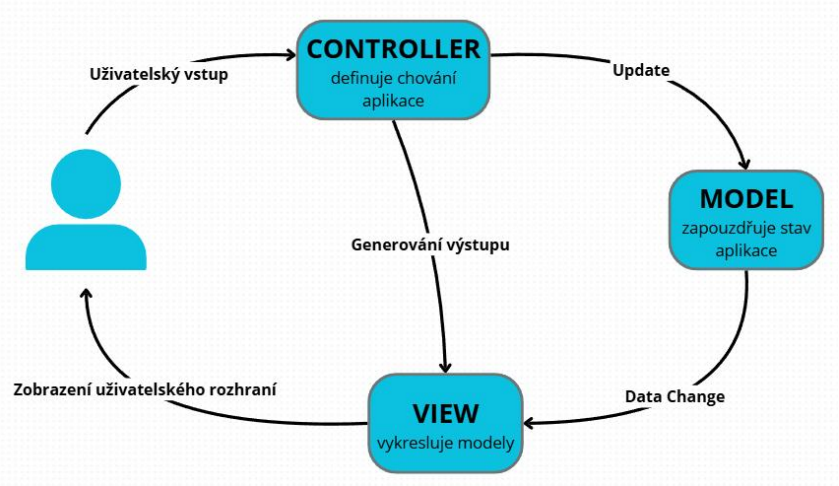
Návrh uživatelského rozhraní nepředstavuje pouze vizuální stránku aplikace, ale zahrnuje také způsob, jakým jsou organizovány jednotlivé komponenty a jak mezi sebou komunikují. Důležitým cílem je především oddělení prezentační vrstvy od aplikační logiky, což umožňuje lepší přehlednost systému, jednodušší údržbu i testování.

V průběhu vývoje softwaru vzniklo několik návrhových přístupů, které se snaží tento problém řešit různými způsoby. Mezi nejznámější patří MVC, MVP, MVVM a MVI. Tyto přístupy se liší především tím, jakým způsobem rozdělují odpovědnosti mezi jednotlivé části aplikace a jak řídí tok dat.

4.1 MVC

MVC (Model–View–Controller) patří mezi základní návrhové vzory používané při návrhu uživatelského rozhraní. Aplikaci rozděluje na tři části: model, view a controller. Smyslem tohoto rozdělení je oddělení odpovědností mezi zpracováním dat, reakcí na uživatelský vstup a zobrazením výsledku [43][44].

Princip tohoto uspořádání ukazuje obrázek 5.



Obrázek 5 Tok komunikace mezi komponentami architektury MVC
Zdroj: (vlastní zpracování)

Controller zpracovává požadavky uživatele a pracuje s modelem. Model nese aplikační data a související logiku. View zajišťuje zobrazení výsledku uživateli [43][44].

V praxi to znamená, že uživatelská akce nejprve vstupuje do controlleru. Ten vyhodnotí požadavek, případně upraví model a následně vybere view, které se má zobrazit. View pak vykreslí data předaná z controlleru [43].

Hlavní výhodou MVC je přehlednější rozdělení rolí v aplikaci. Díky tomu je snazší měnit uživatelské rozhraní, aniž by bylo nutné zasahovat do celé aplikační logiky, a naopak [43][44].

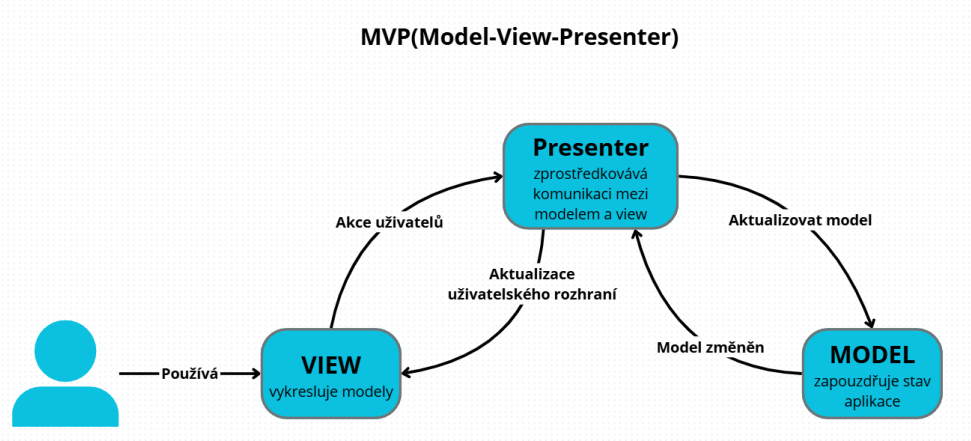
U větších aplikací ale může být obtížné udržet tyto hranice důsledně. Pokud se logika začne přesouvat mezi controllerem, view a modelem bez jasných pravidel, výhody tohoto přístupu se oslabují.

4.2 MVP a MVVM

Vedle MVC se používají i další přístupy, které oddělují logiku od zobrazení. Mezi nejčastější patří MVP (Model–View–Presenter) a MVVM (Model–View–ViewModel). Oba vzory rozdělují odpovědnosti mezi více částí aplikace, ale liší se způsobem komunikace [45][46].

V architektuře MVP zpracovává presenter vstupy z view, pracuje s modelem a připravuje data pro zobrazení. View slouží hlavně k vykreslení rozhraní a předání uživatelských akcí. Presenter obvykle komunikuje s rozhraním view, ne s jeho konkrétní implementací. To usnadňuje testování prezentační logiky [45].

Princip MVP ukazuje obrázek 6.

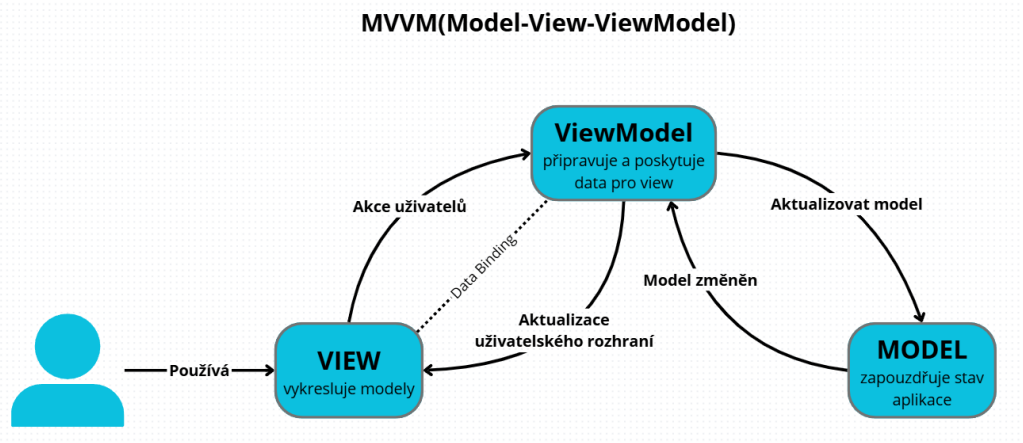


Obrázek 6 MVP architektura
Zdroj: (vlastní zpracování)

MVVM používá podobné rozdělení rolí, ale místo presenteru zavádí viewmodel. Ten poskytuje data a příkazy, na které se view váže pomocí data bindingu. Viewmodel tak zprostředkovává komunikaci mezi view a modelem [46][47].

Pro MVVM je důležitá právě datová vazba. Pokud zdroj dat správně oznamuje změny, mohou se nové hodnoty automaticky promítnout do uživatelského rozhraní. Stejný princip může fungovat i opačně, tedy od view ke zdrojovým datům [47].

Princip MVVM ukazuje obrázek 7.

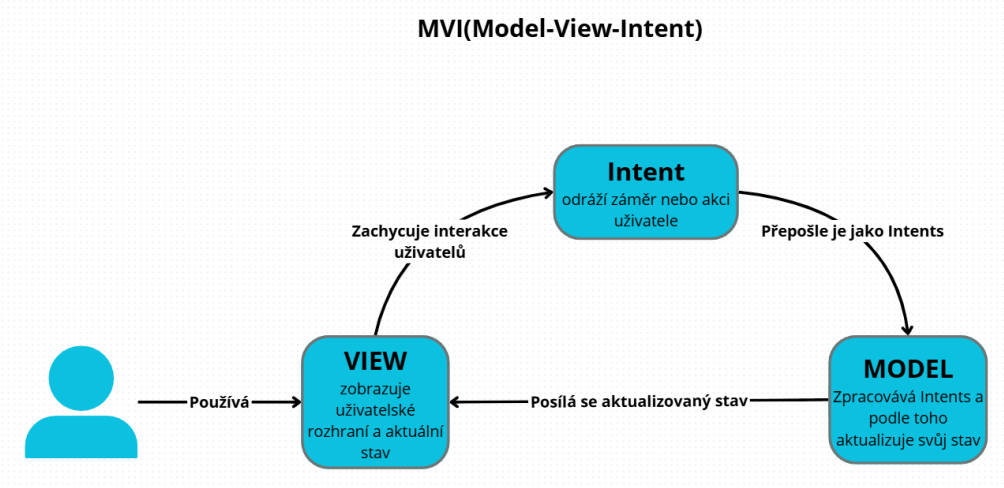


Obrázek 7 MVVM architektura
Zdroj: (vlastní zpracování)

4.3 MVI a jednosměrný tok dat

MVI (Model–View–Intent) je přístup založený na jednosměrném toku dat. Stav se předává směrem k uživatelskému rozhraní a uživatelské akce se vracejí zpět jako události [48][49].

Základní princip tohoto přístupu ukazuje obrázek 8.



Obrázek 8 MVI architektura
Zdroj: (vlastní zpracování)

Uživatel nejprve provede akci ve view. Tato akce se převede na intent. Následně se intent zpracuje a výsledkem je nový stav aplikace. Ten se potom znovu zobrazí ve view [48][49].

Výhodou tohoto přístupu je přehlednější práce se stavem. Tok změn má pevné pořadí, takže je snazší sledovat, odkud změna přišla a jak se promítla do výsledného rozhraní [48]. To je užitečné hlavně u aplikací, kde se stav mění často a z více míst.

Nevýhodou je vyšší nárok na návrh celé struktury. Pokud je aplikace malá, může být takové řešení zbytečně těžkopádné. Pro jednodušší aplikace pak nemusí být tento přístup nejvhodnější.

4.4 Srovnání a zhodnocení architektonických vzorů

Hlavní rozdíly mezi vzory MVC, MVP, MVVM a MVI shrnuje tabulka 5.

Tabulka 5 Srovnání vybraných architektonických vzorů

Parametr	MVC	MVP	MVVM	MVI
Základní princip	Oddělení modelu, view a controlleru	Oddělení view od prezentační logiky pomocí presenteru	Oddělení view od logiky pomocí viewmodelu a datové vazby	Řízení UI přes stav a jednosměrný tok změn
Zpracování vstupu	Vstup zpracovává controller	Vstup zpracovává presenter	Vstup se převádí na změny ve viewmodelu	Vstup se převádí na intent nebo událost
Práce se stavem	Stav je rozdělen mezi více částí aplikace	Presenter řídí prezentační logiku a reakci view	Viewmodel drží data a příkazy pro view	Stav se mění přes jasně definovaný cyklus
Vazba na UI	View zobrazuje data předaná controllerem	View je pasivní a předává akce presenteru	View je navázáno na viewmodel pomocí bindingu	View pouze zobrazuje stav a posílá akce
Typické použití	Klasické webové aplikace a jednodušší UI vrstvy	Aplikace s důrazem na testování prezentační logiky	Aplikace s výraznějším oddělením UI a logiky	Aplikace se stavově řízeným nebo reaktivním UI

Zdroj: (vlastní zpracování podle [43][45][46][47][48][49])

Z porovnání vyplývá, že se tyto přístupy liší hlavně způsobem zpracování vstupu, rolí prostředníka mezi logikou a rozhraním a prací se stavem. MVC je jednodušší a vychází z přímé spolupráce modelu, view a controlleru. MVP přesouvá prezentační logiku do presenteru. MVVM staví na viewmodelu a datové vazbě. MVI řídí změny pomocí stavu a jednosměrného toku dat.

Jednodušší aplikace si obvykle vystačí s MVC, protože tento vzor nevyžaduje složitou správu stavu. Pokud je potřeba lépe oddělit prezentační logiku od zobrazení, může být vhodnější MVP nebo MVVM. U aplikací, kde se často mění stav uživatelského rozhraní, dává smysl uvažovat také o MVI. Výběr vzoru proto závisí hlavně na tom, jak složitá je práce se stavem a jak přísně má být oddělena logika od rozhraní.

5 Analýza systému Time Capsule

Analýza systému Time Capsule vychází z požadavku ukládat uživatelský obsah a zpřístupnit jej až po dosažení nastaveného času. Tato část proto popisuje vlastnosti aplikace, uživatelské role, hlavní funkce, požadavky a omezení, ze kterých vychází následný návrh systému.

5.1 Charakteristika systému

Time Capsule je webová aplikace pro vytváření, správu a pozdější zpřístupňování digitálních kapslí. Uživatel vytvoří obsah, který se plně zpřístupní až po dosažení nastaveného času odemčení (unlockAt). Pokud to vnitřní pravidla dovolují, lze kapsli odemknout i ručně.

Kapsle je hlavní datový objekt aplikace. Kromě samotného obsahu uchovává také informace potřebné pro řízení přístupu, zejména stav, viditelnost a čas odemčení. Do záznamu lze doplnit název, text, volitelnou expiraci, tagy, geolokaci, titulní obrázek a další mediální přílohy.

Architektura systému vychází z modelu klient-server. Frontend využívá knihovnu React, jazyk TypeScript a nástroj Vite. Klientské rozhraní nabízí správu profilů, 2D mapu, 3D glóbus, kalendář, chat a administrační panel. Backend běží na frameworku Spring Boot. Zajišťuje REST API, autentizaci, plánované odemykání kapslí, notifikace a komunikaci v reálném čase přes protokol WebSocket (STOMP).

Datovou vrstvu tvoří databáze MongoDB. Návrh počítá s distribuovaným shardovaným clusterem provozovaným v prostředí Docker Compose. Systém zahrnuje databázová validační schémata a inicializační skripty. Mediální soubory se aktuálně ukládají lokálně na souborový systém serveru. Toto řešení slouží pro vývoj a testování, pro produkční nasazení jej bude nutné nahradit dedikovaným úložištěm.

5.2 Analýza předmětné oblasti

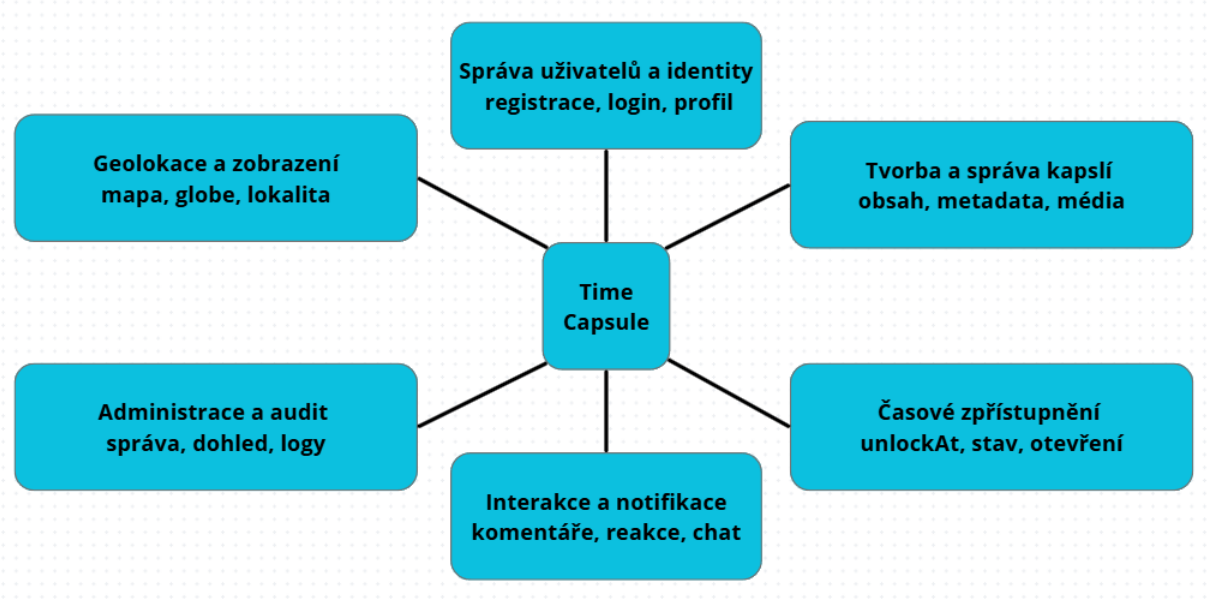
Při návrhu systému Time Capsule bylo potřeba řešit několik oblastí. Nejde jen o uložení textu nebo souborů, ale také o časové omezení přístupu, práci s oprávněními, notifikace, sociální interakce a geolokaci:

- **Osobní archivace dat:** Systém dlouhodobě uchovává uživatelské textové záznamy a multimédia. Kapsle představuje komplexní datový objekt, který spojuje samotný obsah s informací o autorství a čase.
- **Odložené publikování:** Zveřejnění dat je vázáno na konkrétní čas v budoucnosti. Zpřístupnění obsahu podléhá předem definovaným podmínkám a metadatům vázaným na danou kapsli.
- **Správa oprávnění a soukromí:** Uživatel definuje viditelnost každého záznamu. Kapsle může být soukromá, veřejná nebo sdílená pouze s konkrétními uživateli (private, public, shared). Backend při pokusu o zobrazení vyhodnocuje aktuální čas vůči nastavenému pravidlu viditelnosti.
- **Architektura řízená událostmi (event-driven):** Jakmile nastane čas odemčení, backend změní stav kapsle a spustí navazující akce. Může například odeslat notifikaci, aktualizovat dostupnost obsahu a informovat klientské rozhraní v reálném čase.
- **Sociální interakce:** Systém obsahuje komunikační vrstvu. Uživatelé mohou využívat interní chat, navazovat spojení mezi profily (follow graf) a přidávat komentáře či reakce k otevřenému veřejnému obsahu.

- **Geolokace:** Záznamy lze doplnit o přesné souřadnice. Systém tento prostorový kontext využívá k vizualizaci dat na 2D mapě nebo na 3D modelu glóbu.

5.3 Analýza hlavních funkcí systému

Funkce systému Time Capsule lze rozdělit do šesti oblastí. Každá z nich řeší jinou část práce s aplikací a navazuje na ostatní moduly přes aplikační rozhraní. Základní rozložení těchto oblastí ukazuje obrázek 9.



Obrázek 9 Hlavní funkční oblasti systému Time Capsule
Zdroj: (vlastní zpracování)

Správa uživatelů a identity: První oblastí je správa identity. Tento modul zajišťuje registraci, autentizaci uživatelů (včetně podpory protokolu OAuth2) a správu osobního profilu. Modul jednoznačně určuje vlastníka dat a podle toho řídí přístup k neveřejnému obsahu.

Tvorba a správa kapslí: Na správu identity navazuje práce s kapslemi, tedy hlavní část aplikace. Modul umožňuje vytváření nových záznamů, definici metadat a nahrávání mediálních souborů. Kapacita pro obrazové a video přílohy je omezena na velikost 50 MB na jeden soubor. Do okamžiku odemčení může vlastník obsah své kapsle upravovat.

Časové zpřístupnění obsahu: Důležitou částí systému je časové zpřístupnění obsahu. Systém prostřednictvím interního plánovače pravidelně prověřuje časový parametr unlockAt u zapečetěných záznamů. Po dosažení tohoto času backend automaticky mění stav kapsle na otevřenou a zpřístupňuje data podle nastavených pravidel viditelnosti.

Interakce a notifikace: Po otevření kapsle s ní mohou ostatní uživatelé podle nastavených práv dále pracovat. U veřejných záznamů mohou přidávat komentáře a reakce. Modul zároveň ukládá zprávy ze soukromého chatu a odesílá e-mailová upozornění na nové události.

Geolokace a zobrazení: Další oblast pracuje s geolokací uložených kapslí. Pokud má kapsle přiřazené souřadnice, může ji aplikace zobrazit na 2D mapě nebo na 3D glóbu. Uživatel tak může obsah procházet nejen podle času nebo autora, ale také podle místa.

Administrace a audit: Administrace slouží k provozní správě aplikace. Administrátor v tomto modulu spravuje uživatelské účty, moderuje obsah a nastavuje systémové tagy. Součástí je také sledování auditních logů, které zachycují veškeré důležité modifikace dat v databázi.

5.4 Uživatelské role a scénáře použití

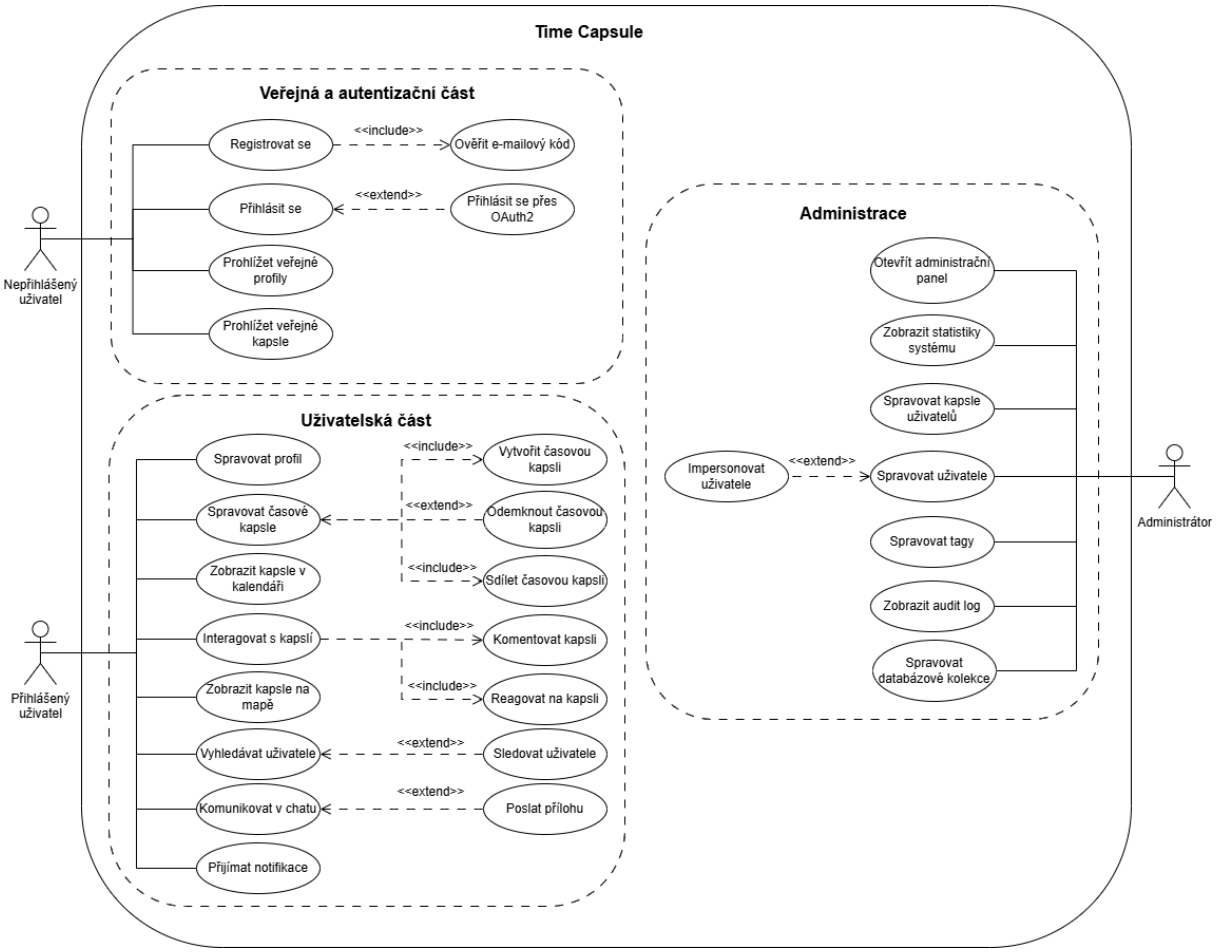
Návrh aplikace Time Capsule pracuje se třemi typy uživatelů: nepřihlášeným uživatelem, přihlášeným uživatelem a administrátorem. Jejich hlavní možnosti v systému ukazuje UML diagram případů užití na obrázku 10. Diagram popisuje funkce z pohledu uživatele, proto nezachycuje vnitřní implementaci aplikace ani konkrétní API rozhraní.

Nepřihlášený uživatel může používat pouze veřejně dostupné části aplikace. Má možnost vytvořit si účet, přihlásit se, využít přihlášení přes OAuth2 a prohlížet veřejné profily nebo veřejné kapsle. Při registraci systém vyžaduje ověření e-mailového kódu.

Přihlášený uživatel pracuje s hlavní částí aplikace. Může spravovat svůj profil, vytvářet a upravovat časové kapsle, sdílet je s ostatními uživateli a podle nastavených pravidel je odemykat. U otevřených kapslí může přidávat komentáře a reakce. Kromě toho může vyhledávat a sledovat jiné uživatele, komunikovat v chatu, zobrazovat kapsle v kalendáři nebo na mapě a přijímat notifikace.

Administrátor má samostatnou roli a používá administrační část aplikace. V ní může zobrazovat statistiky systému, spravovat uživatele, kapsle uživatelů, tagy, auditní log a databázové kolekce. Funkce impersonace slouží pro situace, kdy je potřeba ověřit problém z pohledu konkrétního uživatele.

Případy užití jsou v diagramu rozděleny do tří částí. Veřejná a autentizační část zahrnuje registraci, přihlášení a prohlížení veřejného obsahu. Uživatelská část popisuje běžnou práci přihlášeného uživatele s profilem, kapslemi, chatem, mapou, kalendářem a notifikacemi. Administrační část shrnuje funkce potřebné pro správu aplikace.



Obrázek 10 UML diagram případů užití systému Time Capsule
Zdroj: (vlastní zpracování)

5.5 Funkční požadavky

Funkční požadavky popisují, jaké operace musí systém Time Capsule podporovat. Jsou rozděleny podle hlavních částí aplikace.

Autentizace a správa identity

- **FR-01:** Systém umožní registraci nového uživatele a následné ověření účtu.
- **FR-02:** Aplikace zajistí přihlášení pomocí e-mailu a hesla nebo přes externí OAuth2 poskytovatele.
- **FR-03:** Systém spravuje uživatelské relace, obnovuje přístupové JWT tokeny a umožňuje bezpečné odhlášení.
- **FR-04:** Uživatel může spravovat svůj profil, nahrávat avatar a měnit přístupové heslo.
- **FR-05:** Nepřihlášený návštěvník je při pokusu o přístup k chráněnému obsahu automaticky přesměrován na přihlašovací obrazovku.

Tvorba a správa obsahu (Kapsle)

- **FR-06:** Systém umožní vytvoření kapsle s povinným názvem a definovaným časem odemčení (unlockAt).

- **FR-07:** Kapsle může obsahovat textový obsah, titulní obrázek, mediální přílohy do 50 MB, geolokaci, tagy a pravidla viditelnosti.
- **FR-08:** Systém umožní úpravu kapsle jejímu vlastníku. Editace je blokována po odemčení záznamu.
- **FR-09:** Aplikace validuje zadané časové parametry (unlockAt, expiresAt). Hodnoty musí být v logickém pořadí.

Časový zámek a distribuce dat

- **FR-10:** Backend chrání obsah zamčené kapsle před předčasným přístupem a v API odpovědích skrývá chráněná data.
- **FR-11:** Interní plánovač automaticky zpřístupní kapsli po dosažení času unlockAt.
- **FR-12:** Systém podporuje řízené sdílení kapsle konkrétním uživatelům (režim shared).

Zobrazení a sociální interakce

- **FR-13:** Uživatel si může prohlížet seznam vlastních kapslí a otevřené veřejné záznamy ostatních uživatelů.
- **FR-14:** Aplikace poskytuje kalendářní přehled záznamů a prostorovou vizualizaci na 2D mapě nebo ve 3D globe režimu.
- **FR-15:** Systém umožňuje přidávat komentáře a reakce. Tato funkce respektuje pravidla viditelnosti a aktuální stav kapsle.
- **FR-16:** Aplikace nabízí vyhledávání uživatelů a navazování spojení prostřednictvím follow-relací.
- **FR-17:** Systém poskytuje modul pro soukromý chat s podporou textových zpráv a mediálních příloh.

Administrace a systémový dohled

- **FR-18:** Administrátor má přístup k centrální správě uživatelských účtů, moderaci obsahu kapslí a definici systémových tagů.
- **FR-19:** Administrátor může využít auditovanou funkci impersonace pro dočasné převzetí identity konkrétního uživatele.

5.6 Nefunkční požadavky

Nefunkční požadavky popisují vlastnosti, které má aplikace splňovat při provozu. Týkají se hlavně bezpečnosti, výkonu, škálování a udržovatelnosti.

Bezpečnost a přístup

- **NFR-01:** Veškerá externí komunikace mezi prohlížečem a aplikací v produkčním prostředí musí probíhat výhradně přes šifrovaný protokol HTTPS.
- **NFR-02:** Přístupové a obnovovací JWT tokeny musí být chráněny uložením výhradně do HttpOnly cookies. Frontend nesmí tokeny ukládat do localStorage a backend je nesmí vracet v těle JSON odpovědí.
- **NFR-03:** Systém musí striktně oddělovat veřejná data, chráněný uživatelský obsah a administrační rozhraní pomocí pevných autorizačních pravidel.

Výkon a škálovatelnost

- **NFR-04:** Databázová vrstva musí podporovat horizontální škálování pomocí technologie MongoDB sharded cluster.
- **NFR-05:** Operace upravující data musí využívat cílené updaty s respektováním distribučního klíče (shard key) pro zachování výkonu.
- **NFR-06:** Datově náročné klientské komponenty, typicky 3D glóbus Cesium, musí systém načítat dynamicky až v momentě potřeby (lazy loading).
- **NFR-07:** Systém musí doručovat realtime události přes protokol WebSocket s minimalizací režie.

Kvalita kódu a udržitelnost

- **NFR-08:** Zdrojový kód frontendu musí procházet statickou analýzou (Lint, Typecheck) bez kritických chyb.
- **NFR-09:** Backendová část musí úspěšně absolvovat sadu automatizovaných testů s nulovým počtem selhání.
- **NFR-10:** Uživatelské rozhraní musí být responzivní a přístupné.

5.7 Omezení a specifika navrhovaného řešení

Následující body definují technická omezení, předpoklady a specifika aktuální verze systému Time Capsule.

- **Síťová infrastruktura a TLS:** Zabezpečení komunikace (TLS terminaci) zajišťuje reverzní proxy server Nginx. Komunikace mezi proxy serverem a backendem probíhá v privátní Docker síti přes nezabezpečený protokol HTTP. Požadavky na HTTP port jsou automaticky přesměrovány na HTTPS.
- **Zpracování proxy hlaviček:** Backend nemá vlastní TLS server. Pro správné vyhodnocení externího schématu požadavku, zabezpečených cookies a OAuth2 přesměrování spoléhá výlučně na hlavičky X-Forwarded-* předávané proxy serverem.
- **Lokální certifikáty:** Lokální vývojové prostředí používá automaticky generované self-signed certifikáty pro HTTPS, které prohlížeče označují jako nedůvěryhodné. Produkční prostředí vyžaduje certifikát vydaný důvěryhodnou certifikační autoritou.
- **Ukládání mediálních souborů:** Aplikace aktuálně ukládá nahraná média na lokální souborový systém serveru (backend/uploads). Toto řešení není určeno pro produkci a vyžádá si migraci na cloudové objektové úložiště.
- **Časování událostí:** Mechanismus otevírání kapslí negarantuje režim hard real-time. Přesnost odemčení je závislá na periodicitě běhu interního plánovače na backendu.
- **Závislost na externích službách:** Běh systému spoléhá na dostupnost externích služeb. Jedná se o OAuth2 poskytovatele (Google, GitHub), SMTP server a externí dlaždice pro mapové podklady.
- **Algoritmus doporučování:** Systém navrhuje nové kontakty na základě heuristiky z existujícího follow grafu. Nepoužívá strojové učení. Administrátorské účty jsou z doporučování systémově vyloučeny.

5.8 Modelování procesů systému

Modelování procesů doplňuje předchozí analýzu požadavků a uživatelských rolí. V této části jsou popsány hlavní průchody aplikací z pohledu uživatele i systému. Diagramy aktivit

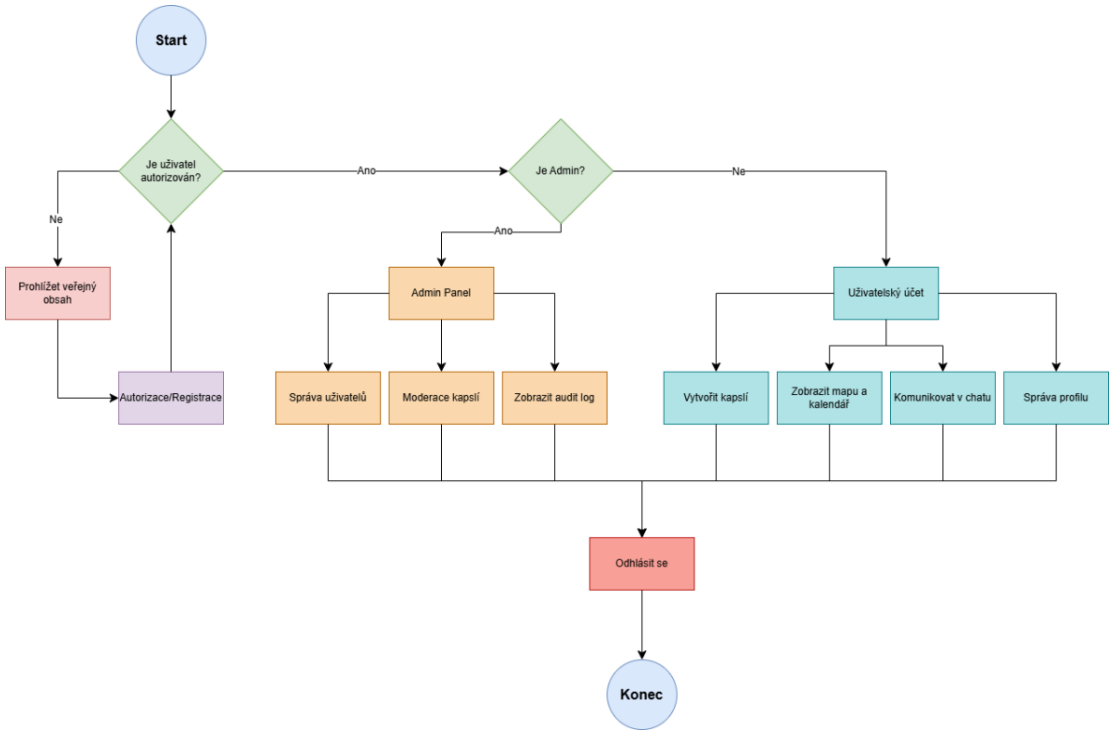
zachycují zejména rozhodovací body, návaznost jednotlivých kroků a situace, kdy se chování aplikace liší podle přihlášení uživatele, jeho role nebo stavu konkrétní kapsle.

Pro analýzu systému Time Capsule byly zpracovány tři diagramy aktivit. První popisuje obecný navigační tok aplikace, druhý se zaměřuje na vytvoření nové časové kapsle a třetí ukazuje postup při prohlížení otevřeného obsahu a následné interakci s ním. Tyto procesy byly zvoleny proto, že pokrývají hlavní práci uživatele s aplikací a zároveň ukazují nejdůležitější vazby mezi autentizací, správou kapslí, viditelností obsahu a sociálními funkcemi.

5.8.1 Hlavní navigační tok systému

Hlavní navigační tok popisuje vstup uživatele do aplikace a následné rozlišení dostupných částí systému podle stavu přihlášení a uživatelské role. Nepřihlášený návštěvník má přístup pouze k veřejně dostupnému obsahu a k možnostem registrace nebo přihlášení. Po úspěšné autentizaci systém rozlišuje běžného uživatele a administrátora.

Běžný uživatel může pracovat se svým profilem, vytvářet a spravovat časové kapsle, používat mapové zobrazení, chat a další uživatelské funkce. Administrátor má navíc přístup do administrační části, kde může spravovat uživatele, moderovat obsah a kontrolovat auditní záznamy. Tento tok je znázorněn na obrázku 11.



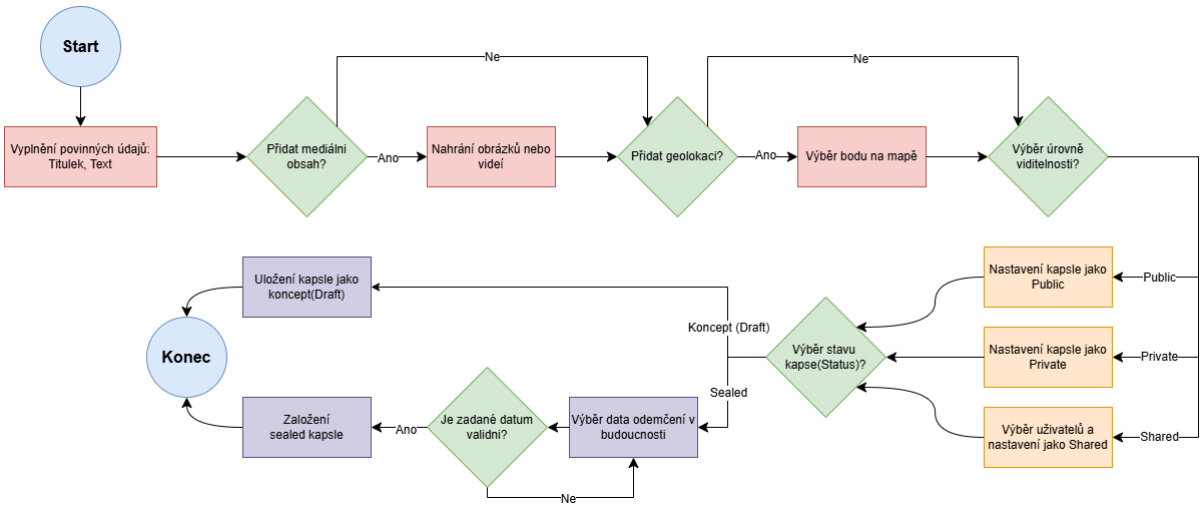
Obrázek 11 UML diagram aktivit – Hlavní navigační tok systému
Zdroj: (vlastní zpracování)

5.8.2 Proces vytvoření časové kapsle

Proces vytvoření časové kapsle začíná ve chvíli, kdy přihlášený uživatel otevře formulář pro nový záznam. Uživatel vyplní základní údaje kapsle, zejména název, textový obsah, datum odemčení a pravidla viditelnosti. Volitelně může přidat mediální soubory, tagy nebo geolokaci.

Po odeslání formuláře systém nejprve ověří správnost zadaných údajů. Kontroluje se především platnost času odemčení, povinné položky a případná omezení pro nahrané soubory. Pokud

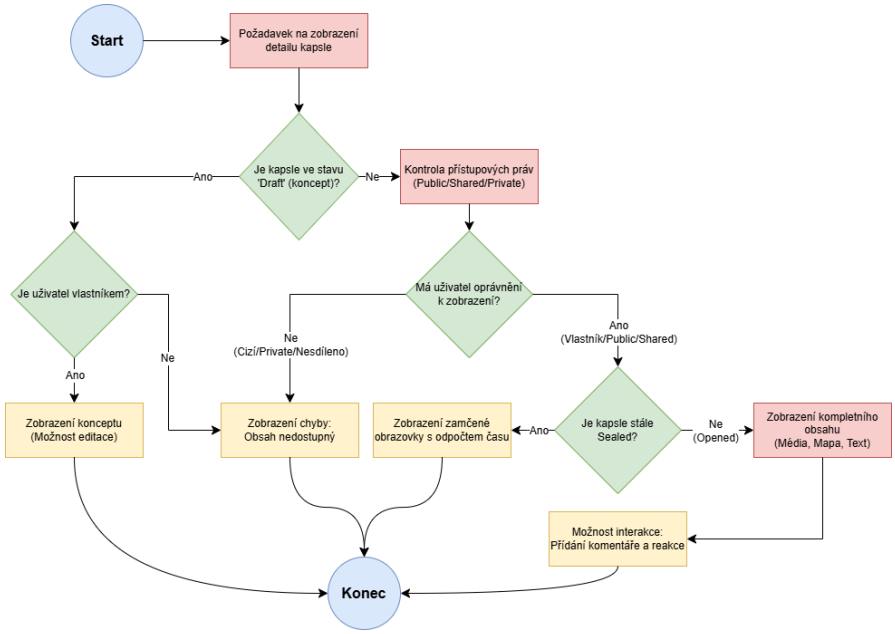
validace selže, uživatel zůstane ve formuláři a může údaje opravit. Pokud jsou data platná, backend vytvoří nový záznam kapsle, uloží metadata a případné přílohy. Výsledkem je uzamčená kapsle, která se zpřístupní až po dosažení nastaveného času. Celý postup ukazuje obrázek 12.



Obrázek 12 UML diagram aktivit – Proces vytvoření časové kapsle
Zdroj: (vlastní zpracování)

5.8.3 Proces prohlížení a interakce s obsahem

Třetí proces popisuje situaci, kdy uživatel otevře detail časové kapsle. Systém nejprve ověří, zda kapsle existuje, v jakém je stavu a zda má uživatel oprávnění ji zobrazit. Rozhodování vychází z nastaveného času odemčení a pravidel viditelnosti. Pokud kapsle ještě není otevřená nebo uživatel nemá potřebné oprávnění, aplikace zobrazí pouze omezené informace nebo přístup odmítne. Po splnění podmínek systém načte text, metadata, mediální přílohy a případné možnosti interakce. Celý postup je znázorněn na obrázku 13.



Obrázek 13 UML diagram aktivit – Proces prohlížení a interakce
Zdroj: (vlastní zpracování)

6 Návrh systému Time Capsule

6.1 Návrh celkové architektury systému

Aplikace Time Capsule je navržena na základě distribuované architektury klient-server. Celý systém je kontejnerizován a skládá se ze čtyř hlavních logických uzlů. Tyto uzly běží v izolovaných prostředích pomocí technologie Docker.

- **Prezentační vrstva (Frontend):** Klientská část je navržena jako Single Page Application (SPA) v knihovně React. Komunikuje s aplikační vrstvou asynchronně prostřednictvím rozhraní REST API. Pro příjem událostí v reálném čase udržuje trvalé spojení přes protokol WebSocket.
- **Aplikační vrstva (Backend):** Jádro systému tvoří monolitická aplikace postavená na frameworku Spring Boot. Zajišťuje veškerou business logiku, autentizaci, autorizaci a plánované úlohy pro otevírání kapslí. Funguje jako výhradní prostředník pro přístup k databázi.
- **Datová vrstva (Databáze):** Pro ukládání dat je navržen databázový systém MongoDB ve formátu sharded cluster. Architektura clusteru zahrnuje směrovač (mongos), konfigurační servery a sady replik. Tento návrh zajišťuje vysokou dostupnost dat a možnost horizontálního škálování.
- **Infrastrukturní a bezpečnostní vrstva (Proxy):** Vstupním bodem do systému je reverzní proxy server Nginx. Zajišťuje TLS terminaci, přesměrování nezabezpečeného HTTP provozu na HTTPS a směrování požadavků na interní služby v rámci privátní sítě Dockeru.

Toto rozdělení splňuje princip oddělení zodpovědností (Separation of Concerns). Frontend a backend lze díky nezávislé architektuře a kontejnerizaci vyvíjet, testovat i nasazovat odděleně.

6.2 Návrh komunikace mezi jednotlivými komponentami

Aplikace Time Capsule využívá pro komunikaci mezi klientskou a serverovou částí kombinaci synchronních a asynchronních protokolů.

- **Synchronní komunikace (REST API):** Základním komunikačním rozhraním je REST API architektura. Výměna strukturovaných dat probíhá ve formátu JSON. Pro nahrávání mediálních příloh (obrázky, videa) systém využívá formát multipart/form-data bez explicitního JSON hlavičkování. Bezpečnost API volání zajišťují HttpOnly cookies obsahující JWT tokeny. Klientská aplikace tyto cookies automaticky přikládá ke každému požadavku pomocí parametru credentials: 'include'.
- **Asynchronní komunikace v reálném čase (WebSocket):** Pro okamžité doručování zpráv v chatu a notifikací o odemčení kapsle slouží protokol WebSocket. Nad tímto transportním protokolem systém využívá textový subprotokol STOMP (Simple Text Oriented Messaging Protocol). Pro zajištění maximální kompatibility se staršími prohlížeči nebo restriktivními sítěmi implementuje backend i frontend technologii SockJS jako fallback mechanismus. Klientská aplikace navazuje toto spojení dynamicky pouze na relevantních obrazovkách, čímž optimalizuje celkový síťový provoz.
- **Komunikace s externími systémy:** Aplikace se integruje se službami třetích stran. Pro externí autentizaci využívá standardizovaný protokol OAuth2, který zprostředkovává

komunikaci se servery společností Google a GitHub. Odesílání e-mailových notifikací zajišťuje backendový modul Spring Mail prostřednictvím připojení na externí SMTP server.

- **Zprostředkování proxy serverem:** Veškerá externí komunikace z internetu směřuje nejprve na Nginx proxy server. Tento server ukončuje zabezpečené HTTPS spojení a požadavky následně předává interní backendové službě přes standardní HTTP protokol.

6.3 Návrh doménového modelu

Doménový model definuje klíčové entity systému, jejich vlastnosti a vzájemné vztahy z pohledu business logiky aplikace. Návrh vychází z požadavků na persistenci dat v dokumentově orientované databázi MongoDB.

Hlavní entity systému

- **Uživatel (User):** Centrální entita reprezentující identitu v systému. Obsahuje autentizační údaje (e-mail, hash hesla), profilové informace (uživatelské jméno, avatar), nastavení OAuth2 poskytovatelů a aktuální stav účtu (aktivní, blokovaný).
- **Kapsle (Capsule):** Stěžejní entita systému nesoucí uživatelský obsah. Uchovává textová data, reference na mediální soubory, geolokační údaje a metadata ve formě tagů. Klíčovými atributy jsou časové parametry unlockAt (datum odemčení) a expiresAt (volitelná expirace), které řídí dostupnost obsahu.
- **Média (Media Items):** Reprezentují nahrané soubory (obrázky, videa) připojené ke kapslím nebo zprávám v chatu. Každá položka obsahuje URL identifikátor, MIME typ a metadata o velikosti souboru.

Sociální a interakční entity

- **Komentáře a reakce (Comments & Reactions):** Entity zajišťující sociální interakci nad otevřenými kapslemi. Komentáře podporují hierarchickou strukturu (vlákna) a jsou pevně vázány na ID konkrétní kapsle.
- **Sledování (Follows):** Reprezentuje orientovaný graf sociálních vazeb mezi uživateli. Tento graf slouží primárně k budování uživatelského profilu (seznamy sledujících a sledovaných) a k algoritmickému generování návrhů na nová propojení (doporučené kontakty).
- **Chatovací zprávy (ChatMessages):** Entity určené pro uchovávání historie komunikace v reálném čase. Zprávy jsou doručovány asynchronně prostřednictvím protokolu WebSocket (STOMP), přičemž v databázi jsou persistovány pro zajištění kontinuity konverzací a podporu multimediálních příloh či sdílení tokenů kapslí.

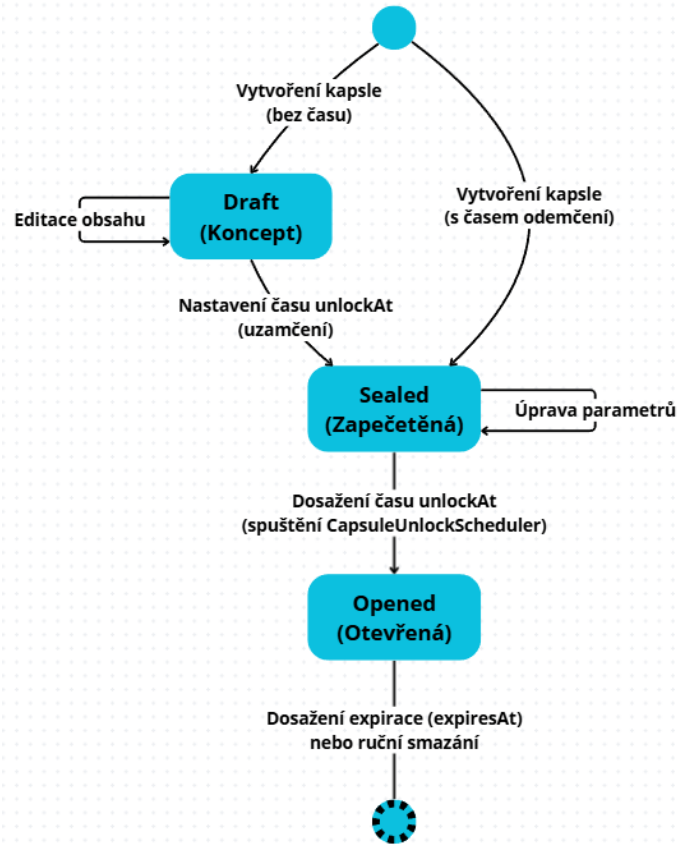
Pomocné a systémové entity

- **Tagy (Tags):** Číselník systémových a uživatelských štítků pro kategorizaci obsahu.
- **Sdílení (Shares):** Logické vazby definující přístupová práva pro konkrétní uživatele u kapslí v režimu shared.
- **Auditní logy (AdminAuditLogs):** Záznamy o kritických operacích a administrativních zásazích pro zajištění zpětné sledovatelnosti změn v systému.

Model využívá flexibilní schéma MongoDB. Metadata médií, geolokační souřadnice a další vnořené údaje proto mohou být uložené přímo v dokumentu kapsle. Tento způsob ukládání odpovídá povaze aplikace, protože většina dat se načítá společně s konkrétní kapslí.

6.4 Životní cyklus kapsle

Životní cyklus kapsle tvoří několik stavů. Ty určují, kdo kapsli vidí, zda ji lze upravovat a kdy je dostupný její obsah. Stav kapsle se mění buď po akci uživatele, nebo automaticky na straně backendu. Vizuální reprezentaci těchto stavů a přechodů znázorňuje Obrázek 14.



Obrázek 14 Stavový diagram životního cyklu kapsle
Zdroj: (vlastní zpracování)

Definované stavy kapsle

- **Draft (Koncept):** Výchozí stav po vytvoření, pokud uživatel kapsli neuzavře. Obsah je viditelný pouze vlastníkovi a lze jej libovolně editovat. Kapsle v tomto stavu nemá nastavený pevný čas odemčení.
- **Sealed (Zapečetěná):** Stav, do kterého uživatel kapsli přepne po dokončení úprav. V tomto okamžiku se zafixuje čas odemčení (unlockAt). Obsah (tělo zprávy a média) je v databázi chráněn a systém jej v API odpovědích maskuje. Vlastník může parametry editovat, dokud nenastane čas odemčení.
- **Opened (Otevřená):** Finální stav, kdy je obsah zpřístupněn určenému publiku (podle nastavení viditelnosti). K tomuto přechodu dochází automaticky prostřednictvím

systémového plánovače. V tomto stavu již běžný uživatel nemůže obsah měnit, aby byla zachována integrita "digitální vzpomínky".

Proces přechodu (Workflow)

1. **Vytvoření a příprava:** Uživatel definuje metadata a obsah. Kapsle existuje jako draft nebo se rovnou ukládá jako sealed.
2. **Časová prodleva:** Kapsle setrvává ve stavu sealed. Backendový proces CapsuleUnlockScheduler monitoruje pole unlockAt.
3. **Automatické odemčení:** Po dosažení stanoveného času systém změní stav na opened a vygeneruje notifikace pro vlastníka.
4. **Expirace (volitelně):** Pokud je nastaven parametr expiresAt, může být kapsle po tomto datu zneplatněna nebo smazána, což uzavírá její životní cyklus.

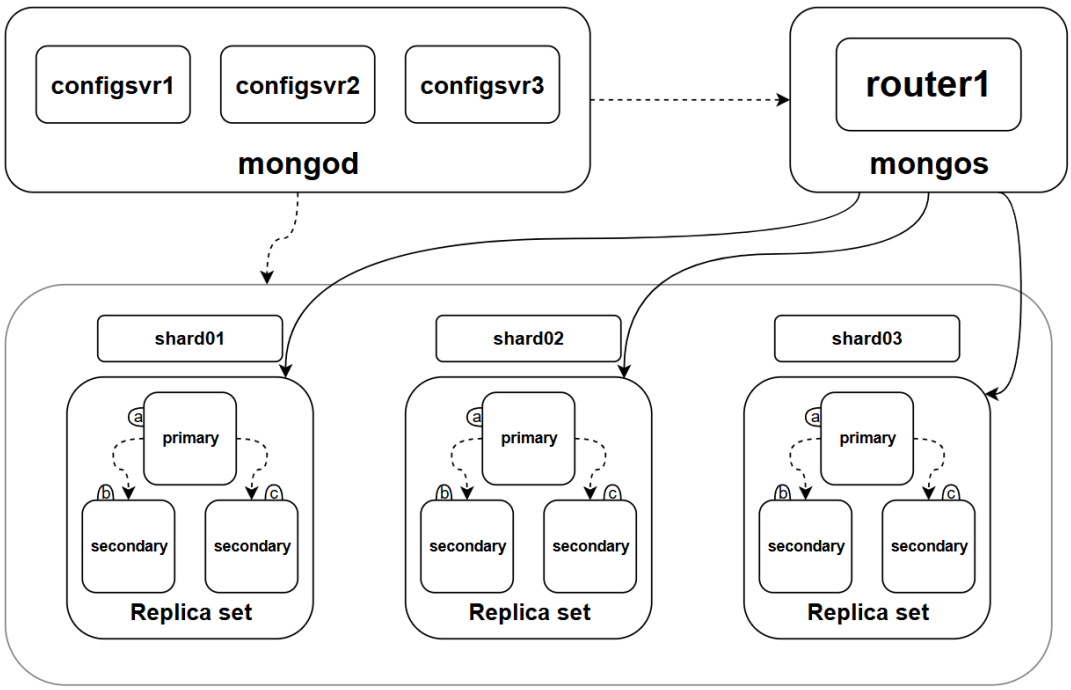
Tento stavový model zajišťuje, že aplikace funguje jako bezpečný digitální archiv s řízeným přístupem v čase.

6.5 Návrh databázového řešení

Pro ukládání a správu dat systému Time Capsule byla zvolena dokumentově orientovaná NoSQL databáze MongoDB. Tato volba reflektuje potřebu ukládat komplexní a hierarchické datové struktury, jako jsou metadata mediálních souborů, geolokační body a vnořená pole tagů, které by v relační databázi vyžadovaly složité spojování tabulek (JOIN operace).

Architektura databázového clusteru

Databázová vrstva není navržena jako samostatná instance, ale využívá pokročilou architekturu MongoDB sharded cluster pro zajištění horizontální škálovatelnosti a vysoké dostupnosti. Schématické znázornění této architektury představuje Obrázek 15.



Obrázek 15 Architektura MongoDB sharded clusteru
Zdroj: (vlastní zpracování)

Cluster se skládá z následujících komponent, které běží v oddělených Docker kontejnerech:

- **Směrovač (mongos):** Funguje jako vstupní brána pro backend aplikaci a stará se o distribuci dotazů na správné uzly.
- **Konfigurační servery (Config servers):** Udržují metadata o clusteru a mapování dat na jednotlivé shardy.
- **Sady replik (Shard replica sets):** Systém využívá tři nezávislé sady replik, které fyzicky uchovávají rozdělená data a zajišťují jejich redundanci.

Struktura kolekcí

Základní logické rozdělení dat je realizováno pomocí kolekcí. Mezi klíčové kolekce patří:

- **users** a **pending_users** pro správu uživatelských identit a registračního procesu.
- **capsules** pro ukládání veškerého obsahu digitálních kapslí, včetně polí unlockAt a visibility.
- **chat_messages**, **comments** a **reactions** pro oddělenou správu sociálních interakcí a synchronní komunikace.
- **admin_audit_logs** pro centrální logování systémových změn, zahrnující běžné uživatelské mutace i specifické akce administrátora.

Vztahy mezi hlavními datovými objekty systému jsou kvůli přehlednosti uvedeny v samostatném ER diagramu v příloze C.

Iniciace a bezpečnost dat

Při prvotním sestavení infrastruktury (pomocí kontejneru init-cluster) databáze automaticky aplikuje validační schémata, nastavuje distribuční klíče (shard keys) a nahrává počáteční (seed) data z připravených CSV souborů.

Pro zachování maximálního výkonu při zápisu a úpravách dat ve fragmentovaném (sharded) prostředí využívá aplikační backend knihovnu MongoTemplate ze Spring Data MongoDB. Ta umožňuje provádět cílené modifikace s explicitním uvedením distribučních klíčů. Tím se systém vyhýbá neefektivním plošným operacím přes všechny dostupné shardy. Dalším ochranným mechanismem je využití "soft delete" (příznak deletedAt), které zabraňuje nevratné fyzické ztrátě propojených dat, jako jsou komentáře či kapsle.

6.6 Návrh bezpečnostního řešení

Bezpečnostní architektura systému Time Capsule je navržena s ohledem na ochranu uživatelských dat a zamezení neoprávněného přístupu. Řešení kombinuje zabezpečení na úrovni sítě, identity a aplikační logiky.

Síťová bezpečnost a TLS terminace

- Hlavní veřejný vstup do systému je veden přes šifrovaný protokol HTTPS a reverzní proxy server Nginx.
- Proxy server předává požadavky na aplikační backend v rámci izolované privátní Docker sítě. Pro zachování informací o původním šifrovaném požadavku využívá hlavičky X-Forwarded-*.

Autentizace a správa relací (Sessions)

- Systém poskytuje tradiční přihlašovací mechanismus (e-mail a heslo) i moderní OAuth2 autentizaci přes poskytovatele Google a GitHub.
- REST API využívá bezestavovou JWT autentizaci; OAuth2 přihlašovací tok však může dočasně využívat server-side session state pro udržení kontextu přesměrování.
- Přístupové (access) a obnovovací (refresh) tokeny jsou chráněny striktním uložením do HttpOnly cookies.
- Frontend k těmto tokenům nemá přístup z prostředí JavaScriptu, čímž je významně omezeno riziko odcizení tokenů v případě XSS (Cross-Site Scripting) útoků. Klientská aplikace přikládá cookies k požadavkům automaticky pomocí konfigurace credentials: 'include'.

Autorizace a ochrana datového modelu

- Přístupová práva v systému jsou definována na základě identity (anonymní návštěvník, přihlášený uživatel, administrátor) a na základě vlastnictví či sdílení konkrétního obsahu.
- Backend aktivně chrání obsah neotevřených kapslí. Pokud je kapsle ve stavu sealed a požadavek nepochází z editačního režimu oprávněného vlastníka, systém z API odpovědi odstraní (namaskuje) citlivá pole body a media.

6.7 Návrh notifikační a časové logiky

Základní přidanou hodnotou systému Time Capsule je práce s časem jako aktivní součástí datového modelu. Obsah není publikován okamžitě, ale čeká na definovaný okamžik odemčení, což vyžaduje spolehlivou časovou a notifikační infrastrukturu.

Časová logika a plánovač (Scheduler)

- Otevírání kapslí je primárně řízeno asynchronním procesem na pozadí. Komponenta CapsuleUnlockScheduler se spouští v konfigurovatelných pravidelných intervalech (ve výchozím nastavení každých 5 sekund), vyhledává v databázi připravené kapsle ve stavu sealed a mění jejich stav na opened.
- Plánovač negarantuje provedení v režimu hard real-time. Přesný okamžik odemčení je závislý na periodicitě běhu této úlohy a aktuální zátěži systému.
- Mechanismus plánovače je pro zvýšení robustnosti doplněn o takzvanou lazy (fallback) kontrolu. Při načítání seznamu kapslí, detailu konkrétní kapsle nebo při explicitním volání odemykacího endpointu servisní vrstva proaktivně kontroluje časové značky a případně aktualizuje stav.

Notifikační logika a doručování zpráv Systém využívá tříúrovňový model doručování upozornění:

- **Real-time události (WebSocket):** Při změně stavu kapsle nebo přijetí nové zprávy je odeslána událost přes protokol WebSocket (STOMP).
- **Okamžité transakční e-maily:** Pro kritické systémové události (úspěšné odemčení) systém odesílá e-mailovou notifikaci vlastníkovi bez zpoždění.
- **Souhrnné e-maily (Digest mechanismus):** Pro interakce s vysokou frekvencí aplikace využívá dávkování. Notifikace jsou sdružovány a odesílány hromadně s časovým odstupem.

6.8 Zdůvodnění výsledného návrhu

Návrh systému Time Capsule vychází z požadavků na časově řízené zpřístupnění obsahu, ochranu uživatelských dat a možnost pozdějšího rozšíření aplikace. Z těchto požadavků vycházejí také hlavní technologická a architektonická rozhodnutí.

- **Oddělení frontendové a backendové části:** Aplikace je rozdělena na klientskou část v Reactu a backend ve Spring Bootu. Uživatelské rozhraní je tak odděleno od aplikační logiky, což zjednodušuje údržbu systému. Spuštění jednotlivých částí v kontejnerech Docker zároveň usnadňuje přípravu vývojového a produkčního prostředí.
- **Použití MongoDB:** Pro ukládání kapslí byla zvolena dokumentová databáze MongoDB. Kapsle obsahují nejen základní textová data, ale také tagy, geolokaci, metadata médií a informace o viditelnosti. Dokumentový model umožňuje tyto údaje ukládat přirozeně v rámci jednoho záznamu. Sharded cluster je v návrhu použit jako možnost pozdějšího horizontálního rozšíření databázové vrstvy.
- **Zabezpečení pomocí HttpOnly cookies:** Přístupové a obnovovací tokeny jsou uloženy v HttpOnly cookies. Tento způsob omezuje přímý přístup k tokenům ze strany JavaScriptu v prohlížeči a snižuje riziko jejich zneužití při XSS útoku. Prohlížeč zároveň cookies přikládá k požadavkům automaticky podle nastavených bezpečnostních pravidel.
- **Kombinace REST API a WebSocket:** REST API je použito pro běžné operace, například načítání, vytváření a úpravu dat. Pro chat a notifikace je vhodnější WebSocket, protože server může klientovi odeslat zprávu ihned po vzniku události. Aplikace tak nemusí tyto změny zjišťovat opakovaným dotazováním.
- **Logika odemykání kapslí:** Otevírání kapslí je řešeno kombinací plánovače na pozadí a kontroly při přístupu přes API. Plánovač pravidelně vyhledává kapsle, které mají být otevřeny. Kontrola v API slouží jako doplňková ochrana pro situace, kdy uživatel požádá o kapsli dříve, než ji plánovač stihl zpracovat.

7 Implementace

Tato kapitola popisuje implementaci systému Time Capsule. Zaměřuje se na strukturu backendu a frontendu, použité knihovny a vybrané části zdrojového kódu na straně serveru i klientské aplikace.

7.1 Implementace backendové části

Backendová část aplikace je postavena na platformě Java a frameworku Spring Boot. Spring Boot zde slouží pro vytvoření REST API, práci s databází a správu plánovaných úloh.

7.1.1 Struktura backendové části

Zdrojový kód backendu je striktně uspořádán podle principů vrstvené architektury a oddělení zodpovědností (Separation of Concerns). Kompletní stromová struktura klíčových balíčků a tříd je pro přehlednost uvedena v Příloze A, která rovněž popisuje obsah přiloženého ZIP archivu.

Jednotlivé balíčky plní v systému jasně definovanou roli. Zatímco controllers zajišťují pouze validaci a směrování HTTP požadavků, balíček services obsahuje veškerá byznys pravidla. Významnou roli hraje oddělení databázových modelů (models) od transportních objektů (dto a responses). Tento přístup zabraňuje nechtěnému odhalení interních databázových polí klientovi a poskytuje stabilní rozhraní API i při změnách v databázi. Balíček config sdružuje kritická bezpečnostní nastavení, filtry a interceptory potřebné pro běh celého frameworku.

7.1.2 Použité závislosti a frameworky

Pro vývoj backendové části byl zvolen ekosystém Spring Boot, který poskytuje ucelenou infrastrukturu pro moderní javové aplikace. Jednotlivé knihovny a frameworky (definované v souboru pom.xml) jsou ve zdrojovém kódu rozděleny do logických celků podle jejich funkčního zaměření.

Aplikační jádro a struktura aplikace

- spring-boot-starter-web: Tvoří základ pro budování RESTful API. Poskytuje vestavěný server Tomcat a mechanismy pro mapování HTTP požadavků na metody kontrolerů.
- spring-modulith-starter-core: Tato knihovna je využívána pro logickou strukturaci monolitické aplikace do nezávislých modulů. Pomáhá udržovat čistou architekturu tím, že vynucuje pravidla pro interakci mezi jednotlivými doménami (např. oddělení modulu uživatelů od modulu kapslí).

Datová vrstva a perzistence

- spring-boot-starter-data-mongodb: Zajišťuje integraci s dokumentově orientovanou databází. Framework automatizuje tvorbu dotazů a poskytuje abstrakci MongoRepository, která výrazně zjednodušuje CRUD operace a mapování dokumentů na Java objekty.
- Třída MongoSslConfig: Vlastní specifická konfigurace zajišťující šifrované spojení s databázovým clusterem, což je klíčové pro ochranu dat při komunikaci v distribuovaném prostředí.

Bezpečnost a správa identit

- spring-boot-starter-security: Komplexní framework pro zajištění autentizace a autorizace. V aplikaci definuje bezpečnostní filtry, chrání koncové body API a spravuje kontext přihlášeného uživatele.
- spring-boot-starter-oauth2-client a spring-boot-starter-oauth2-resource-server: Tyto moduly umožňují delegovanou autentizaci. Systém díky nim bezpečně spolupracuje s poskytovateli Google a GitHub, přičemž standard OAuth2 zajišťuje, že aplikace nikdy nepracuje s hesly uživatelů z těchto platforem.
- jjwt-api, jjwt-impl, jjwt-jackson (Java JWT): Knihovny (verze 0.11.5) použité pro nízkoúrovňovou práci s JSON Web Tokeny. Umožňují bezpečné podepisování tokenů pomocí tajného klíče, jejich parsování a kontrolu validity (např. expirace).

Komunikace a real-time funkce

- spring-boot-starter-websocket: WebSocket slouží k obousměrné komunikaci mezi serverem a klientem. STOMP nad ním přidává fronty, přes které backend posílá zprávy konkrétním uživatelům.
- spring-boot-starter-mail: Framework pro integraci se službami SMTP. V aplikaci zajišťuje automatické odesílání transakčních e-mailů, jako jsou potvrzení o registraci nebo upozornění, že nastal čas otevření kapsle.

Validace, produktivita a konfigurace

- spring-boot-starter-validation (včetně jakarta.validation-api a hibernate-validator): Implementuje standard Jakarta Validation. Umožňuje definovat pravidla pro integritu dat přímo v DTO objektech pomocí anotací (např. @NotNull, @Size). Tím se předchází zpracování nevalidních dat.
- lombok: Knihovna pro redukci "boilerplate" kódu. Pomocí anotací automaticky generuje konstruktory, gettery, settery a metody toString. Automatické generování těchto metod zvyšuje čitelnost a udržovatelnost zdrojových kódů.
- spring-dotenv: Nástroj pro správu citlivých konfiguračních údajů (API klíče, databázová hesla) prostřednictvím souborů .env. To umožňuje oddělit konfiguraci od samotného kódu a zvyšuje bezpečnost při nasazení do různých prostředí.

7.1.3 REST API a kontrolery

Vrstva kontrolerů (balíček controllers) slouží jako primární vstupní bod pro veškeré HTTP požadavky od klientské aplikace. Její hlavní odpovědností je příjem požadavku, validace příchozích datových struktur a následné delegování složitější byznys logiky na servisní vrstvu, čímž je dodržen princip oddělení zodpovědností.

Vstupy od uživatele jsou mapovány do takzvaných Data Transfer Objects (např. CreateCapsuleRequest), u kterých rovnou probíhá kontrola integrity dat pomocí anotace @Valid (Bean Validation). Následující ukázka ze třídy CapsuleController ilustruje implementaci koncových bodů pro vytvoření nové digitální kapsle a její manuální odemčení:

Výpis 1 Implementace koncových bodů ve třídě CapsuleController

```
@RestController
@RequestMapping("/api/capsules")
public class CapsuleController {

    @PostMapping
```

```
    public ResponseEntity<CapsuleResponse> create(
            @Valid @RequestBody CreateCapsuleRequest request,
            Authentication auth
    ) {
        String ownerId = currentUserId(auth);
        return ResponseEntity.ok(capsuleService.create(ownerId, request));
    }

    @PostMapping("/{id}/unlock")
    public ResponseEntity<CapsuleResponse> unlock(@PathVariable String id, Authentication auth) {
        String ownerId = currentUserId(auth);
        return ResponseEntity.ok(capsuleService.unlockCapsule(id, ownerId));
    }
}
```

V kódu je rovněž patrné využití objektu Authentication, který kontrolerům umožňuje bezpečně extrahovat identifikátor aktuálně přihlášeného uživatele přímo z bezpečnostního kontextu Spring Security. Tento postup odstraňuje nutnost zasílat ID uživatele zranitelnou cestou z klienta.

7.1.4 Servisní vrstva a business logika

Jádro aplikace a veškerá doménová pravidla jsou plně izolována v servisních třídách (balíček **services**), jako je například CapsuleService. Tato vrstva je zcela nezávislá na HTTP protokolu, což usnadňuje její testovatelnost a potenciální znovupoužitelnost.

Tvorba nového obsahu

Metoda **create** demonstruje přípravu entity před jejím uložením do databáze. Systém zde provádí normalizaci textových vstupů (odstranění prázdných znaků), výpočet správných časových značek pro odemčení a expiraci (unlockAt, expiresAt) a případné vygenerování privátního sdílecího tokenu (shareToken), pokud uživatel zvolil viditelnost kapsle jako SHARED:

Výpis 2 Metoda pro vytvoření nové časové kapsle

```
public CapsuleResponse create(String ownerId, CreateCapsuleRequest request) {
    validateCapsuleRequest(request);
    CapsuleVisibility visibility = CapsuleVisibility.fromValue(request.getVisibility());
    CapsuleStatus status = resolveCreateStatus(request);
    Instant unlockAt = normalizeUnlockAt(status, request.getUnlockAt());
    Instant expiresAt = normalizeExpiresAt(status, unlockAt, request.getExpiresAt());
    Capsule capsule = new Capsule();
    capsule.setOwnerId(new ObjectId(ownerId));
    capsule.setTitle(request.getTitle().trim());
    capsule.setBody(normalizeBlankToNull(request.getBody()));
    capsule.setVisibility(visibility);
    capsule.setStatus(status);
    capsule.setUnlockAt(unlockAt);
    capsule.setExpiresAt(expiresAt);
    capsule.setTags(normalizeTags(request.getTags()));
    if (CapsuleVisibility.SHARED.equals(visibility) && !status.isDraft()) {
        capsule.setShareToken(generateShareToken());
    }
    return toResponse(capsuleRepository.save(capsule));
}
```

Zajištění konzistence dat při odemykání

Další kritickou operací v této vrstvě je bezpečné odemčení kapsle. Kapsle může být odemčena uživatelem nebo automatickým plánovačem. Aby systém předešel souběhu dat (race conditions) v distribuovaném prostředí, využívá metoda unlockCapsule atomickou databázovou operaci findAndModify prostřednictvím knihovny MongoTemplate:

Výpis 3 Atomické odemčení časové kapsle pomocí MongoTemplate

```
public CapsuleResponse unlockCapsule(String id, String ownerId) {
    Query query = new Query(
        Criteria.where("_id").is(id)
            .and("ownerId").is(new ObjectId(ownerId))
            .and("deletedAt").is(null)
            .and("status").is(CapsuleStatus.SEALED.getValue())
            .and("unlockAt").lte(Instant.now())
    );
    Update update = new Update()
        .set("status", CapsuleStatus.OPENED.getValue())
        .set("openedAt", Instant.now())
        .set("updatedAt", Instant.now());
    Capsule updated = mongoTemplate.findAndModify(
        query,
        update,
        FindAndModifyOptions.options().returnNew(true),
        Capsule.class
    );
    if (updated == null) {
        throw new IllegalArgumentException("Capsule cannot be unlocked yet or not found");
    }
    notifyOwner(ownerId, updated);
    return toResponse(updated);
}
```

Díky explicitně definovaným podmínkám v objektu Query (včetně kontroly unlockAt a deletedAt) je zaručeno, že stav kapsle se změní pouze tehdy, pokud plně vyhovuje všem časovým a bezpečnostním kritériím.

7.1.5 Přístup k databázi

Komunikace s databázovým clusterem MongoDB je realizována pomocí abstrakce Spring Data MongoDB. Architektura datové vrstvy kombinuje vysokou úroveň abstrakce pro standardní operace s nízkoúrovňovým přístupem pro kritické a výkonnostně náročné dotazy.

Využití deklarativních repozitářů

Základní manipulace s daty probíhá přes rozhraní dědící z MongoRepository (balíček repositories). Implementace se záměrně vyhýbá psaní explicitních dotazů v jazyce JSON; namísto toho využívá mechanismus odvozených dotazů (derived queries). Tento přístup umožňuje frameworku generovat optimální dotazy pro MongoDB přímo z názvů metod v Javě.

Klíčovým architektonickým prvkem je implementace mechanismu „soft delete“. Všechny metody v rozhraní CapsuleRepository jsou definovány s podmínkou DeletedAtIsNull. Tato podmínka na úrovni datové vrstvy garantuje, že logicky smazané záznamy nebudou načteny a zobrazeny uživatelům.Následující výpis ukazuje kompletní definici tohoto rozhraní:

Výpis 4 Rozhraní CapsuleRepository pro práci s kapslemi

```
package com.oleksandrmytro.timecapsule.repositories;

import com.oleksandrmytro.timecapsule.models.Capsule;
import org.bson.types.ObjectId;
import org.springframework.data.mongodb.repository.MongoRepository;

import java.util.List;
import java.util.Optional;

public interface CapsuleRepository extends MongoRepository<Capsule, String> {
    List<Capsule> findByOwnerIdAndDeletedAtIsNullOrderByCreatedAtDesc(ObjectId ownerId);
```

```
    Optional<Capsule> findByIdAndOwnerIdAndDeletedAtIsNull(String id, ObjectId ownerId);
    Optional<Capsule> findByIdAndDeletedAtIsNull(String id);
    List<Capsule> findByVisibilityAndDeletedAtIsNullOrderByCreatedAtDesc(String visibility);
    long countByOwnerIdAndDeletedAtIsNull(ObjectId ownerId);
    long countByOwnerIdAndVisibilityAndDeletedAtIsNull(ObjectId ownerId, String visibility);
    List<Capsule> findByOwnerIdAndVisibilityAndDeletedAtIsNullOrderByCreatedAtDesc(ObjectId
ownerId, String visibility);
}
```

Cílené operace přes MongoTemplate

Pro složitější scénáře, které vyžadují flexibilnější podmínky (například porovnávání dynamického času s uloženým datem odemčení) nebo explicitní kontrolu nad updaty, systém využívá třídu MongoTemplate.

Dotazy jsou v tomto případě konstruovány manuálně pomocí objektů Query a Criteria. To umožňuje řešit pokročilé požadavky:

- **Atomické aktualizace:** Provedení vyhledání a úpravy stavu kapsle jako jedné nedělitelné operace (findAndModify, updateFirst).
- **Práce s distribučními klíči (Shard Keys):** Do dynamicky tvořeného objektu Query je vždy explicitně přidán shardovací klíč (např. ownerId). Databázový směrovač (router) tak přesně ví, na kterém uzlu clusteru se data nacházejí. Díky tomu systém efektivně předchází nechtěnému rozesílání dotazů na všechny shardy (tzv. scatter-gather problému).

7.1.6 Real-time komunikace pomocí WebSocket a STOMP

Pro funkce vyžadující okamžitou odezvu (chat, notifikace) byla zvolena technologie WebSocket, která na rozdíl od HTTP umožňuje trvalé obousměrné spojení. Nad tímto protokolem běží nadstavba STOMP (Simple Text Oriented Messaging Protocol) zajišťující směrování zpráv.

Architektura a zabezpečení kanálu

Jak dokazuje konfigurace ve třídě WebSocketConfig, systém využívá vlastní interceptory pro dvoufázové ověření uživatele (během HTTP handshaku a následně při každé STOMP zprávě), čímž obchází omezení prohlížečů, které neumí do WebSocketa vložit HTTP hlavičky:

Výpis 5 Konfigurace WebSocket komunikace ve třídě WebSocketConfig

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic", "/queue");
        config.setUserDestinationPrefix("/user");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns(allowedOriginPatterns)
            // Fáze 1: Extrakce JWT z HttpOnly cookie během handshaku
            .addInterceptors(cookieHandshakeInterceptor)
            .withSockJS();
    }
```

```
    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        // Fáze 2: Validace JWT tokenu pro každou příchozí zprávu
        registration.interceptors(webSocketJwtChannelInterceptor);
    }
}
```

Směrování zpráv uživatelům (Push model)

Backend aplikuje push model – server sám iniciuje odeslání dat klientovi. Logika odesílání je zapouzdřena do specializovaných služeb. Například při změně stavu kapsle systém generuje událost, kterou CapsuleNotificationService odešle přes SimpMessagingTemplate přímo do soukromé fronty konkrétního uživatele:

Výpis 6 Odesílání real-time událostí pomocí SimpMessagingTemplate

```
@Service
public class CapsuleNotificationService {
    private final SimpMessagingTemplate messagingTemplate;

    public CapsuleNotificationService(SimpMessagingTemplate messagingTemplate) {
        this.messagingTemplate = messagingTemplate;
    }

    public void sendStatus(String userId, CapsuleStatusEvent event) {
        messagingTemplate.convertAndSendToUser(
            userId,
            "/queue/capsules/status",
            event
        );
    }
}
```

Stejný princip je prokazatelně využit v modulu chatu (ChatService). Po uložení nové zprávy do databáze je zkonstruován datový objekt (payload), který se okamžitě doručí adresátovi (přes convertAndSendToUser) bez nutnosti, aby se klient dotazoval serveru:

Výpis 7 Doručení chatové zprávy příjemci pomocí WebSocket

```
private Map<String, Object> deliver(ChatMessage saved, String senderId, String peerId) {
    Map<String, Object> payloadToPeer = new HashMap<>();
    payloadToPeer.put("id", saved.getId());
    payloadToPeer.put("type", saved.getType());
    payloadToPeer.put("text", saved.getText() == null ? "" : saved.getText());
    payloadToPeer.put("fromUserId", senderId);
    payloadToPeer.put("fromMe", false);
    payloadToPeer.put("timestamp", saved.getCreatedAt().toString());

    // Okamžité doručení do relace příjemce
    messagingTemplate.convertAndSendToUser(peerId, "/queue/chat", payloadToPeer);

    Map<String, Object> payloadToSender = new HashMap<>(payloadToPeer);
    payloadToSender.put("fromMe", true);
    return payloadToSender;
}
```

7.1.7 Scheduler a automatické otevírání kapslí

Klíčovou doménovou funkcí systému TimeCapsule je garantované odemčení digitálního obsahu v přesně stanovený čas. Zatímco uživatel může kapsli odemknout manuálně přes REST API, systém musí být schopen proces provést plně autonomně. Tuto roli plní třída

CapsuleUnlockScheduler, spouštěná na pozadí pomocí anotace @Scheduled s konfigurovatelným intervalem (standardně každých 5000 milisekund).

Návrh tohoto plánovače musel reflektovat dvě zásadní inženýrské výzvy: ochranu operační paměti při velkém množství současně expirujících kapslí a prevenci konfliktů (race conditions) v distribuovaném prostředí. Podrobný průběh automatického otevření kapsle je vzhledem k rozsahu uveden v příloze B.

Dávkové zpracování (Batch Processing)

Aby se předešlo vyčerpání paměti (OutOfMemoryError) a zahlcení databáze, systém nečte všechny připravené kapsle najednou. Místo toho využívá cyklus, který zpracovává data po striktně omezených dávkách (batchSize). Metoda unlockReadyBatch načte pouze určitý limit záznamů seřazených od nejstaršího (Sort.Direction.ASC, "unlockAt"). Pokud je vrácena plná dávka, plánovač ví, že v databázi pravděpodobně čekají další záznamy, a cyklus opakuje, dokud nenarazí na limit maxBatchesPerRun.

Atomické operace a prevence souběhu V moderních cloudových prostředích může aplikace běžet ve více instancích (horizontální škálování). Pokud by dvě instance spustily plánovač ve stejnou milisekundu, mohly by se pokusit odemknout stejnou kapsli. Systém tomuto předchází tím, že nevyužívá standardní postup "načíst-upravit-uložit" (save()), ale rovnou odesílá do MongoDB atomický příkaz updateFirst:

Výpis 8 Dávkové automatické otevírání kapslí v plánovači

```
private boolean unlockReadyBatch(Instant now) {
    // 1. Načtení dávky (s limitem)
    Query query = new Query(
        Criteria.where("status").is(CapsuleStatus.SEALED.getValue())
            .and("deletedAt").is(null)
            .and("unlockAt").lte(now)
    );
    query.with(Sort.by(Sort.Direction.ASC, "unlockAt"));
    query.limit(batchSize);

    List<Capsule> ready = mongoTemplate.find(query, Capsule.class);
    if (ready.isEmpty()) return false;

    for (Capsule capsule : ready) {
        // 2. Atomická úprava: Změní stav pouze tehdy, pokud je v databázi stále SEALED
        UpdateResult result = mongoTemplate.updateFirst(
            new Query(Criteria.where("_id").is(capsule.getId())
                .and("status").is(CapsuleStatus.SEALED.getValue())),
            new Update()
                .set("status", CapsuleStatus.OPENED.getValue())
                .set("openedAt", now),
            Capsule.class
        );

        // 3. Kontrola modifikace: Pokud záznam nezměnil tento proces, notifikace se neodesílá
        if (result.getModifiedCount() == 0) continue;

        // 4. Rozeslání událostí
        CapsuleStatusEvent event = new CapsuleStatusEvent(
            capsule.getId(), CapsuleStatus.OPENED.getValue(), false,
            capsule.getUnlockAt(), now, capsule.getTags()
        );
        userRepository.findById(capsule.getOwnerId().toHexString())
            .ifPresent(user -> notificationService.sendStatus(user.getId(), event));

        emailService.sendCapsuleOpened(capsule.getOwnerId().toHexString(), capsule);
```

```
    }

    // Návratová hodnota řídí, zda se má spustit další iterace (dávka)
    return ready.size() == batchSize;
}
```

Objekt Query v metodě updateFirst funguje jako takzvaný optimistický zámek (optimistic lock). Do podmínky where je kromě ID znovu vložena kontrola status == SEALED. Pokud jiná instance serveru kapsli odemkla o zlomek sekundy dříve, databáze v tomto kroku nenajde žádný odpovídající dokument (protože stav už je OPENED). Metoda result.getModifiedCount() pak vrátí 0. Díky kontrolní podmínce if (result.getModifiedCount() == 0) continue; je absolutně vyloučeno, aby systém odeslal uživateli duplicitní notifikaci nebo dvakrát spustil proces odemčení.

7.1.8 Bezpečnost a autorizace

Bezpečnostní architektura aplikačního rozhraní je navržena jako plně bezestavová (stateless) a opírá se o standard JSON Web Token (JWT). Toto rozhodnutí umožňuje snadné horizontální škálování backendu, jelikož server nemusí udržovat relace (sessions) uživatelů v operační paměti.

Konfigurace bezpečnostního kontextu

Základní pravidla přístupu jsou definována ve třídě SecurityConfiguration. Konfigurace explicitně vypíná výchozí mechanismy ochrany relací, jako jsou formulářové přihlašování (formLogin) a HTTP Basic autentizace, a nastavuje politiku vytváření relací na STATELESS. Přístup k datům je rozdělen pomocí selektorů: veřejné endpointy (např. /api/auth/**, čtení veřejných kapslí) jsou povoleny všem (permitAll()), zatímco modifikační operace vyžadují platnou autentizaci (authenticated()):

Výpis 9 Konfigurace zabezpečení REST API pomocí SecurityFilterChain

```
@Bean
@Order(2)
public SecurityFilterChain apiSecurityFilterChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher("/api/**")
        .csrf(AbstractHttpConfigurer::disable)
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .formLogin(AbstractHttpConfigurer::disable)
        .httpBasic(AbstractHttpConfigurer::disable)
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**", "/api/hello").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/tags/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/capsules/*").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/users/**").permitAll()
            .anyRequest().authenticated()
        )
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authenticationProvider(authenticationProvider)
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

Do zpracování HTTP požadavku je vložen vlastní JwtAuthenticationFilter, který se spouští před standardním UsernamePasswordAuthenticationFilter. Tento filtr extrahuje a validuje token z každého požadavku a následně plní SecurityContextHolder identitou uživatele.

Delegovaná autentizace a interní JWT

Aplikace podporuje standardní přihlašování heslem i delegovanou autentizaci přes OAuth2 poskytovatele (Google, GitHub). Systém využívá sofistikovaný architektonický vzor: neudržuje OAuth2 relaci s externím poskytovatelem, ale používá jej pouze pro jednorázové ověření identity.

Integrace poskytovatelů a identifikace uživatele

Proces začíná přesměrováním na autorizační server poskytovatele (např. accounts.google.com nebo github.com/login/oauth/authorize). Po úspěšném ověření obdrží aplikace atributy uživatele (email, jméno, avatar, unikátní ID poskytovatele).

Klíčovou výzvou je spárování externí identity s interním uživatelským účtem. Systém implementuje víceúrovňovou logiku vyhledávání:

- **Primární identifikace:** Vyhledání podle kombinace registrationId a providerId. To je nejspolehlivější metoda, která není závislá na změně e-mailové adresy uživatele.
- **Sekundární identifikace (Fallback):** Pokud uživatel s daným ID poskytovatele není nalezen, provede se vyhledání podle e-mailové adresy. Pokud existuje účet se stejným e-mailem, je k němu externí poskytovatel automaticky připojen (Account Linking).
- **Vytvoření profilu:** Pokud identita není nalezena, je vytvořen nový uživatelský profil.

Jakmile je uživatel úspěšně ověřen poskytovatelem, zasáhne vlastní OAuth2AuthenticationSuccessHandler, který dokončí proces a vygeneruje interní JWT tokeny:

Výpis 10 Zpracování OAuth2 přihlášení a vytvoření autentizačních tokenů

```
@Override
public void onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response,
Authentication authentication)
        throws IOException, ServletException {
    OAuth2AuthenticationToken oauthToken = (OAuth2AuthenticationToken) authentication;
    OAuth2User oauth2User = (OAuth2User) authentication.getPrincipal();
    String registrationId = oauthToken.getAuthorizedClientRegistrationId();

    // Logika vyhledání/vytvoření uživatele (ID poskytovatele vs Email)
    Optional<User> existingByOAuth =
userRepository.findByAuthProvidersProviderAndAuthProvidersProviderId(registrationId,
oauth2User.getName());
    Optional<User> existingByEmail = existingByOAuth.isEmpty() ?
userRepository.findByEmail(extractEmail(oauth2User)) : Optional.empty();

    User user = existingByOAuth.orElse(existingByEmail.orElseGet(() -> createNewUser(oauth2User,
registrationId)));

    saveUserRespectingShardKey(user);
    LoginResponse tokens = buildTokens(user);

    if (shouldRedirect(request)) {
        sendRedirectWithCookies(request, response, redirectUri, tokens);
    } else {
        authCookieService.writeAuthCookies(request, response, tokens);
        writeJsonResponse(response, tokens);
    }
}
```

Tím je dosaženo sjednocení autorizačního mechanismu – z pohledu zbytku aplikace (kontrolery, služby, WebSockety) nezáleží na tom, zda se uživatel přihlásil heslem nebo přes Google; vždy je autorizován jednotným interním JWT tokenem.

Ochrana tokenů (HttpOnly Cookies)

Zásadním bezpečnostním rozhodnutím je způsob doručování tokenů frontendové aplikaci. Aby se zabránilo krádeži identity pomocí útoků typu Cross-Site Scripting (XSS) (kdy by škodlivý JavaScript mohl token přečíst z localStorage), jsou tokeny ukládány do HTTP cookies. Logika ve třídě AuthCookieService nastavuje těmto cookies klíčové bezpečnostní vlajky:

Výpis 11 Nastavení bezpečnostních atributů autentizační cookie

```
private String buildCookie(HttpServletRequest request, String name, String value, int
maxAgeSeconds, String path) {
    return name + "=" + value
        + "; HttpOnly"
        + "; SameSite=" + resolveSameSite(request, isSecureRequest(request))
        + "; Path=" + path
        + "; Max-Age=" + maxAgeSeconds
        + (isSecureRequest(request) ? "; Secure" : "");
}
```

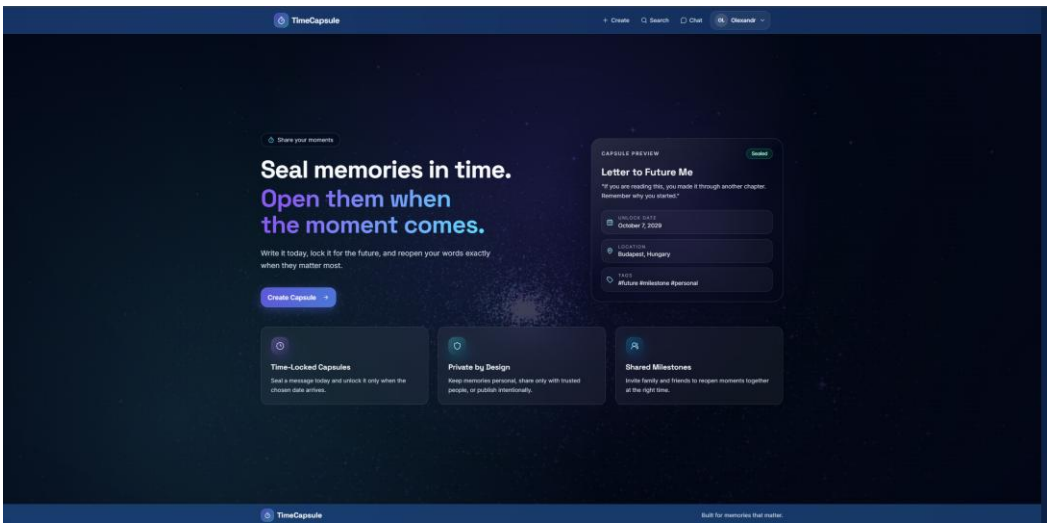
- **HttpOnly:** Zabraňuje přístupu ke cookie z klientských skriptů. Tím výrazně omezuje riziko krádeže tokenu přes XSS.
- **SameSite:** Poskytuje obranu proti útokům Cross-Site Request Forgery (CSRF) tím, že omezuje odesílání cookie na požadavky pocházející ze stejné domény.
- **Secure:** Garantuje, že cookie bude odeslána pouze přes šifrované HTTPS spojení.

Vlastní JwtAuthenticationFilter je proto naprogramován tak, aby v případě chybějící HTTP hlavičky Authorization automaticky vyhledal token v těchto zabezpečených cookies. Tento přístup umožňuje plynulou a bezpečnou komunikaci s frontendem.

7.2 Implementace frontendové části

Klientská část aplikace je vyvinuta jako moderní Single Page Application (SPA) pomocí knihovny React a sestavovacího nástroje Vite. Architektura je navržena s důrazem na modularitu, rychlost načítání (výkon) a typovou bezpečnost díky integraci TypeScriptu.

Uživatelské rozhraní je rozděleno do několika částí: úvodní stránka, správa kapslí, profil uživatele, mapa, chat a administrační část. Úvodní obrazovka představuje základní myšlenku systému Time Capsule a nabízí uživateli rychlý přístup k hlavním funkcím aplikace. Její výsledná podoba je znázorněna na Obrázku 16.



Obrázek 16 Hlavní stránka (Landing page) aplikace představující základní funkce
Zdroj: (vlastní zpracování)

Po představení výsledné vizuální podoby aplikace se následující část zaměřuje na vnitřní organizaci frontendového projektu.

7.2.1 Struktura frontendové části

Organizace souborů v klientské aplikaci reflektuje snahu o striktní oddělení UI komponent, byznys logiky a komunikačních služeb. Detailní výpis adresářové struktury frontendu je k dispozici v Příloze A

Toto rozdělení umožňuje vysokou znovupoužitelnost kódu. Například komponenty v components/ui/ jsou čistě prezentační a nezávislé na datech, zatímco složka services/ izoluje veškerou logiku síťové komunikace od samotného vykreslování.

7.2.2 Komponenty a routing

Klientská aplikace je vyvinuta jako Single Page Application (SPA) pomocí knihovny React. Veškerá navigace mezi obrazovkami probíhá na straně klienta bez nutnosti znovunačítání stránky. Tuto navigaci zajišťuje knihovna react-router-dom.

Optimalizace výkonu pomocí Lazy Loading

Vzhledem k přítomnosti "těžkých" komponent (např. mapový engine Cesium nebo editory s bohatým textem) je kritickým požadavkem minimalizace velikosti úvodního JavaScriptového balíčku (bundlu). Systém proto nevyužívá standardní statické importy pro všechny stránky, ale aplikuje techniku Code Splitting přes funkce React.lazy a Suspense. Komponenty stránek jsou stahovány ze serveru asynchronně až v okamžiku, kdy uživatel na danou URL adresu skutečně přejde:

Výpis 12 Konfigurace routingu a lazy loadingu ve frontendové části

```
import { lazy, Suspense } from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';

// Asynchronní načítání "těžkých" komponent
const LazyCreateCapsuleForm = lazy(() => import('./components/capsules/create-capsule-
form').then(m => ({ default: m.CreateCapsuleForm })));
const LazyCapsulesMapView = lazy(() => import('./components/capsules/capsules-map-view').then(m
=> ({ default: m.CapsulesMapView })));

export const AppRoutes = ({ isAuthenticated, isAdmin }) => (
    <Suspense fallback={<GlobalPageLoader />}>
        <Routes>
            <Route element={<MainLayout />}>
                <Route path="/" element={<Home />} />
                <Route path="/map" element={<LazyCapsulesMapView />} />

                {/* Chráněná cesta s podmíněným vykreslením a ochranou proti návratu */}
                <Route
                    path="/create"
                    element={isAuthenticated ? <LazyCreateCapsuleForm /> : <Navigate to="/login"
replace />}
                />
            </Route>
        </Routes>
    </Suspense>
);
```

Použití komponenty <Navigate replace /> efektivně řeší bezpečnost uživatelského rozhraní – zajišťuje, že se neautorizovaný uživatel nemůže vrátit zpět na chráněnou stránku pořízením zpětného kroku v historii prohlížeče.

Kompozice rozložení (Layout Components)

Jak ukazuje předchozí kód, systém hojně využívá koncept tzv. vnořeného směrování (nested routing). Místo toho, aby každá stránka musela explicitně importovat navigační lištu a patičku, je v aplikaci definována komponenta MainLayout. Ta slouží jako architektonická obálka pro sdílené prvky uživatelského rozhraní.

Tato komponenta využívá speciální element <Outlet /> z React Routeru, který funguje jako dynamický zástupný znak. Router do tohoto místa automaticky "vstříkne" podkomponentu na základě aktuální URL adresy. Tím se výrazně redukuje duplicita kódu:

Výpis 13 Komponenta MainLayout pro společné rozvržení aplikace

```
import { Outlet } from 'react-router-dom';
import { NavigationBar } from '../ui/navigation-bar';
import { Sidebar } from '../ui/sidebar';

export const MainLayout: React.FC = () => {
    return (
        <div className="app-container flex min-h-screen">
            <Sidebar />
            <div className="main-content flex-grow flex flex-col">
                <NavigationBar />
                <main className="content-wrapper p-6 flex-1">
                    {/* Místo, kam react-router vykreslí konkrétní stránku (např.
LazyCreateCapsuleForm) */}
                    <Outlet />
                </main>
            </div>
        </div>
    );
};
```

Tímto spojením techniky Lazy Loading a vnořených layoutů aplikace dosahuje nejen vynikajících metrik načítání (Time to Interactive), ale udržuje si i velmi čistou a modulární strukturu komponentového stromu.

7.2.3 Správa stavu a práce s hooky

V aplikaci TimeCapsule je správa stavu řešena pomocí nativních React hooků, přičemž architektura rozlišuje mezi lokálním stavem formulářů a globálním stavem aplikace. Pro klíčové globální entity (např. uživatelský profil, relace, seznam kapslí) byl zvolen vzor "Lifting State Up", kdy je stav spravován na nejvyšší úrovni v komponentě App.tsx a předáván níže.

Základem tohoto přístupu je využití hooku useState pro definici datových struktur a useEffect pro inicializaci aplikace. Při prvním načtení aplikace (tzv. bootstrap) systém asynchronně ověřuje platnost relace a stahuje profil uživatele:

Výpis 14 Správa stavu uživatelské relace pomocí hooků useState a useEffect

```
function App() {
  const [profile, setProfile] = useState<UserProfile | null>(null);
  const [tokens, setTokens] = useState<{ session: string } | null>(null);
  const [error, setError] = useState<ApiError | null>(null);
  const [capsules, setCapsules] = useState<Capsule[]>([]);

  // Funkce pro ustavení aktivní relace a načtení profilu
  const establishSession = async (): Promise<UserProfile> => {
    const data = await getCurrentUser();
    if (!isValidProfile(data)) throw { status: 401, message: 'Invalid profile response' };
    setProfile(data);
```

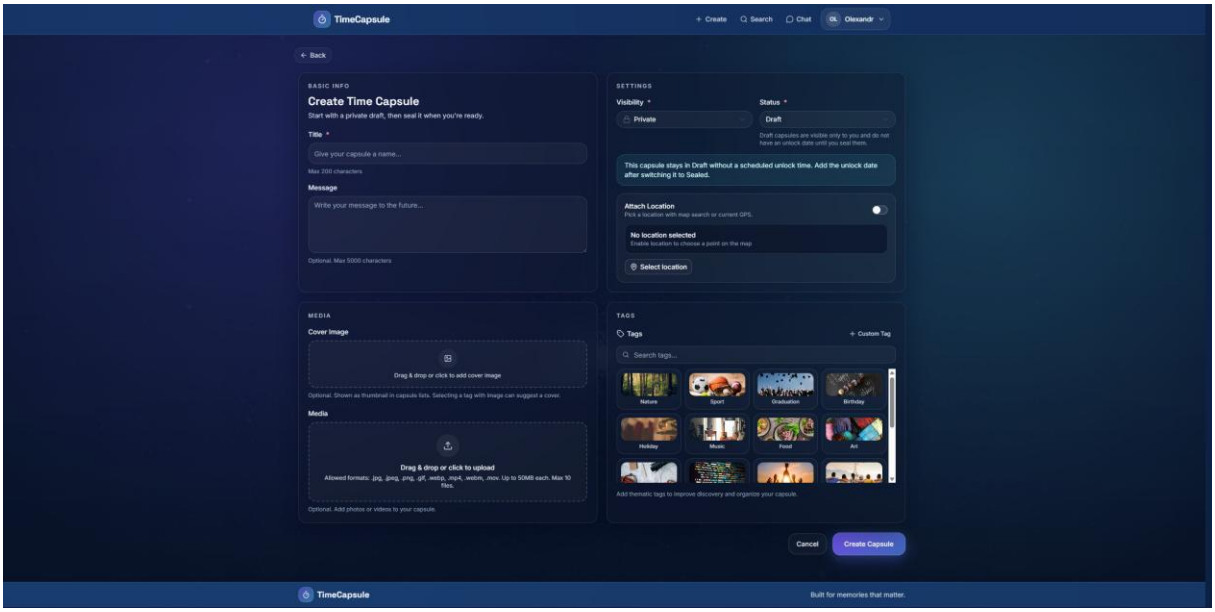
```
    setTokens({ session: 'cookie' }); // Potvrzení, že backend spravuje relaci přes HttpOnly
cookie
    return data;
  };

  const bootstrapSession = async () => {
    const data = await apiRequest('/api/auth/session', { method: 'GET' });
    if (data?.authenticated) {
      await establishSession();
      return;
    }
    setTokens(null);
    setProfile(null);
  };
  // Spuštění inicializace při prvním vykreslení aplikace
  useEffect(() => {
    if (location.pathname === '/auth/oauth2/redirect') return;
    bootstrapSession()
      .catch(() => { setTokens(null); setProfile(null) })
      .finally(() => setSessionResolved(true));
  }, []);
  // ... zbytek komponenty
}
```

Tato inicializační logika (tzv. side-effect izolovaný v useEffect) efektivně řeší přetrvávající přihlášení (persistence). Aplikace navíc ihned po úspěšném ověření zaznamenává, že tokeny jsou spravovány prostřednictvím cookies (session: 'cookie'), což reflektuje dříve zmíněnou ochranu proti XSS útokům popsanou v backendové části.

7.2.4 Formuláře a práce s daty

Jednou z hlavních interakčních částí aplikace je formulář pro vytvoření nové časové kapsle. Uživatel v něm vyplňuje základní obsah kapsle, nastavuje její viditelnost, čas odemčení, geolokaci a případné mediální přílohy. Výsledné uživatelské rozhraní tohoto formuláře je znázorněno na Obrázku 17.



Obrázek 17 Komplexní formulář pro vytvoření nové časové kapsle s nastavením viditelnosti a času odemčení
Zdroj: (vlastní zpracování)

Zpracování uživatelských vstupů je v aplikaci striktně kontrolováno na straně klienta dříve, než jsou data odeslána na server. Komponenty, jako je CreateCapsuleForm, využívají komplexní manuální validaci a transformaci dat.

Klíčovým architektonickým rozhodnutím je zachycení události formuláře a potlačení výchozího chování prohlížeče (e.preventDefault()). Následná asynchronní metoda handleSubmit kontroluje povinná pole, platnost datových typů a obchodní logiku – například ověření, zda je čas odemčení kapsle nastaven do budoucnosti:

Výpis 15 Zpracování formuláře pro vytvoření časové kapsle

```
async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
  e.preventDefault();
  setLocalError(null);

  const trimmedTitle = title.trim();
  if (!trimmedTitle) {
    setLocalError({ status: 0, message: "Title is required" });
    return;
  }

  const now = new Date();
  let unlockDate: Date | null = null;

  // Byznys logika pro zapečetěné kapsle s časovačem
  if (requiresUnlockSchedule) {
    if (!unlockAt) {
      setLocalError({ status: 0, message: "Unlock date is required" });
      return;
    }
    unlockDate = new Date(unlockAt);
    if (Number.isNaN(unlockDate.getTime())) {
      setLocalError({ status: 0, message: "Unlock date is invalid" });
      return;
    }
    if (unlockDate <= now) {
      setLocalError({ status: 0, message: "Unlock date must be in the future for sealed capsules"
});
      return;
    }
  }

  // Příprava dat (DTO) pro REST API a řízení stavu UI
  setIsLoading(true);
  try {
    const savedCapsule = await onSubmit({
      title: trimmedTitle,
      body: body.trim() || null,
      visibility,
      status,
      unlockAt: requiresUnlockSchedule && unlockDate ? unlockDate.toISOString() : null,
      // ... mapování dalších vlastností
    });

    if (!onSubmitted) {
      navigate(isEditMode ? `/capsules/${savedCapsule.id}` : "/account");
    }
  } catch (err: any) {
    setLocalError(err);
  } finally {
    setIsLoading(false);
  }
}
```

Tento úsek kódu demonstruje robustní manipulaci se stavy (setIsLoading, setLocalError). Tím, že aplikace převádí lokální data na standardizovaný formát ISO 8601 (toISOString()), zajišťuje bezproblémovou interoperabilitu s Java backendem nezávisle na časovém pásmu uživatele.

7.2.5 Integrace mapy

Jednou z technicky nejnáročnějších částí frontendové aplikace je modul pro geografickou vizualizaci kapslí (CapsulesMapView). Architektura této komponenty je unikátní tím, že podporuje dva vizualizační režimy: 2D mapu (využívající knihovnu Leaflet) a 3D prostorový glóbus (využívající CesiumJS).

Vzhledem k tomu, že knihovna Cesium je datově extrémně objemná, bylo nutné předejít situaci, kdy by stahování tohoto balíčku blokovalo načtení zbytku aplikace. Systém to řeší asynchronním dynamickým importem, který se spustí uvnitř hooku useEffect pouze v případě, že uživatel aktivuje režim "globe":

Výpis 16 Dynamické načítání 3D glóbu pomocí knihovny Cesium

```
useEffect(() => {
  let disposed = false;
  if (mode !== "globe") return;
  if (!globeHostEl) return;
  if (cesiumViewerRef.current || globeInitInFlightRef.current) return;

  // Asynchronní inicializace 3D mapy s ochranou proti vícenásobnému spuštění
  (async () => {
    globeInitInFlightRef.current = true;
    setIsGlobeInitializing(true);

    // Dynamický import knihovny Cesium a jejích stylů
    const [{ default: _unusedCss }, Cesium] = await Promise.all([
      import("cesium/Build/Cesium/Widgets/widgets.css"),
      import("cesium"),
    ]);

    if (disposed) return;
    cesiumRef.current = Cesium;

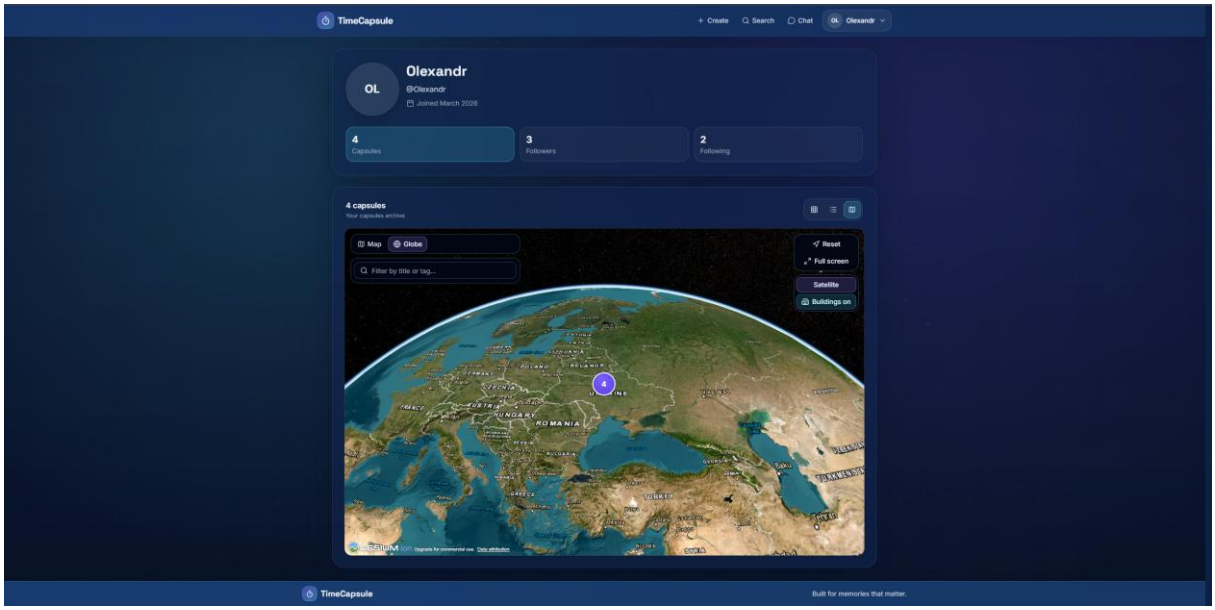
    // Vytvoření instance Cesium Viewer
    const viewer = new Cesium.Viewer(globeHostEl, {
      animation: false,
      timeline: false,
      geocoder: false,
      baseLayerPicker: false,
      requestRenderMode: true,
    });

    cesiumViewerRef.current = viewer;
    setGlobeReady(true);
  })();

  // Cleanup funkce pro uvolnění prostředků při zničení komponenty
  return () => { disposed = true; };
}, [mode, globeHostEl]);
```

Pro 2D režim (mode === "map") je podobným způsobem inicializován Leaflet, přičemž kód obsahuje explicitní logiku pro čištění interního stavu mapy (např. delete container._leaflet_id), aby nedocházelo ke kolizím při opětovném vykreslování komponenty (re-rendering). Aplikace navíc využívá technologii tzv. shlukování (clustering) přes markerClusterGroup. Tato technika shlukování zajišťuje plynulý běh uživatelského rozhraní i při zobrazení stovek kapslí na jedné mapě.

Výsledkem této implementace je uživatelské rozhraní, ve kterém lze kapsle zobrazovat nejen v klasické 2D mapě, ale také v interaktivním 3D glóbu. Tento režim je součástí uživatelského profilu a umožňuje vizuálně prezentovat geografickou polohu vytvořených kapslí, jak ukazuje Obrázek 18.



Obrázek 18 Uživatelský profil s interaktivním 3D glóbem a zobrazením lokace vytvořených kapslí
Zdroj: (vlastní zpracování)

7.2.6 Notifikace a WebSocket klient

Pro zajištění okamžité odezvy uživatelského rozhraní na asynchronní události ze serveru implementuje aplikace plnohodnotného STOMP klienta běžícího nad protokolem WebSocket. Komunikační vrstva je zapouzdřena ve službě ws.ts a využívá knihovny @stomp/stompjs v kombinaci se sockjs-client jako fallback mechanismem.

Architektura real-time aktualizací

Klíčovým případem užití WebSocketů v této aplikaci není samotná modifikace dat, ale real-time synchronizace stavu. Například proces odemčení kapsle je plně v režii backendu (úloha CapsuleUnlockScheduler). Jakmile server změní stav kapsle v databázi, odešle cílovému uživateli událost přes STOMP kanál.

Frontendový klient se po připojení přihlašuje k odběru této privátní fronty /user/queue/capsules/status. Po přijetí zprávy aplikace automaticky iniciuje znovunačtení dat, například voláním metody pro načtení kapslí. Tento mechanismus vede k okamžité aktualizaci uživatelského rozhraní bez nutnosti manuálního obnovení stránky prohlížeče:

Výpis 17 Připojení klienta k WebSocket streamům notifikací a chatu

```
export function connectCapsuleStream({ onEvent, onError }: CapsuleStreamCallbacks): void {
  if (client) return;

  client = new Client({
    // Využití SockJS s HttpOnly cookies pro autentizaci spojení
    webSocketFactory: () => new SockJS(getWsUrl(), undefined, { withCredentials: true } as any),
    reconnectDelay: 5000,
```

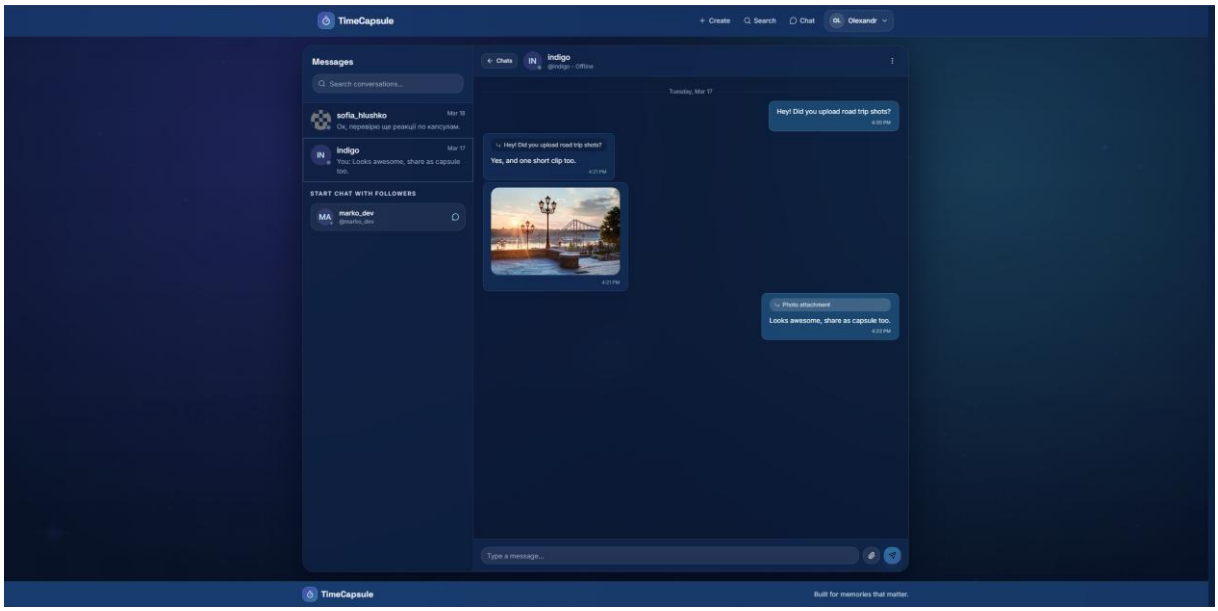
```
    heartbeatIncoming: 10000,
    heartbeatOutgoing: 10000,
    onConnect: () => {
      connected = true;

      // Odběr systémových notifikací o stavu kapslí
      capsuleSub = client!.subscribe('/user/queue/capsules/status', (msg: IMessage) => {
        try {
          const body = JSON.parse(msg.body);
          onEvent?.(body); // Spouští aktualizaci UI (např. loadCapsules)
        } catch (e) {
          console.error('WS parse error', e);
        }
      });

      // Odběr pro real-time modul chatu
      chatSub = client!.subscribe('/user/queue/chat', (msg: IMessage) => {
        try {
          const body = JSON.parse(msg.body) as ChatWsMessage;
          chatCallbacks.onMessage?.(body);
        } catch (e) {
          console.error('WS chat parse error', e);
        }
      });
    },
    // ... ošetření chybových stavů
  });
  client.activate();
}
```

Toto architektonické rozhodnutí (event-driven UI update) se ukázalo jako vysoce efektivní. Alternativní přístupy, jako je neustálé dotazování serveru (polling), by neúměrně zatěžovaly síťovou infrastrukturu, zatímco spoléhání se na manuální obnovení stránky by degradovalo uživatelský zážitek (UX).

Stejná WebSocket infrastruktura je využita také pro modul soukromého chatu. Díky odběru uživatelské fronty může aplikace zobrazovat nové zprávy bez manuálního obnovení stránky. Výsledné rozhraní real-time chatu je znázorněno na Obrázku 19.



Obrázek 19 Rozhraní modulu pro real-time chat a soukromé zprávy mezi uživateli
Zdroj: (vlastní zpracování)

7.2.7 Stylování uživatelského rozhraní

Prezentační vrstva aplikace je postavena na nejnovější verzi utility-first frameworku Tailwind CSS 4. Architektura stylování plně využívá nativní integraci prostřednictvím Vite pluginu (@tailwindcss/vite). Tím odpadá dříve nutná údržba externího konfiguračního souboru. Klientská část kombinuje Tailwind třídy aplikované přímo v JSX s globálním souborem **index.css** a dedikovanými komponentními styly.

Globální proměnné a perceptuální barvy

Základní designový systém je definován v kořenovém souboru. Uživatelské rozhraní využívá moderní přístup k definici barevné palety pomocí barevného prostoru OKLCH (Oklab Lightness, Chroma, Hue). Tento barevný prostor oproti tradičnímu RGB/HSL zajišťuje mnohem konzistentnější vnímání jasu na různých typech displejů. Pomocí direktivy @theme inline jsou tyto proměnné přímo injektovány do konfiguračního stromu Tailwindu:

Výpis 18 Definice motivu a CSS proměnných v Tailwind CSS

```
@import 'tailwindcss';
@import 'tw-animate-css';

/* Podpora pro tmavý režim (Dark Mode) */
@custom-variant dark (&:is(.dark *));

:root {
  --background: oklch(0.98 0.005 260);
  --foreground: oklch(0.17 0.03 260);
  --primary: oklch(0.25 0.06 260);
  --accent: oklch(0.72 0.15 195);
  --radius: 0.75rem;
}

/* Integrace vlastních proměnných do Tailwind theme */
@theme inline {
  --font-sans: 'Inter', ui-sans-serif, system-ui, sans-serif;
  --color-primary: var(--primary);
  --color-background: var(--background);
}

@layer base {
  body {
    @apply bg-background text-foreground antialiased;
  }
}
```

Utility-first přístup v komponentách

Vizuální pravidla jsou následně aplikována primárně přímo na elementech v komponentách. Následující ukázka z rozvržení aplikace ilustruje, jak Tailwind umožňuje skládat vysoce komplexní uživatelská rozhraní – včetně vrstvených radiálních gradientů (bg-[radial-gradient...]), absolutního pozicování a efektů rozostření pozadí (backdrop-blur-xl) – aniž by vývojář musel opustit kontext deklarativního React kódu:

Výpis 19 Stylování komponenty pomocí utility tříd Tailwind CSS

```
<section  className="relative  isolate  box-border  h-[calc(100svh-var(--tc-shell-offset,4rem))]
overflow-hidden bg-[#0c1f45] px-3 py-3 sm:px-4 sm:py-4 lg:px-8 lg:py-5">
  {/* Komplexní vrstvení pozadí pomocí CSS gradientů */}
  <div className="pointer-events-none absolute inset-0 -z-10" aria-hidden="true">
    <div                 className="absolute                 inset-0                 bg-[linear-
gradient(180deg,rgba(10,20,46,0.56)_0%,rgba(9,18,40,0.68)_58%,rgba(8,16,34,0.8)_100%)]" />
```

```
    <div                  className="absolute                  inset-0                  bg-[radial-
gradient(circle_at_22%_20%,rgba(124,92,255,0.18)_0%,rgba(124,92,255,0)_42%)]" />
  </div>

  {/* Kontejner s efektem matného skla (Glassmorphism) */}
  <div className="mx-auto h-full w-full max-w-7xl">
    <div className="flex h-full w-full overflow-hidden rounded-3xl border border-white/12 bg-slate-
950/45 backdrop-blur-xl ring-1 ring-inset ring-white/5 shadow-[0_28px_80px_rgba(2,6,23,0.62)]">
      {/* Obsah aplikace */}
    </div>
  </div>
</section>
```

Tento přístup vede k vytvoření vizuálně bohatého rozhraní (využívajícího prvků tzv. Glassmorphismu), přičemž si kód zachovává vysokou čitelnost a snadnou udržovatelnost.

7.3 Implementace databázové vrstvy

Jako hlavní datové úložiště backendu slouží NoSQL databáze MongoDB. Její dokumentově orientovaný model byl zvolen pro přirozenou podporu proměnlivé struktury časových kapslí, které mohou v jednom záznamu obsahovat text, vnořená média či geolokaci bez nutnosti složité normalizace.

Pro práci s databází využívá backend framework Spring Data MongoDB, který kombinuje abstrakci repozitářů pro běžné CRUD operace a nástroj MongoTemplate pro specifičtější atomární zásahy.

Mapování entit a indexace

Transformace Java objektů do databázových dokumentů probíhá pomocí anotací. Flexibilitu tohoto přístupu dokonale ilustruje entita Capsule, sdružující různorodé datové typy v jediném dokumentu:

Výpis 20 Mapování entity Capsule na dokument MongoDB

```java
@Document(collection = "capsules")
public class Capsule {
    @Id
    private String id;

    @Field("ownerId")
    private ObjectId ownerId;

    @Field("media")
    private List<Media> media = new ArrayList<>();

    @Field("unlockAt")
    private Instant unlockAt;

    @Field("geoMarkerId")
    private ObjectId geoMarkerId;

    @Field("location")
    private GeoPoint location;

    @Field("tags")
    private List<String> tags = new ArrayList<>();
}
```

Pro zajištění datové integrity a optimalizaci výkonu čtení systém extenzivně využívá indexy. Například pole pro e-mail v entitě uživatele nese anotaci @Indexed(unique = true). Tato anotace garantuje jedinečnost záznamu a dramaticky zrychluje autentizační proces.

Deklarativní repozitáře (MongoRepository)

Běžné databázové interakce obstarávají rozhraní dědící z MongoRepository, kde framework automaticky generuje dotazy na základě deklarativních názvů metod. Následující ukázka z CapsuleRepository demonstruje vyhledávání a implementaci tzv. "Soft Delete" (podmínka DeletedAtIsNull):

Výpis 21 Deklarativní dotazy v rozhraní CapsuleRepository

```
public interface CapsuleRepository extends MongoRepository<Capsule, String> {
    List<Capsule> findByOwnerIdAndDeletedAtIsNullOrderByCreatedAtDesc(ObjectId ownerId);
    Optional<Capsule> findByIdAndOwnerIdAndDeletedAtIsNull(String id, ObjectId ownerId);
    Optional<Capsule> findByIdAndDeletedAtIsNull(String id);
    List<Capsule> findByVisibilityAndDeletedAtIsNullOrderByCreatedAtDesc(String visibility);
    long countByOwnerIdAndDeletedAtIsNull(ObjectId ownerId);
}
```

Atomární operace přes MongoTemplate

V kritických částech systému, kde je nutné předejít problémům se souběhem (race conditions), se využívá nízkoúrovňový MongoTemplate. Příkladem je plánovač CapsuleUnlockScheduler, který musí kapsli odemknout striktně atomárně:

Výpis 22 Atomická aktualizace stavu kapsle pomocí MongoTemplate

```
UpdateResult result = mongoTemplate.updateFirst(
        new Query(
                Criteria.where("_id").is(capsule.getId())
                        .and("ownerId").is(capsule.getOwnerId())
                        .and("status").is(CapsuleStatus.SEALED.getValue())
                        .and("deletedAt").is(null)
                        .and("unlockAt").lte(now)
        ),
        new Update()
                .set("status", CapsuleStatus.OPENED.getValue())
                .set("openedAt", now)
                .set("updatedAt", now),
        Capsule.class
);
```

Volání updateFirst zajistí úpravu dokumentu jen a pouze tehdy, pokud plně vyhovuje všem bezpečnostním a časovým podmínkám v objektu Query. Tím je zajištěna konzistence stavu dat.

7.4 Infrastruktura a nasazení

Nasazení aplikace TimeCapsule je rozděleno do samostatných kontejnerů, aby bylo možné oddělit backend, frontend, Nginx a databázovou část. Systém využívá kontejnerizaci prostřednictvím technologie Docker a je rozdělen do několika mikroslužeb spravovaných nástrojem Docker Compose. Infrastruktura striktně odděluje aplikační vrstvu od databázové, což umožňuje nezávislé škálování a údržbu jednotlivých komponent.

Přehled nasazení jednotlivých služeb a jejich síťového propojení je uveden v diagramu nasazení v příloze D.

7.4.1 Docker a Docker Compose

Srdcem aplikační vrstvy je soubor deploy/docker-compose.yml, který orchestruje tři klíčové služby: backend, frontend a nginx.

Optimalizace obrazů (Multi-stage builds)

Jak frontendový (Bun/Vite), tak backendový (Spring Boot) repozitář využívá techniku vícestupňového sestavování (multi-stage builds) v souborech Dockerfile. Backendový kontejner například definuje fázi dev pro lokální vývoj se Spring Boot DevTools, fázi build pro kompilaci produkčního JAR archivu a finální fázi runtime, která obsahuje pouze minimální JRE prostředí. Produkční obraz je díky tomu menší a neobsahuje vývojové nástroje, které nejsou při běhu aplikace potřeba. Výsledkem je významné zvýšení bezpečnosti, zejména díky zmenšení útočné plochy.

Pro urychlení lokálního vývoje využívá Docker Compose systém tzv. "bind mounts" (např. připojení složek src a target/classes) v kombinaci s pojmenovanými svazky (volumes) jako m2-cache pro závislosti Mavenu a node-mod-cache pro balíčky frontendu.

7.4.2 Konfigurace Nginx

Služba nginx funguje jako jediný veřejný vstupní bod (Single Point of Entry) do aplikace. Publikuje porty 80 a 443 na hostitelském systému a primárně plní roli reverzní proxy a terminátoru HTTPS spojení.

Zabezpečení a směrování

Veškerý nešifrovaný HTTP provoz (port 80) je automaticky přesměrován (stavový kód 301) na zabezpečenou variantu HTTPS (port 443) s podporou protokolu HTTP/2. Pokud v lokálním prostředí chybí produkční certifikáty, kontejner Nginxu obsahuje startovací skript, který dynamicky vygeneruje self-signed certifikát a tím simuluje bezpečné produkční prostředí již během vývoje.

Směrování provozu uvnitř Docker sítě je definováno následovně:

- **REST API a autentizace:** Cesty /api/, /oauth2/ a /login/oauth2/ jsou proxy serverem předávány na upstream cíl backend**:**8080.
- **Real-time komunikace:** Požadavky na cestu /ws/ jsou přesměrovány na backend s explicitním nastavením HTTP hlaviček Upgrade a Connection. Toto nastavení je kritické pro úspěšné navázání dlouhodobého WebSocket spojení.
- **Statický obsah:** Systém obsahuje proxy pravidla pro statické soubory (/static/tags/, /uploads/) a aplikuje direktivy pro mezipaměť prohlížeče (Cache-Control, expires). Tím se snižuje zátěž backendového serveru.
- **Frontend (SPA routing):** Veškerý zbývající neidentifikovaný provoz je směrován na upstream cíl frontend:5173.

7.4.3 Propojení služeb a sítě

Síťová topologie aplikace je postavena na izolaci. Aplikační stack (backend, frontend, Nginx) využívá pro vzájemnou komunikaci dedikovanou interní síť app-net.

Databázová vrstva je provozována ve zcela izolovaném Docker Compose stacku (database/docker-compose.yml), který tvoří komplexní shardovaný cluster MongoDB (zahrnující konfigurační servery a shard replica sety) s vlastní externí sítí database_default. Kontejner backendu je záměrně připojen do obou těchto sítí současně – přijímá požadavky z uživatelského rozhraní přes app-net a následně komunikuje s routerem clusteru (router01:27017) přes database_default.

Závislost služeb je řízena pomocí funkcí Docker health check. Nginx například nezačne obsluhovat veřejné požadavky dříve, než backend úspěšně odpoví na kontrolní endpoint (http://localhost:8080/ws/info). Tento mechanismus zabraňuje výpadkům při spouštění celého ekosystému.

7.4.4 Lokální a produkční spuštění

Lokální spuštění aplikace probíhá ve dvou krocích. Vývojář nejprve inicializuje sdílený databázový stack a teprve poté aplikační vrstvu:

Výpis 23 Postup lokálního spuštění databázové a aplikační vrstvy

```
# 1. Spuštění databázového clusteru (vytvoření sítě database_default)
cd database
docker compose up -d --build

# 2. Spuštění aplikační vrstvy
cd ../deploy
docker compose up -d –build
```

Konfigurační údaje, například OAuth2 klíče nebo SMTP hesla, nejsou uloženy přímo ve zdrojovém kódu. Aplikace je načítá ze souborů .env, na které odkazují jednotlivé Docker Compose soubory. Pro produkční provoz by bylo nutné doplnit platné TLS certifikáty a nahradit lokální ukládání uživatelských médií vhodným síťovým nebo objektovým úložištěm.

7.5 Testování backendové části

Backendová část aplikace byla ověřena sadou automatizovaných testů. Testy jsou umístěny v adresáři backend/src/test/java a jsou napsány pomocí JUnit 5, Mockito, Spring Boot Test a MockMvc. Testovací sada se spouští příkazem mvn -f backend/pom.xml test. Při posledním spuštění proběhlo celkem 27 testů bez chyby.

Testy nejsou zaměřeny pouze na spuštění aplikace. Ověřují také vybrané části autentizace, práci s časovými kapslemi, validaci vstupních dat, autorizaci, nastavení cookies, auditní logování a chatovou komunikaci.

7.5.1 Spuštění aplikace a testování webové vrstvy

Základní test TimeCapsuleApplicationTests kontroluje, zda se Spring Boot aplikace spustí a zda se správně načte aplikační kontext. Tento test sám o sobě neověřuje business logiku, ale dokáže odhalit chybu v konfiguraci nebo špatně zapojené závislosti.

Pro testování controllerů je použito MockMvc. Díky tomu lze ověřovat HTTP odpovědi bez spuštění skutečného serveru. Například AuthenticationControllerTest kontroluje refresh token flow a ověřuje, že access token a refresh token nejsou vraceny v JSON odpovědi, ale zůstávají řešeny přes cookies.

7.5.2 Testování API pro časové kapsle

API pro práci s kapslemi je testováno ve třídě CapsuleControllerTest. Testy kontrolují jak úspěšné scénáře, tak chybové stavy. Ověřuje se například získání kapsle pro editaci, vytvoření draft kapsle, aktualizace kapsle, odmítnutí neplatných dat a odmítnutí požadavku při nedostatečném oprávnění.

Ukázka ve výpisu 24 kontroluje situaci, kdy uživatel odešle prázdný titulek. V takovém případě má controller vrátit chybu validace a servisní vrstva se nemá vůbec zavolat.

Test ověřuje, že neplatná data se zastaví ještě před vstupem do servisní logiky. Neplatný požadavek se tak nedostane do servisní vrstvy.

Výpis 24 Test validace vstupu při úpravě kapsle

```
payload.setTitle(" ");

mockMvc.perform(put("/api/capsules/{id}", CAPSULE_ID)
        .principal(mockAuth())
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(payload)))
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.code").value("validation_error"));

verify(capsuleService, never())
    .update(eq(CAPSULE_ID), eq("user"), any(UpdateCapsuleRequest.class));
```

7.5.3 Testování servisní logiky kapslí

Servisní logika kapslí je testována ve třídě CapsuleServiceUpdateTest. Tyto testy se soustředí na pravidla, která nejsou vidět přímo na úrovni HTTP požadavku. Patří sem vlastnictví kapsle, práce se stavy draft, sealed a opened, kontrola časových údajů a pravidla pro sdílené kapsle.

Jedním z testovaných scénářů je pokus běžného uživatele upravit cizí kapsli. Servis má takový požadavek odmítnout a nesmí provést zápis do databáze.

Výpis 25 Test zákazu úpravy cizí kapsle běžným uživatelem

```
assertThrows(SecurityException.class,
    () -> capsuleService.update(CAPSULE_ID, OTHER_USER_ID, validRequest()));

verify(mongoTemplate, never())
    .updateFirst(any(Query.class), any(Update.class), eq(Capsule.class));
```

Test zde nekontroluje jen samotnou výjimku. Ověřuje také to, že po odmítnutí požadavku neproběhne žádná databázová aktualizace. To je podstatné pro ochranu uživatelských dat.

7.5.4 Testování autentizačních cookies

Samostatně je testována služba AuthCookieService, která vytváří a maže autentizační cookies. Testy ověřují nastavení atributů HttpOnly, SameSite, Secure, Path a Max-Age. Tyto atributy ovlivňují, jak se cookies chovají při komunikaci mezi frontendem a backendem a zda k nim může přistupovat JavaScript na straně klienta.

Ve výpisu 26 je ukázka testu, který kontroluje nastavení cookies pro cross-site HTTPS požadavek.

Výpis 26 Test bezpečnostních atributů autentizačních cookies

```
authCookieService.writeAuthCookies(request, response, tokens);

List<String> cookies = response.getHeaders("Set-Cookie");

assertTrue(cookies.get(0).contains("SameSite=None"));
assertTrue(cookies.get(0).contains("Secure"));
assertTrue(cookies.get(1).contains("Path=/api/auth"));
```

Tento test chrání chování, na kterém stojí přihlášení uživatele. Pokud by se nastavení cookies změnilo chybně, mohlo by dojít buď k problémům s přihlášením, nebo ke zhoršení bezpečnosti.

7.5.5 Testování chatu a auditního logování

Testy pokrývají také části aplikace, které souvisejí s komunikací a provozní kontrolou. ChatServiceTest ověřuje odeslání textové zprávy, zprávy s médiem a odmítnutí prázdné zprávy. U prázdné zprávy se kontroluje, že se neuloží do databáze, neodešle přes WebSocket a nevytvoří se e-mailová notifikace.

Výpis 27 Test odmítnutí prázdné chatové zprávy

```
assertThrows(IllegalArgumentException.class, () ->
    chatService.sendMessage(SENDER_ID, PEER_ID, " ", null, null, null, null)
);

verify(chatMessageRepository, never()).save(any(ChatMessage.class));
verify(messagingTemplate, never()).convertAndSendToUser(any(), any(), any());
verify(emailService, never()).enqueueChatDigest(any(), any(), any());
```

Auditní logování je testováno ve třídě ApiAuditInterceptorTest. Testy ověřují, že úspěšné mutační požadavky vytvářejí auditní záznam, zatímco čtecí požadavky nebo neúspěšné mutace se nelogují jako provedené akce. Auditní záznamy tak zůstávají přehlednější a lépe odpovídají skutečným změnám v systému.

7.5.6 Zhodnocení testování

Aktuální testovací sada pokrývá hlavní části backendové implementace. Testuje spuštění aplikace, autentizaci, refresh tokeny, cookies, práci s kapslemi, validaci vstupů, autorizační pravidla, chat a auditní logování.

Testy jsou převážně unit a controller testy. Testy se proto spouštějí rychle, protože nepotřebují skutečný server ani databázi. Zároveň je potřeba počítat s tím, že tato sada nepokrývá celý systém od uživatelského rozhraní až po databázi. Jako další krok by dávalo smysl doplnit integrační testy s testovací MongoDB databází a frontendové end-to-end testy hlavních scénářů, například registraci, vytvoření kapsle, její odemčení a zobrazení notifikace.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout a vytvořit webovou aplikaci Time Capsule pro ukládání digitálního obsahu, který se zpřístupní až v nastaveném čase. Tento cíl se podařilo splnit. Výsledkem práce je aplikace, ve které může uživatel vytvořit časovou kapsli, nastavit její viditelnost, doplnit textový nebo mediální obsah a nechat systém, aby kapsli otevřel podle zadaného času.

V práci byl nejprve popsán teoretický základ webových aplikací, architektury, databází, komunikace v reálném čase a bezpečnostních principů. Na tuto část navázala analýza systému Time Capsule, kde byly určeny hlavní funkce, uživatelské role, požadavky a omezení řešení. Z analýzy pak vycházel návrh architektury, datového modelu, bezpečnostních pravidel a logiky spojené s životním cyklem kapsle.

Praktická část práce se zaměřila na implementaci jednotlivých částí aplikace. Backend je vytvořen ve frameworku Spring Boot. Obsahuje logiku pro práci s kapslemi, přihlášení uživatelů, kontrolu oprávnění, plánované otevírání obsahu, notifikace a komunikaci přes WebSocket. Frontend je napsaný v Reactu a TypeScriptu. Uživatel přes něj spravuje kapsle, upravuje profil, pracuje s mapou, používá chat a přistupuje k dalším částem aplikace. Pro ukládání dat byla použita databáze MongoDB. Spuštění aplikace je připraveno pomocí Dockeru a reverzní proxy Nginx.

Součástí práce bylo také ověření backendové části pomocí automatizovaných testů. Testy se zaměřují hlavně na autentizaci, práci s cookies, REST API pro kapsle, validaci vstupních dat, autorizační pravidla, servisní logiku, chatovou komunikaci a auditní logování. Tím se ověřily části systému, u kterých by chyba mohla ovlivnit bezpečnost, uživatelská data nebo běžné používání aplikace.

Aplikace splňuje hlavní požadavky zadání, ale některé části by bylo vhodné dále rozšířit. Ukládání mediálních souborů je nyní řešeno lokálně, což je vhodné hlavně pro vývoj a testování. Pro produkční provoz by dávalo smysl přesunout média do cloudového objektového úložiště. Dalším krokem by mohlo být rozšíření testování o integrační testy s testovací databází a o end-to-end testy hlavních uživatelských scénářů. Do budoucna by bylo možné vytvořit také mobilní aplikaci, která by lépe využila práci s geolokací a notifikacemi.

POUŽITÁ LITERATURA

[1] CARRASCO, S., SANZ-REQUENA, R., MORATAL, D., BODÍ, V. a RIETA, J. J. Sharing Acute Myocardial Infarction Databases through the Internet with MySQL and PHP: A Web-Accessible Database for Clinical Research Networks. In: Computers in Cardiology 2007. 2007, s. 605–608. DOI 10.1109/CIC.2007.4745558. Dostupné z: https://www.researchgate.net/publication/216554589_Sharing_Acute_Myocardial_Infarction_Databases_through_the_Internet_with_MySQL_and_PHP_A_Web-Accessible_Database_for_Clinical_Research_Networks

[2] MDN Web Docs. Client-server overview [online]. [cit. 2026-04-15]. Dostupné z: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview

[3] MDN Web Docs. HTTP messages [online]. [cit. 2026-04-15]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages

[4] FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures [online]. Irvine: University of California, 2000 [cit. 2026-04-15]. Dostupné z: https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf

[5] FETTE, Ian a MELNIKOV, Alexey. The WebSocket Protocol. RFC 6455 [online]. Fremont: RFC Editor, 2011 [cit. 2026-04-15]. Dostupné z: https://www.rfc-editor.org/rfc/rfc6455.html

[6] Microsoft Learn. Architecture styles [online]. [cit. 2026-04-15]. Dostupné z: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

[7] Microsoft Learn. N-tier architecture style [online]. [cit. 2026-04-15]. Dostupné z: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier

[8] ALGORITHM ALCHEMIST. Layered Architecture Pattern: Understanding Software Architecture Patterns [online]. GoPenAI, 20. 9. 2024 [cit. 15. 4. 2026]. Dostupné z: https://blog.gopenai.com/layered-architecture-pattern-understanding-simple-software-designs-f50e8256d5cc

[9] FOWLER, Martin. Monolith First [online]. 3. 6. 2015 [cit. 15. 4. 2026]. Dostupné z: https://martinfowler.com/bliki/MonolithFirst.html

[10] Microsoft Learn. Microservices architecture style [online]. [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices

[11] BROWN, Simon. Modular monoliths [online]. [cit. 15. 4. 2026]. Dostupné z: https://static.simonbrown.je/modular-monoliths.pdf

[12] NODE.JS. About Node.js [online]. [cit. 15. 4. 2026]. Dostupné z: https://nodejs.org/en/about

[13] NESTJS. Documentation [online]. [cit. 15. 4. 2026]. Dostupné z: https://docs.nestjs.com/

[14] DJANGO SOFTWARE FOUNDATION. Django documentation [online]. Verze 6.0 [cit. 15. 4. 2026]. Dostupné z: https://docs.djangoproject.com/en/6.0/

[15] FASTAPI. FastAPI [online]. [cit. 15. 4. 2026]. Dostupné z: https://fastapi.tiangolo.com/

[16] SPRING. Spring Boot [online]. [cit. 15. 4. 2026]. Dostupné z: https://spring.io/projects/spring-boot/

[17] MICROSOFT LEARN. Develop ASP.NET Core MVC apps [online]. [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/develop-asp-net-core-mvc-apps

[18] THE GO PROGRAMMING LANGUAGE. Documentation [online]. [cit. 15. 4. 2026]. Dostupné z: https://go.dev/doc/

[19] THE GO PROGRAMMING LANGUAGE. Data Race Detector [online]. [cit. 15. 4. 2026]. Dostupné z: https://go.dev/doc/articles/race_detector

[20] REACT. React [online]. [cit. 15. 4. 2026]. Dostupné z: https://react.dev/

[21] VUE.JS. Introduction [online]. [cit. 15. 4. 2026]. Dostupné z: https://vuejs.org/guide/introduction

[22] ANGULAR. Angular [online]. [cit. 15. 4. 2026]. Dostupné z: https://angular.dev/

[23] NEXT.JS. Next.js Docs [online]. [cit. 15. 4. 2026]. Dostupné z: https://nextjs.org/docs

[24] FAIN, Yakov a MOISEEV, Anton. Exploring Modern Web Development. Shelter Island: Manning, 2019. ISBN 978-1-61729-534-9.

[25] SCIORE, Edward. Database design and implementation. Hoboken, N.J.: John Wiley, 2009. ISBN 978-0-471-75716-0.

[26] POSTGRESQL GLOBAL DEVELOPMENT GROUP. Table Basics [online]. [cit. 15. 4. 2026]. Dostupné z: https://www.postgresql.org/docs/current/ddl-basics.html

[27] POSTGRESQL GLOBAL DEVELOPMENT GROUP. Glossary [online]. [cit. 15. 4. 2026]. Dostupné z: https://www.postgresql.org/docs/current/glossary.html

[28] MONGODB. Documents [online]. [cit. 15. 4. 2026]. Dostupné z: https://www.mongodb.com/docs/manual/core/document/

[29] MONGODB. Data Modeling in MongoDB [online]. [cit. 15. 4. 2026]. Dostupné z: https://www.mongodb.com/docs/manual/data-modeling/

[30] REDIS. Key-Value Databases [online]. [cit. 15. 4. 2026]. Dostupné z: https://redis.io/nosql/key-value-databases/

[31] NEO4J. Graph database concepts [online]. [cit. 15. 4. 2026]. Dostupné z: https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/

[32] FOWLER, Martin. Polyglot Persistence [online]. 16. 11. 2011 [cit. 15. 4. 2026]. Dostupné z: https://martinfowler.com/bliki/PolyglotPersistence.html

[33] DOCKER. What is Docker? [online]. [cit. 15. 4. 2026]. Dostupné z: https://docs.docker.com/get-started/docker-overview/

[34] DOCKER. What is a container? [online]. [cit. 15. 4. 2026]. Dostupné z: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

[35] DOCKER. Docker Compose [online]. [cit. 15. 4. 2026]. Dostupné z: https://docs.docker.com/guides/docker-compose/

[36] OWASP. Authentication Cheat Sheet [online]. [cit. 15. 4. 2026]. Dostupné z: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[37] OWASP. Authorization Cheat Sheet [online]. [cit. 15. 4. 2026]. Dostupné z: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[38] JONES, Michael, BRADLEY, John a SAKIMURA, Nat. JSON Web Token (JWT). RFC 7519 [online]. Fremont: RFC Editor, 2015 [cit. 15. 4. 2026]. Dostupné z: https://www.rfc-editor.org/rfc/rfc7519.html

[39] HARDT, Dick. The OAuth 2.0 Authorization Framework. RFC 6749 [online]. Fremont: RFC Editor, 2012 [cit. 15. 4. 2026]. Dostupné z: https://www.rfc-editor.org/rfc/rfc6749.html

[40] MDN Web Docs. HTTPS [online]. [cit. 15. 4. 2026]. Dostupné z: https://developer.mozilla.org/en-US/docs/Glossary/HTTPS

[41] OWASP. Cross Site Scripting Prevention Cheat Sheet [online]. [cit. 15. 4. 2026]. Dostupné z: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[42] OWASP. Cross-Site Request Forgery Prevention Cheat Sheet [online]. [cit. 15. 4. 2026]. Dostupné z: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[43] MICROSOFT LEARN. *Overview of ASP.NET Core MVC* [online]. 17. 6. 2024 [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-10.0

[44] MICROSOFT. *ASP.NET MVC Overview* [online]. 15. 6. 2023 [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/asp-net-mvc-overview

[45] MICROSOFT LEARN. Design Patterns: Model View Presenter [online]. August 2006 [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/archive/msdn-magazine/2006/august/design-patterns-model-view-presenter

[46] MICROSOFT LEARN. Model-View-ViewModel (MVVM) [online]. 10. 9. 2024 [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm

[47] MICROSOFT LEARN. Data binding overview - WPF [online]. 6. 5. 2025 [cit. 15. 4. 2026]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/

[48] ANDROID DEVELOPERS. Compose UI Architecture [online]. [cit. 15. 4. 2026]. Dostupné z: https://developer.android.com/develop/ui/compose/architecture

[49] ORBIT MVI. MVI and Orbit [online]. [cit. 15. 4. 2026]. Dostupné z: https://orbit-mvi.org/Core/architecture/

SEZNAM PŘÍLOH

Příloha A: Obsah přiloženého ZIP archivu a struktura zdrojových kódů

Příloha B: UML sekvenční diagram automatického otevření kapsle

Příloha C: ER diagram datového modelu MongoDB

Příloha D: UML diagram nasazení aplikace

Příloha A: Obsah přiloženého ZIP archivu a struktura zdrojových kódů

Tato příloha obsahuje popis adresářové struktury přiloženého elektronického média (ZIP archivu) a detailní rozdělení zdrojových kódů backendové i frontendové části.

Obsah přiloženého ZIP archivu

Přiložený archiv obsahuje kompletní zdrojové kódy a konfigurační soubory potřebné pro spuštění aplikace.

/backend – Zdrojové kódy serverové části (Spring Boot, Java).

/frontend – Zdrojové kódy klientské části (React, TypeScript).

/database – Inicializační skripty a konfigurace pro MongoDB sharded cluster.

/deploy – Soubory docker-compose.yml a konfigurace Nginx pro nasazení.

README.md – Detailní návod k instalaci a spuštění celého systému.

Struktura backendové části

Následující strom ilustruje rozdělení klíčových tříd a komponent serverové části (složka com/oleksandrmytro/timecapsule):

```
com/oleksandrmytro/timecapsule
├── config/                 # Konfigurace bezpečnosti, JWT, OAuth2, WebSocket a plánovače
│   ├── SecurityConfiguration.java
│   ├── JwtAuthenticationFilter.java
│   ├── OAuth2AuthenticationSuccessHandler.java
│   ├── AuthCookieService.java
│   └── CapsuleUnlockScheduler.java
├── controllers/            # REST API koncové body pro komunikaci s klientem
│   ├── AuthenticationController.java
│   ├── CapsuleController.java
│   └── ChatController.java
├── services/               # Jádro byznys logiky, nezávislé na HTTP
│   ├── AuthenticationService.java
│   ├── CapsuleService.java
│   └── EmailService.java
├── repositories/           # Rozhraní pro přístup k datům (MongoDB)
│   ├── CapsuleRepository.java
│   └── UserRepository.java
├── models/                 # Databázové entity a výčtové typy (enums)
│   ├── Capsule.java
│   ├── User.java
│   └── enums/
├── dto/                    # Objekty pro přenos dat od klienta (Data Transfer Objects)
│   └── CreateCapsuleRequest.java
├── responses/              # Formátované odpovědi odesílané klientovi
│   └── CapsuleResponse.java
└── events/                 # Aplikační a real-time události
    └── CapsuleStatusEvent.java
```

Struktura frontendové části

Následující výpis ukazuje organizaci souborů v klientské aplikaci (složka frontend/src):

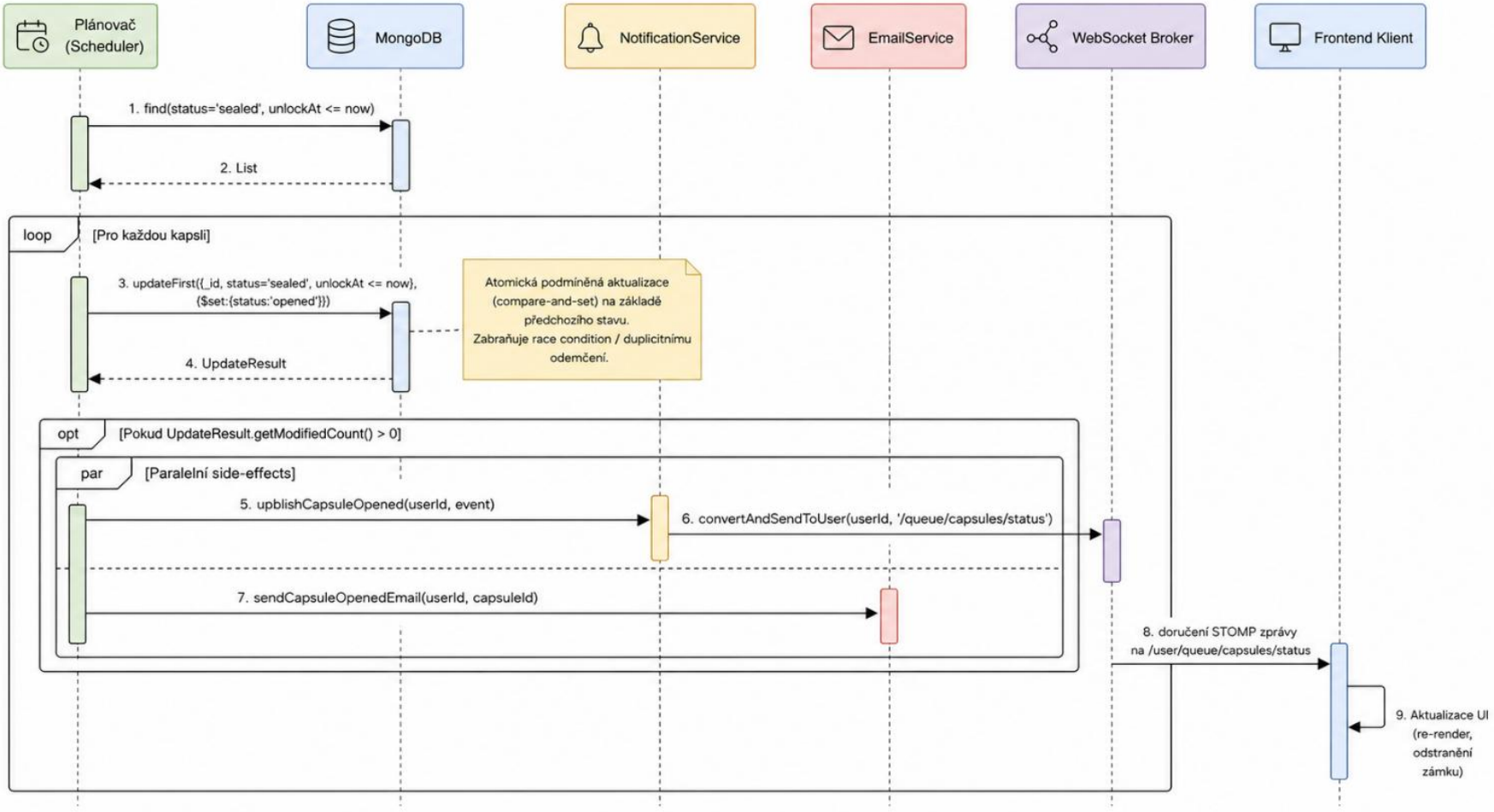
```
frontend/src
├── components/          # UI a funkční komponenty rozdělené podle domén
│   ├── auth/            # Login, registrace, ověření
│   ├── capsules/        # Vytváření, detail, mapa, kalendář kapslí
│   ├── chat/            # Rozhraní pro real-time chat
│   ├── ui/              # Znovupoužitelné atomické komponenty (tlačítka, vstupy)
│   └── ...              # admin, account, media, landing
├── services/            # Integrace externích API a WebSocketů
│   ├── api.ts           # Centrální HTTP klient a definice API volání
│   └── ws.ts            # Real-time STOMP klient pro WebSockety
├── lib/                 # Pomocné utility, typové definice a konstanty
│   ├── media-types.ts   # Definice podporovaných formátů médií
│   └── utils.ts         # Formátování dat, validace, pomocné funkce
├── App.tsx              # Hlavní orchestrátor, definice routingu a globálního stavu
└── main.tsx             # Vstupní bod aplikace, bootstrap Reactu
```

Struktura databázové části

Následující výpis ukazuje organizaci souborů pro inicializaci a konfiguraci distribuované databáze (složka database/):

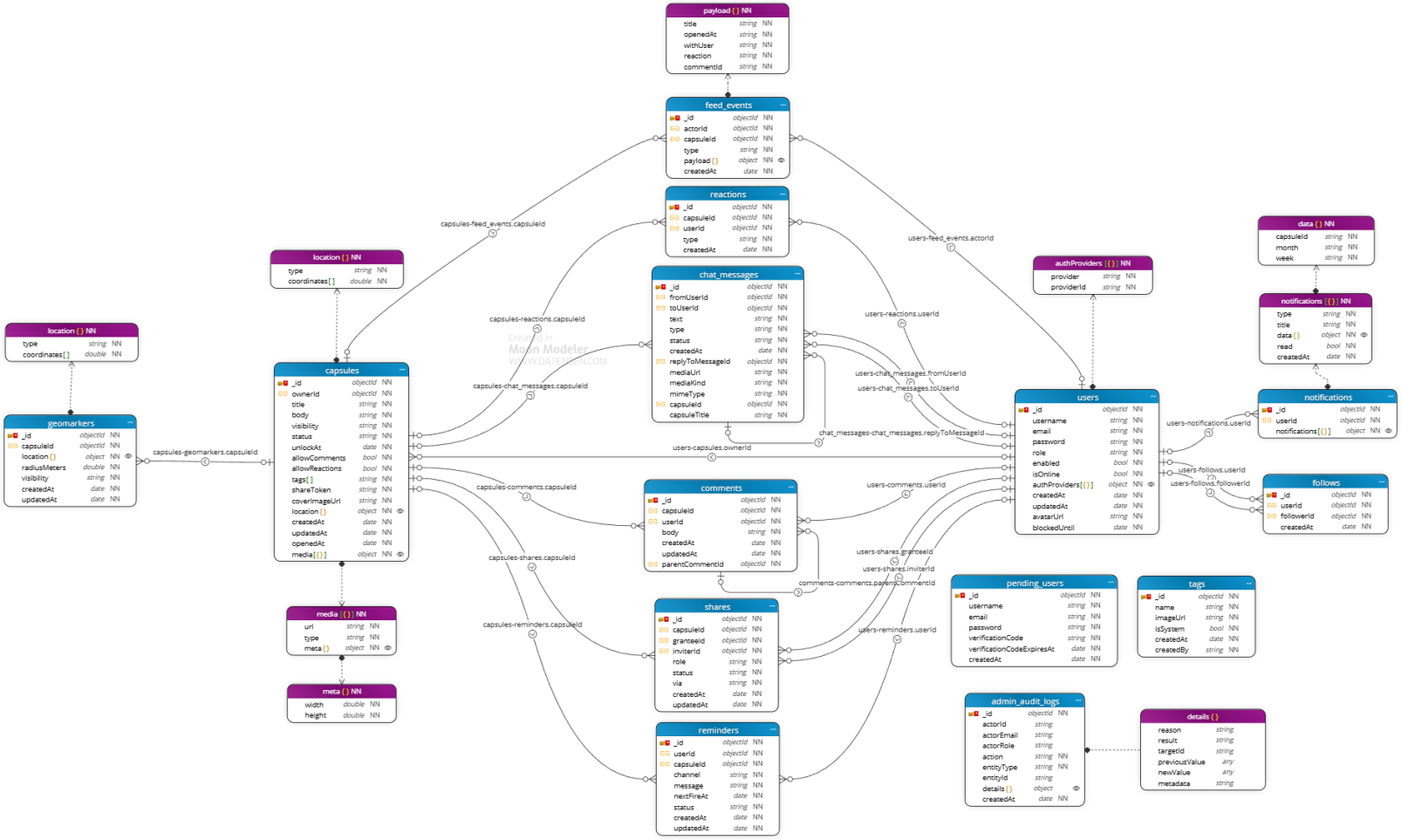
```
database/
├── data/                # Počáteční data (CSV) pro naplnění databáze (capsules.csv a další)
├── init-cluster/        # Konfigurace a Dockerfile pro inicializaci clusteru
├── keyfile/             # Soubory pro interní zabezpečení a autentizaci replik
├── router/              # Konfigurace pro databázový směrovač (mongos)
├── schemas/             # Databázová validační schémata pro kolekce (Capsule.js a další)
├── scripts/             # Inicializační a konfigurační skripty pro nastavení shardingu
├── .env                 # Proměnné prostředí pro databázi
└── docker-compose.yml   # Konfigurace pro spuštění MongoDB sharded clusteru
```

Příloha B: UML sekvenční diagram automatického otevření kapsle



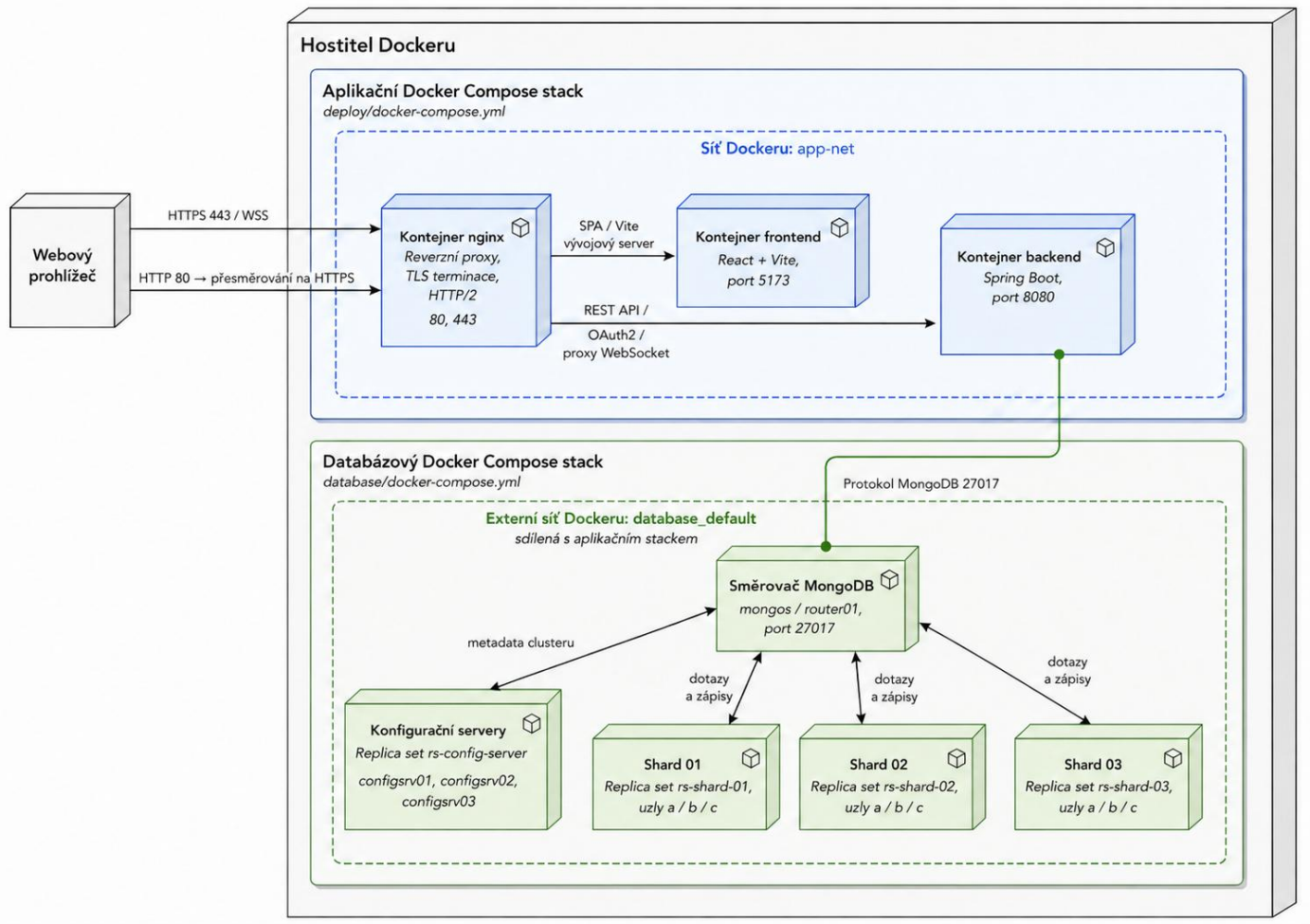
Obrázek 20 UML Sekvenční diagram – Asynchronní proces automatického otevření kapsle s ochranou proti souběhu
Zdroj: (vlastní zpracování s využitím umělé inteligence)

Příloha C: ER diagram datového modelu MongoDB



Obrázek 21 Entity-Relationship (ER) diagram datového modelu MongoDB
Zdroj: (vlastní zpracování s využitím softwarového nástroje pro návrh databází)

Příloha D: UML diagram nasazení aplikace



Obrázek 22 UML diagram nasazení a síťové architektury v prostředí Docker
Zdroj: (vlastní zpracování s využitím umělé inteligence)